

X3DH with Deniable Authentication without Trusted Third Parties

Stanislaw Jarecki[✉], Phillip Nazarian[✉], and Apurva Rai[✉]

University of California, Irvine, USA {sjarecki,pnazaria,apurvr1}@uci.edu

Abstract. Message Authentication in the Short Authenticated String model (SAS-MA) allows Alice and Bob to establish a secure channel without trust in any third party, as long as they can exchange *short authenticated strings*, e.g. 20 bits. In a recent paper, Gu et al. [17] showed a SAS-MA scheme based on Verifiable Random Functions (VRF), which can utilize the ephemeral keys sent in the X3DH Authenticated Key Exchange (AKE), allowing for extending X3DH to SAS-MA with minimal round complexity and no changes to X3DH key distribution. X3DH is used in many messaging apps, including WhatsApp and Signal, and a SAS-MA extension of X3DH would allow app users to authenticate their connections without trust in PKI or the app’s Key Distribution Center (KDC), as long as they can exchange *short authenticated strings* (SAS), using out of band authenticated channels.

However, a major motivation behind using X3DH as an AKE is its *deniability* property, i.e. that an X3DH transcript cannot serve as a proof that either Alice or Bob established a secure connection with each other. The VRF-based SAS-MA extension of X3DH of [17] violates deniability, essentially because a VRF is a signature. We show an alternative SAS-MA scheme which offers the same ease of integration with X3DH as the VRF-based SAS-MA of [17], but it (almost) maintains the deniability of X3DH. The proposal is based on a *private* VRF (PVRF), which allows only ‘designated-verifier’ verification of correctness. We show a low-cost PVRF variant of ECVRF, and we show that X3DH extended by our PVRF-based SAS-MA adds human-centric no-trust-in-KDC authentication to X3DH while preserving the deniability properties of X3DH.

1 Introduction

Authenticated Key Exchange in the SAS model. Message Authentication (MA) in a Short Authenticated String model (SAS), introduced by Serge Vaudenay [34], allows two parties to (mutually) authenticate an arbitrary long message if the participants can exchange and compare *short authenticated strings*, e.g. of $d = 20$ bits. This form of authentication is *human centric*: short bitstrings can be encoded in user-friendly formats, e.g. as 6-digit decimal strings ($d = 20$), 6-character alphanumeric strings ($d = 30$), or 3-word English word sequences ($d = 42$). If Alice and Bob meet face-to-face then they can exchange these short

strings to establish security. Moreover, if the participants can trust the authenticity of each other’s voices or video feeds, they can also exchange short messages over remote connections, although attacks on voice-based authentication using audio slicing and speech synthesis tools [29], show that remote voice-based authentication can only serve as a secondary authentication factor. Since authenticating a Key Exchange transcript implies an *Authenticated Key Exchange* (AKE), SAS-MA implies SAS-AKE, and allows parties to establish secure communication channels without pre-shared keys, passwords, or trust in third parties, e.g. a Key Certification Authority or a Key Distribution Service (KDC).

To see that achieving a SAS-MA scheme is not trivial, note that if the SAS string is a d -bit hash of the message, $\text{sas} = H(m)$, then in time $2^{d/2}$ the attacker can find m, m' s.t. $H(m) = H(m')$, and if the sender adds a nonce r to the message then a man-in-the-middle attacker can use a precomputed table of size 2^d to find r' s.t. $H(r, m) = H(r', m')$. By contrast, a secure SAS-MA [34] ensures that if the two parties hold different messages $m \neq m'$ then even a man-in-the-middle attacker can make the verifier accept with at most 2^{-d} probability.¹ A series of works [15,34,22,23] showed SAS-MA schemes that use 3 protocol flows, which is minimal [34], and non-malleable commitments, achieving message *cross-authentication* (MCA) [24], i.e. authentication of both the initiator’s and the responder’s messages where the latter is not known beforehand.² Using such SAS-MCA with any 2-flow Key Exchange (KE), notably Diffie-Hellman [10], forms a 3-flow Authenticated Key Exchange in the SAS model (SAS-AKE).

X3DH and the nature of authentication it provides. An AKE scheme used to implement secure channels in several messaging apps, including WhatsApp and Signal, is X3DH [5], a variant of ‘triple Diffie-Hellman’ key exchange similar to HMQV [21]. Messaging apps use X3DH because of its several very attractive features: It is light, with only 4-5 exponentiations per party, it is as secure as Diffie-Hellman, and after the initiator retrieves the target counterparty’s keys from the KDC administered by the app, a secure channel with the counterparty is established with a single protocol flow, allowing the initiator to encrypt/authenticate its first payload already with that single AKE flow.

However, authentication provided by an AKE like X3DH relies on trust that the KDC supplies each protocol party with the correct public key of their intended counterparty. One attempt to assure users that the KDC is providing correct keys are *Key Transparency* (KT) schemes [25,26], but KT schemes still require the user to trust the current hash of the KT data-structure, and to trust public auditors to verify the proofs stored in the logs of key dictionary updates.

¹ This is optimal because an attacker that follows the protocol still has 2^{-d} chance that the d -bit string expected by the verifier matches the one produced by the prover.

² The intuition for a 3-flow *uni-directional* SAS-MA scheme on which protocols of [15,34,22,23] are based, is that it is a generalization of a two-party coin-toss, i.e. the prover commits to (sas_0, m) for a random d -bit nonce sas_0 , the receiver contributes a random d -bit nonce sas_1 , and a short string $\text{sas} = \text{sas}_0 \oplus \text{sas}_1$ is accepted by the receiver if the first-round commitment is opened to (sas_0, m) .

SAS-AKE extension of X3DH. In a recent paper, Gu et al. [17] showed a SAS-MA scheme based on Verifiable Random Functions (VRF) [27], denoted VRF-SAS, with a novel property that VRF keys can be re-used across multiple instances, and after the initial VRF key distribution each VRF-SAS instance takes *only 2 protocol flows*. Further, VRF-SAS instantiated with ECVRF [16] can be integrated with X3DH so that the ECVRF keys used in VRF-SAS are the keys already present in X3DH, and VRF-SAS adds only 1 protocol flow to the X3DH handshake, which is minimal for bidirectional authentication in the SAS model. Consequently, with no changes to the X3DH key distribution process, and at low communication and computation costs, X3DH extended with VRF-SAS would allow the users of a messaging app to end-to-end authenticate the X3DH-established channel without trust in a third party, including the KDC.

Deniability of X3DH. However, the above list of X3DH benefits omits another motivating factor behind its choice in messaging apps, which is that it is *deniable*, i.e. that X3DH transcripts cannot serve as proofs, used to convince some third party, that an X3DH participant established a secure channel with its counterparty. The notion of Deniable AKE builds on deniable authentication of Dwork, Naor, and Sahai [12]. An AKE is deniable if Alice who intends to run an AKE with Bob should not worry about anyone, including the potentially dishonest Bob, acquiring a proof that could convince a third party distinguisher, Charlie, that Alice intended to execute an AKE with Bob. More technically, an AKE is deniable if any information an attacker can learn from an AKE interaction with Alice can be *simulated* solely from the attacker’s own information.

Deniability of authentication and AKE has been studied in many works, e.g. [12,9,14], with several considering specifically the deniability of X3DH and related protocols like HMQV, 3DH, or PQXDH, e.g. [33,13,20]. The strongest variant of this notion is *online* deniability, considered e.g. by [11,31,32], where the distinguisher Charlie can ask Bob to use Charlie-specified messages in an interaction with Alice. The work of [11] showed a 3-flow online deniable AKE (in the semi-adaptive setting where the attacker can compromise either party but not during the protocol execution) using non-committing encryption and *dual-receiver encryption*, where ciphertexts can be decrypted by either of two public keys. This deniable AKE was optimized in the works of [31,32], adding ring signatures to the above tools, who showed a 2-flow scheme that matches the communication pattern of X3DH, i.e. the first flow that can be uploaded to the KDC as a *pre-key bundle* (see more on X3DH mechanics below), which achieves full on-line deniability for one party and a limited form of deniability for the other. X3DH is not online deniable: If Charlie asks Bob to use Charlie’s keys as Bob’s ephemeral DH key contribution, then Bob cannot compute the resulting X3DH key, and thus cannot simulate an interaction with an honest Alice. However, *off-line deniability*, where the distinguisher sees only Bob’s outputs after an interaction with Alice, was shown for X3DH and related protocols even against active attackers, in the Generic Group Model (GGM) or under a ‘knowledge of exponent’ assumption implied by the Algebraic Group Model (AGM) [33,13,20].

VRF-based SAS-AKE extension of X3DH breaks deniability. The SAS-AKE extension of X3DH via the VRF-SAS scheme of Gu et al. [17] preserves X3DH security but it violates its deniability properties. When VRF-SAS is integrated with X3DH, the short d -bit SAS token to be authenticated between Alice and Bob is derived from a VRF computed by the X3DH responder on the interaction transcript, using either her semi-static or one-time keys. (In addition, in VRF-SAS of [17] both parties use a VRF, but in the 2-party context the initiator’s VRF can be replaced by a nonce.) A verifiable random function is a pseudorandom function with a non-interactive proof (NIZK) that $y = \text{PRF}_k(x)$ is computed correctly given a public key vk which commits to k . VRF outputs computed on AKE transcripts yield secure SAS tokens because honest party’s VRF outputs are unpredictable, but any key vk' which the attacker injects into the protocol binds the attacker to a SAS contribution it must provide on any AKE transcript. However, the NIZK provided by a VRF is effectively a signature on the transcript, which breaks deniability for the responder.

To see why VRF creates a deniability break in X3DH let us recall X3DH. In X3DH each party holds a long-term *identity key*, and it deposits this key at the KDC together with a *pre-key bundle* which includes a *semi-static key* and a batch of *one-time keys*, all of which are Diffie-Hellman KE contributions. The initiator Alice retrieves Bob’s identity key, semi-static key, and a single unused one-time key if Bob’s KDC-held bundle still includes them (the KDC erases each one-time key it sends out). The initiator then sends to the receiver a random *ephemeral key*, and the protocol then forks as follows: If Alice holds Bob’s one-time key then the session key is derived by hashing *four* DH values which combine two Alice’s DH contributions, her identity and ephemeral keys, with three Bob’s DH contributions, his identity, semi-static, and one-time keys. If Bob’s one-time keys run out, a case known as a *last resort*, then the protocol defaults to 3DH where Bob’s semi-static key plays the role of his ephemeral DH contribution.

In the proposed X3DH and VRF-SAS integration [17], the responder issues a VRF using a one-time key in the default case, and a semi-static key in the last resort case. In the latter the responder loses its deniability unconditionally because its semi-static key can be retrieved from the KDC, signed by its identity key, but in the former case deniability is also lost if the distinguisher gains access to the KDC and sees the responder’s pre-key bundle used in this instance.

Our Contribution: X3DH with SAS authentication *and* deniability. The deniability violation of X3DH extended by VRF-SAS of [17] comes from the use of VRFs which are publicly verifiable, like signatures. In this work we show a 2-party SAS-MA protocol which follows the VRF-based logic of the SAS-MA of [17], but replaces a VRF with a *Privately Verifiable Random Function* (PVRF), which we introduce. Using our PVRF-based SAS-MA protocol denoted PVRF-SAS, for the SAS-AKE extension of X3DH, yields an X3DH extension that maintains the same efficiency and ease of adoption properties as [17] but in addition it (almost) maintains the same deniability properties as X3DH itself.

PVRF is an interactive ‘designated verifier’ VRF counterpart, involving a prover holding a PRF key k and a verifier holding the key commitment vk : The

verifier first creates a one-time *verification token* vt together with the private verification state vt_s . The prover can then issue a proof π that PRF output $y = \text{PRF}_k(x)$ is computed correctly under a key committed in vk , but the proof will be targeted to a specific token vt , and verifiable only by the party who holds the corresponding verification state vt_s . The proof system is sound, i.e. a cheating prover cannot create a proof that verifies on the wrong statement for an honestly created verification token vt . However, interaction with the prover is straight-line simulatable, and thus deniable, because tuple (vt, vt_s, π) can be efficiently sampled on the statement (vk, x, y) without the prover's witness k .

To keep the SAS-MA protocol at minimum round complexity we observe that the 2-party version of the VRF-SAS scheme of [17] remains secure if the initiator's SAS contribution is not computed via a VRF and is replaced by a nonce, and the SAS contribution of only the second party, the responder, is a verifiable PRF on the authenticated message. (The freedom this gives to the first party is not a threat, since it must decide its SAS contribution before the second party reveals its contribution.) Consequently, replacing the responder's VRF with a 2-flow PVRF keeps the protocol within 2 flows, which are minimal for the SAS model bidirectional authentication. We call the resulting SAS-MA scheme PVRF-SAS.

Key Building Block: Efficient UC PVRF from ECVRF and SPHF's.

We model PVRF in a Universally Composable (UC) model [4]. We base the UC PVRF functionality on the UC VRF functionality [8,1,2], amending it to the setting of a single target verifier. A universally composable PVRF is easier to use as a subprotocol in applications, as exemplified by the SAS-MA scheme PVRF-SAS. Scheme PVRF-SAS relies on the pseudorandomness, simulatability, output uniqueness, and soundness properties of PVRF, and since the same PVRF key is reused in multiple SAS-MA instances, the SAS-MA security argument inherently considers a composition of PVRF instances. The UC PVRF model captures all these properties, and a PVRF that realizes it can be substituted with an ideal PVRF functionality in the PVRF-SAS security proof, which makes that proof clearer and easier to inspect.

We construct an efficient UC PVRF scheme, called PECVRF, as a privately verifiable version of ECVRF [16], which is a well-known VRF based on the Diffie-Hellman function. Scheme PECVRF uses the same underlying Diffie-Hellman PRF as ECVRF, i.e. $vk = g^k$ and $\text{PRF}_k(x) = \text{Hash}(vk, x)^k$ (we simplify this overview by omitting the outer hash applied to this value), but it replaces the NIZK that (g, vk, g_x, y) is a DH tuple for $g_x = \text{Hash}(vk, x)$, with a well-known Smooth Projective Hash Function (SPHF) [7] for the same language, i.e. the verifier picks a pair of random field elements (α, β) and sends $h = g^\alpha \cdot g_x^\beta$ to the prover, who responds with $\pi = h^k$, which the verifier accepts only if $\pi = vk^\alpha \cdot y^\beta$. This satisfies the requirement of prover deniability, because tuple (α, β, h, π) can be sampled given only the statement (g, vk, g_x, y) .

However, an SPHF is not a zero-knowledge proof, and indeed in the above system the prover exposes an exponentiation oracle $(\cdot)^k$ which could be used e.g. to evaluate the PVRF on $x' \neq x$. To fix this, we turn this SPHF into a

designated verifier zero-knowledge proof [19] by adding a simulation trapdoor $h' = g_1^\alpha \cdot g_2^\beta$, where $g_i = \text{Hash}_i(vk, x)$, and forming the verification token as $vt = (h, h', \tau)$ where τ is a NIZK that (h, h') are computed correctly using some α, β . The real-world prover outputs $\pi = h^k$ only if the prover's NIZK τ verifies, and this enables simulation because under the DDH assumption the simulator can embed correlated trapdoored responses (vk^r, y^r) into (g_1, g_2) pairs output by $\text{Hash}_1, \text{Hash}_2$, and compute π as $\pi = (h')^{1/r}$.³

Deniability of the proposed SAS-AKE X3DH extension. Our end-goal is protocol X3DH + PVRF-SAS, i.e. a SAS-AKE extension of X3DH, with the same security, efficiency, and ease of adoption properties as the proposal in [17], but which would maintain the same deniability properties as X3DH. We achieve this objective with a caveat. Table 1 shows a comparison of the deniability properties, as defined by Katsumata et al. [20], achieved (1) by X3DH itself, [20], (2) by the X3DH SAS-AKE extension ‘X3DH+VRF-SAS’ proposed by [17], and (3) by the X3DH SAS-AKE extension ‘X3DH+PVRF-SAS’ proposed in this work.

	X3DH	X3DH+VRF-SAS	X3DH+PVRF-SAS
$(s, r) \in \mathcal{H} \times \mathcal{C}$	high	high	high
$(s, r) \in \mathcal{C} \times \mathcal{H}, \text{otpk} \neq \perp$	high	none/high*	high
$(s, r) \in \mathcal{C} \times \mathcal{H}, \text{otpk} = \perp$	high	none	medium
$(s, r) \in \mathcal{H} \times \mathcal{H}, \text{otpk} \neq \perp$	high	none/high*	high
$(s, r) \in \mathcal{H} \times \mathcal{H}, \text{otpk} = \perp$	medium	none	medium

Table 1: Deniability of protocols X3DH, X3DH + VRF-SAS [17], and X3DH+PVRF-SAS [this work]. This table lists the highest level of leakage for which deniability holds. In all cases, deniability is achieved for **high** disclosure and KDC leakage, except * indicates cases deniability given no KDC leakage.

The five rows in Table 1 correspond to 5 deniability attack scenarios: $\mathcal{H}$ and $\mathcal{C}$ denote the sets of resp. honest and corrupted parties, and (s, r) denote resp. the ‘sender’ and ‘receiver’ parties of X3DH, i.e. resp. the initiator and the responder. Using this terminology, $(s, r) \in \mathcal{H} \times \mathcal{C}$ is a case of an honest initiator/sender whose intended counterparty is corrupt and attempts to gain a proof of the initiator’s engagement in the protocol. Similarly, $(s, r) \in \mathcal{C} \times \mathcal{H}$ is the case of an honest

³ One could also implement vt as a one-time public key amended by a NIZK proof of knowledge of the corresponding private key, and replace the prover’s NIZK with a Designated-Verifier NIZK (DV-NIZK) proof of knowledge [19] or a ‘disjunctive’ Non-Interactive Witness Indistinguishable (NIWI) proof [6], where the prover proves that either (g, vk, g_x, y) is a DH tuple or that the prover holds the secret key corresponding to vt . Constructions using DV-NIZK proofs have been used to implement deniable authentication, e.g. [32, 14], hence PVRF constructed using these techniques probably passes a corresponding notion of PVRF deniability. Our SPHF-based PECVRF has roughly the same costs as ECVRF using DV-NIZK as above, but it offers a performance trade-off of 4 (multi)-exps for the verifier and 2 for the prover, while a DV-NIZK implementation using a disjunctive proof [6] uses 3 (multi)-exps per party.

receiver/responder whose intended counterparty is corrupt and attempts to gain a proof of the responder’s engagement in the protocol. Finally, $(s, r) \in \mathcal{H} \times \mathcal{H}$ considers the deniability of two honest interacting parties and whether their transcript can be efficiently simulatable from public information. The last two cases are split into two sub-cases, where $\text{otpk} \neq \perp$ stands for the default case where the initiator retrieves the responder’s one-time pre-key from the KDC (see above), and $\text{otpk} = \perp$ is the *last resort* case where the KDC runs out of the responder’s one-time pre-keys. The **high** and **medium** deniability levels correspond to the amount of honest party information which is leaked to the distinguisher: In both cases the distinguisher learns the private key corresponding to the honest party’s long-term identity key, but in the **high** level the distinguisher also learns their dynamic state, including their secret semi-static key and any *unused* private one-time pre-keys. Finally, we assume ‘KDC leakage’ by default, i.e. that the distinguisher learns the accused party’s full pre-key bundle, and the two entries which assume ‘no KDC leakage’ are marked with an asterisk *.

All deniability properties listed are achieved in the face of both honest-but-curious and malicious adversaries, except the cryptographic assumptions differ between the Random Oracle Model (ROM) for the former and the ROM + the Algebraic Group Model (AGM) for the latter. As can be seen from the table, our proposal X3DH+PVRF-SAS maintains the deniability properties of X3DH except in the case where the receiver responds in the last resort case, i.e. when its store of one-time keys in the pre-key bundle is exhausted, and, furthermore, the receiver’s private semi-static key leaks to the distinguisher. Note that X3DH failed to fully protect receiver’s deniability in this case as well, but in the $(s, r) \in \mathcal{H} \times \mathcal{H}$ case where the protocol transcript can frame either the sender or the receiver but not both, whereas in X3DH+PVRF-SAS the receiver in this case would be personally un-deniable if it engaged with a corrupted counterparty. This appears a modest degradation of deniability compared to X3DH, and a moderate price for extending X3DH to SAS-AKE, i.e. achieving human-centric secure channel authentication without trust in third parties like KDC.

Organization. We recall the standard definitions of the Computational (CDH), Decisional (DDH), and Gap (GapDH) Diffie-Hellman assumptions in Appendix A. In Section 2 we define PVRF and show our PVRF scheme based on ECVRF. In Section 3 we define SAS-MA and we specify a SAS-MA scheme based on PVRF. In Section 4 we recall a definition of Deniable AKE, and in Section 5 we show an intergration of our PVRF-based SAS-MA with X3DH and argue that it is a secure *and deniable* AKE. For the reasons of space constraints much material, including security models and proofs, is deferred to the appendices.

2 Privately-Verifiable Random Functions

A Verifiable Random Function (VRF), introduced by Micali, Rabin, and Vadhan [27], is a Pseudorandom Function (PRF) extended by the presence of a verification key vk that is created by the key generation algorithm along with the PRF key k . The VRF evaluation algorithm computes the PRF output z along with

a noninteractive proof π that $z = \text{PRF}_k(x)$, where PRF is the underlying PRF and k is the key committed to by verification key vk . Any verifier holding vk can verify this proof. Crucially, the function remains pseudorandom in the view of parties that have seen vk . VRFs must also satisfy soundness, meaning that it is infeasible to create a verification key vk and proofs π, π' that two different values z, z' are both correct VRF outputs on the same argument x .

A VRF proof acts like a signature on message m under key vk , and the use of VRFs in the SAS-MA protocol of [17] is therefore deleterious to deniability. In this work, we consider the replacement of VRFs with *Privately-Verifiable Random Functions* (PVRFs), which behave much like VRFs except that their proofs are only verifiable by a single target verifier who has created a verification token vt . As explained in Section 1, this notion is inspired by the verification properties of SPHFs and DV-NIZKs.

Formally, we define a PVRF as a tuple of the following 4 algorithms:

- $\text{KeyGen}(1^\kappa) \rightarrow (k, vk)$, used by the prover to generate a verification key vk with corresponding secret key k .
- $\text{PreVerify}(vk, x) \rightarrow (vt, vts)$, used by a verifier of a PRF evaluation on argument x under key vk , generating the verification token vt and state vts .
- $\text{Eval}(k, x, vt) \rightarrow (z, \pi)$, used by the prover to evaluate the PRF on argument x under key k , generating output z and proof π which targets token vt .
- $\text{Verify}(vk, x, vts, z, \pi) \rightarrow 0/1$, used by the verifier holding state vts to verify proof π that z is the correct PRF output on argument x under key vk .

Figure 1 is protocol PECVRF , a privately-verifiable variant of the ECVRF protocol. As discussed in the Introduction, PECVRF computes the same pseudo-random function as ECVRF , but it achieves private (and deniable) verifiability using smooth projective hash functions (SPHFs) instead of non-interactive zero-knowledge proofs (NIZKs). However, for simulatability of the prover’s SPHF response the verification token created by the verifier includes a simulation-enabling trapdoor together with a NIZK proof that it is computed correctly.

In Appendix B we define a UC model for PVRF and prove that it is realized by PECVRF . This functionality, however, only ensures PVRF security, i.e. correctness, pseudorandomness, and soundness. These properties are sufficient to prove the security of our SAS-MA scheme PVRF-SAS (see Section 3), but not to prove the deniability of the key-reusing composition of PVRF-SAS with X3DH . We therefore provide a separate, game-based definition for PVRF *deniability* in Figure 2. PVRF deniability has two variants: standard, defined by game $\text{Game}_{\text{D,PVRF}}^{\text{Den}}$, and strong, defined by game $\text{Game}_{\text{D,PVRF}}^{\text{SDen}}$. Both games involve an algorithm D who seeks to distinguish between real PVRF outputs generated by an honest prover and “faked” outputs generated by a simulator FakeEval . In the standard deniability game the real prover executes on a verification token vt that is honestly generated by PreVerify , and FakeEval takes as input the corresponding secret vts . In the strong deniability game, there is an active adversary $\mathcal{A}$ that maliciously generates vt , and FakeEval is allowed black-box access to $\mathcal{A}$. Protocol PVRF is *deniable* if there exists an efficient algorithm FakeEval and a negligible function ε s.t. for all efficient D $\Pr[1 \leftarrow \text{Game}_{\text{D,PVRF}}^{\text{Den}}] \leq \frac{1}{2} + \varepsilon(\kappa)$.

$\mathbb{G}$ is a group of prime-order p with generator g , and H_i, H_o, H_a, H_b , and H_{FS} are hash functions whose codomains are respectively $\mathbb{G}, \mathbb{Z}, \mathbb{G}, \mathbb{G}$, and $\mathbb{Z}_p$.

On (KeyGen, kptr), P does:

- sample $k \leftarrow_{\$} \mathbb{Z}_p$, set $vk := g^k$, and store (kptr, k, vk)
- output (VerificationKey, kptr, P, vk)

On (PreVerify, vk, x), V does:

- sample $\alpha, \beta \leftarrow_{\$} \mathbb{Z}_p$, set $h := g^\alpha \cdot H_i(vk, x)^\beta$ and $h' := H_a(vk, x)^\alpha \cdot H_b(vk, x)^\beta$
- sample $r_1, r_2 \leftarrow_{\$} \mathbb{Z}_p$, and set $\eta := g^{r_1} \cdot H_i(vk, x)^{r_2}$, $\eta' := H_a(vk, x)^{r_1} \cdot H_b(vk, x)^{r_2}$, $c := H_{FS}(vk, x, h, h', \eta, \eta')$, $s := (r_1 - c \cdot \alpha, r_2 - c \cdot \beta)$, and $\tau = (c, s)$
- set $vt := (h, h', \tau)$, and store $(vt, vk, x, \alpha, \beta)$
- output (VerificationToken, vt)

On (Eval, kptr, x, vt), P retrieves (kptr, k, vk) and does:

- parse $(h, h', \tau) \leftarrow vt$ and $(c, (s_1, s_2)) \leftarrow \tau$
- set $\eta := g^{s_1} \cdot H_i(vk, x)^{s_2} \cdot h^c$ and $\eta' := H_a(vk, x)^{s_1} \cdot H_b(vk, x)^{s_2} \cdot h'^c$, and if $c \neq H_{FS}(vk, x, h, h', \eta, \eta')$, then abort
- set $w := H_i(vk, x)^k$, $z := H_o(w)$, $v := h^k$, and $\pi := (w, v)$
- output (Evaluated, vk, x, vt, z, π)

On (Verify, vk, x, vt, z, π), V retrieves $(vt, vk, x, \alpha, \beta)$ and does:

- parse $(w, v) \leftarrow \pi$
- set $f \leftarrow 1$ if $z = H_o(w)$ and $v = vk^\alpha \cdot w^\beta$; otherwise, set $f \leftarrow 0$
- output (Verified, vk, x, vt, z, π, f)

Fig. 1: Protocol PECVRF: the *private* elliptic curve verifiable random function

Protocol PVRF is *strongly deniable* if for all efficient algorithms $\mathcal{A}$ there exists an efficient algorithm $\text{FakeEval}_{\mathcal{A}}$ and a negligible function ε s.t. for all efficient D $\Pr[1 \leftarrow \text{Game}_{D, \mathcal{A}, \text{PVRF}}^{\text{SDen}}] \leq \frac{1}{2} + \varepsilon(\kappa)$.

Theorem 1. PECVRF is deniable if the DDH problem is hard in $\mathbb{G}$ and H_i is modeled as a random oracle.

Theorem 2. PECVRF is strongly deniable in the AGM if the DDH problem is hard in $\mathbb{G}$ and H_i is modeled as a random oracle.

We defer the proofs of Theorems 1 and 2 to Appendix B.

3 Message Authentication in the SAS Model

In this section we recall the syntax and the security goals of Message Authentication in the Short Authenticated String model (SAS-MA), and we show a SAS-MA scheme based on Private VRF (PVRF), introduced in Section 2. Our SAS-MA scheme requires only two flows after key generation, but unlike the VRF-based SAS-MA of [17] it does not create a publicly verifiable transcript,

$\text{Game}_{D, \text{PVRF}}^{\text{Den}}(1^\kappa):$	$\text{Game}_{D, \mathcal{A}, \text{PVRF}}^{\text{SDen}}(1^\kappa):$
$\text{ppm} \leftarrow \text{ParGen}(1^\kappa)$	$\text{ppm} \leftarrow \text{ParGen}(1^\kappa)$
$(\text{sk}, \text{vk}) \xleftarrow{\$} \text{KeyGen}(\text{ppm})$	$(\text{sk}, \text{vk}) \xleftarrow{\$} \text{KeyGen}(\text{ppm})$
$b \xleftarrow{\$} \{0, 1\}$	$b \xleftarrow{\$} \{0, 1\}$
$b' \leftarrow D^{\text{TestEval}}(\text{sk}, \text{vk}, b, \cdot)(\text{vk})$	$b' \leftarrow D^{\text{STestEval}}(\text{sk}, \text{vk}, b)(\text{vk})$
return $(b = b')$	return $(b = b')$
Oracle $\text{TestEval}(\text{sk}, \text{vk}, b, x):$	Oracle $\text{STestEval}(\text{sk}, \text{vk}, b):$
$(\text{vt}, \text{vts}) \leftarrow \text{PreVerify}(\text{vk}, x)$	if $(b = 0)$ then
if $(b = 0)$ then	$(x, \text{vt}, \text{st}) \leftarrow \mathcal{A}(\text{vk})$
$(z, \pi) \leftarrow \text{Eval}(\text{sk}, \text{vt}, x)$	$(z, \pi) \leftarrow \text{Eval}(\text{sk}, x, \text{vt})$
else $(z, \pi) \leftarrow \text{FakeEval}(\text{vk}, \text{vt}, \text{vts}, x)$	else $(x, \text{vt}, \text{st}, z, \pi) \leftarrow \text{FakeEval}_{\mathcal{A}}(\text{vk})$
return $(\text{vt}, \text{vts}, z, \pi)$	return $(x, \text{vt}, \text{st}, z, \pi)$

Fig. 2: PVRF deniability game, with standard and strong variants.

and hence it can be used in conjunction with a *deniable* Authenticated Key Exchange protocol, as we show in Section 5.

SAS-MA was first defined by Vaudenay [34], but we follow [17] in adjusting SAS-MA syntax for schemes in which each party generates a public key which it can re-use in multiple SAS-MA protocol instances. Unlike [17], we restrict our definition for the case of only 2 parties, P_1 and P_2 , who want to mutually authenticate a message to each other. Their respective inputs are denoted m_1 and m_2 , and they want to confirm that $m_1 = m_2$. The parties communicate over insecure channels and have no shared source of trust, except for an additional communication resource of a low-bandwidth bi-directional authenticated channel. Namely, for some parameter d , each P_i can send one d -bit message, denoted sas_i , on the *Short Authenticated String* (SAS) *channel* connecting P_i and its counterparty. This channel is not secret, so the adversary learns sas_i , but it preserves integrity and source authentication, i.e. the adversary cannot modify this message or replace it with its own. In more detail, we define a SAS-MA scheme as a following algorithm tuple:

- $\text{PreProc}(1^\kappa) \rightarrow (k, \text{vk})$, a pre-processing algorithm used by P_2 to generate a re-usable public key vk and a corresponding private key k .
- $\text{Init}(\text{vk}, m) \rightarrow (pm_1, \text{st})$, initiator P_1 's generation of a protocol message and an ephemeral state, given counterparty's vk and an authentication target m .
- $\text{Resp}(k, m, pm_1) \rightarrow (pm_2, \text{sas}_2)$, responder P_2 's generation of a protocol message and a SAS value, given its private key k and a target message m .
- $\text{InitFin}(\text{st}, pm_2) \rightarrow \text{sas}_1$, the initiator P_1 's generation of its SAS value.

Finally, each P_i sends its SAS value $\text{sas}_i \in \{0, 1\}^d$ on its SAS channel to the counterpart, and each party accepts if and only if $\text{sas}_1 = \text{sas}_2$. Figure 3 shows PVRF-SAS, our PVRF-based SAS-MA scheme. In this scheme the short authentication string $\text{sas} \in \{0, 1\}^d$ is formed as $z \oplus \text{nonce}$, where z is the output of responder P_2 's PVRF on input m , and nonce is a random string chosen by P_1 .

PVRF = (KeyGen, PreVerify, Eval, Verify) is a PVRF with range $\{0, 1\}^d$.	
<u>PreProc(1^κ):</u>	<u>Init(vk, m):</u>
$(k, vk) \leftarrow \text{KeyGen}(1^\kappa)$	$\text{nonce} \leftarrow_{\$} \{0, 1\}^d$
return (k, vk)	$(vt, vts) \leftarrow \text{PreVerify}(vk, m)$
	return $((vt, \text{nonce}), (vk, m, \text{nonce}, vts))$
<u>Resp($k, m, pm_1 = (vt, \text{nonce})$):</u>	<u>InitFin($st = (vk, m, \text{nonce}, vts), pm_2 = (z, \pi)$):</u>
$(z, \pi) \leftarrow \text{Eval}(k, vt, m)$	assert $\text{Verify}(vk, vts, m, z, \pi)$
return $((z, \pi), z \oplus \text{nonce})$	return $z \oplus \text{nonce}$

Fig. 3: SAS-MA scheme PVRF-SAS

Security of PVRF-SAS follows from the properties of the underlying PVRF. In particular, PVRF pseudorandomness (and input m freshness) ensures that z is effectively random until after adversary $\mathcal{A}$ commits to nonce by executing **Resp**. Clearly nonce is random until $\mathcal{A}$ executes **Init**, and PVRF soundness ensures that z is effectively fixed by $\mathcal{A}$ at that time. Thus, neither party's sas can be predicted and controlled by a man-in-the-middle adversary, and $\mathcal{A}$ has no attack that succeeds with probability better than $1/2^d$. We defer to Appendix C the formal definition of SAS-MA security and the proof that PVRF-SAS achieves it. This proof is extremely similar to that of [17]; recall that we replace VRF with PVRF in order to strengthen deniability—not security—properties.

4 Deniable Authenticated Key Exchange

To model the security properties of X3DH, Hashimoto, Katsumata, and Wiggers [18] introduced the notion of Bundled Authenticated Key Exchange (BAKE), which extends the standard definition of AKE with *prekey bundles*. Each party in BAKE creates a bundle containing some number L of one-time prekeys, and BAKE key exchange is an asymmetric interaction between a *sender* and *receiver*, who must agree on an index $t \in [L]$ in the receiver's prekey bundle. The BAKE security model enforces that each prekey is only used once, but BAKE also allows for a fallback mechanism designated $t = \perp$ whereby a special *last-resort prekey* is used instead, in case the receiver's bundle has run out of one-time keys. In that eventuality, the last-resort key (also called the *semi-static key*) might be reused across many key exchanges, but the forward security properties of these key exchanges are diminished compared to those that use one-time keys.

Katsumata et al. [20] later defined *deniability* (with various gradations) for BAKE protocols, but in order to use their models we must make some adaptations. Adding SAS-MA authentication to X3DH involves an extra communication flow from the receiver to the sender. A *modified* bundled authenticated key exchange $\text{BAKE}^\dagger$ is a tuple of algorithms (IdKeyGen , PreKeyBundleGen , Send , Receive , SenderReceive), parameterized by bundle size L , with the following syntax:

- $\text{BAKE}^\dagger.\text{IdKeyGen}(1^\kappa) \rightarrow (\text{ipk}, \text{isk})$, an identity key generation algorithm that outputs an identity public key ipk and corresponding secret key isk .
- $\text{BAKE}^\dagger.\text{PreKeyBundleGen}(\text{isk}_u) \rightarrow (\mathbf{prek}_u, \mathbf{st}_u)$, a prekey bundle generation algorithm that, on input user u 's identity secret key, outputs a bundle of L prekeys (and a last-resort prekey) $\mathbf{prek}_u = (\text{prek}_{u,t})_{t \in [L] \cup \{\perp\}}$, and a user state $\mathbf{st}_u$. (The state may for example include the secret keys associated to the public keys in $\mathbf{prek}_u$.)
- $\text{BAKE}^\dagger.\text{Send}(\text{isk}_s, \text{ipk}_r, \text{prek}_{r,t}) \rightarrow (K, \rho, \text{temp})$, a sender algorithm that, on input sender s 's identity secret key isk_s and the intended receiver r 's identity key ipk_r and prekey $\text{prek}_{r,t}$, outputs a key K , a handshake message ρ , and a temporary state temp for the sender.
- $\text{BAKE}^\dagger.\text{Receive}(\text{isk}_r, \mathbf{st}_r, \text{ipk}_s, t, \rho) \rightarrow (K', m, \text{sas}_r, \mathbf{st}_r)$, a receiver algorithm that, on input receiver r 's identity secret key isk_r and state $\mathbf{st}_r$, sender s 's identity key ipk_s , prekey index $t \in [L] \cup \{\perp\}$, and handshake message ρ , outputs a key K' , a response message m , a short authentication string sas_r , and an updated state $\mathbf{st}_r$. (Key agreement may fail, in which case $K' = \perp$.)
- $\text{BAKE}^\dagger.\text{SenderReceive}(\text{temp}, m) \rightarrow \text{sas}_s$, a sender completion algorithm that, on input state temp from Send and a response message m , outputs a short authentication strings sas_s . (Authentication may fail, in which case $\text{sas}_s = \perp$.)

We highlight the differences between BAKE and $\text{BAKE}^\dagger$ in yellow text. The syntax of IdKeyGen and PreKeyBundleGen is unchanged. The sender saves a temporary state temp , awaits a response message m output from Receive , and then uses SenderReceive to complete SAS authentication. Note that K is output as expected in BAKE algorithms, and out-of-band verification of $\text{sas}_r = \text{sas}_s$ is only an ‘add-on’ to the regular BAKE execution.

4.1 Deniability Against Honest-but-Curious Accusers

Figures 4 and 5 are the deniability game $\text{Game}_{\mathcal{D}, \mathcal{H}, \mathcal{L}_{\text{leak}}, \mathcal{D}_{\text{disc}}, \mathcal{L}_{\text{KDC}}}^{\text{ATK}}$ for honest-but-curious accusers (parameterized by leak , disc , $\mathcal{L}_{\text{KDC}}$, and ATK , explained below), with differences from [20] marked in yellow text. The game concerns a *distinguisher* $\mathcal{D}$ (also called a judge) attempting to distinguish between real and simulated transcripts of $\text{BAKE}^\dagger$ key exchanges between N parties. Some subset $\mathcal{H} \subseteq [N]$ of the parties are honest and wish to deny their $\text{BAKE}^\dagger$ interactions. The rest $\mathcal{C} = [N] \setminus \mathcal{H}$ are *accusers* who wish to incriminate the honest parties.

The deniability game runs either in $\text{mode} = \text{real}$ or $\text{mode} = \text{sim}$. Distinguisher $\mathcal{D}$ is given the public keys of all parties and access to an oracle that adaptively directs pairs $(s, r) \in [N] \times [N]$ of parties to perform key exchanges. In the *real*

```

GameD, H, Lleak, Ddisc, LKDCATK(1κ, mode):
  C := N \ H
  for user u ∈ N do
    (ipku, isku)  $\xleftarrow{\$}$  BAKE†.IdKeyGen(1κ)
    counteru := 0
    if (u ∈ H) then
      (preku, stu)  $\xleftarrow{\$}$  BAKE†.PreKeyBundleGen(isku)
  if (mode = real) then
    for user u ∈ C do
      (preku, stu)  $\xleftarrow{\$}$  BAKE†.PreKeyBundleGen(isku)
      stuinit := stu
  else if (mode = sim) then
    ((preku*)u ∈ H, (preku)u ∈ C, stsim)  $\xleftarrow{\$}$  SimPreK((ipku)u ∈ N, (isku)u ∈ C, (preku)u ∈ H)
    if ¬LKDC then (preku)u ∈ H ← (preku*)u ∈ H
  stD  $\xleftarrow{\$}$  DATK((ipku, preku)u ∈ N)
  if (mode = sim) then
    (stu, stuinit)  $\xleftarrow{\$}$  SimStC(stsim)
  leakD := Lleak((isku, stu)u ∈ H)
  discD := Ddisc((isku, stu, stuinit)u ∈ C)
  b  $\xleftarrow{\$}$  D(stD, leakD, discD)
  return b

```

Fig. 4: Local and global BAKE[†] deniability game for *honest-but-curious* accusers, with changes relative to BAKE deniability [20] marked in yellow.

mode, the real BAKE[†] algorithms are used to generate a transcript and return it to D. In the *sim* mode, there is instead a simulator algorithm *SimTrans* that generates the returned transcript. The simulator is given the secrets of accusers, but not those of honest parties. Intuitively, deniability is implied if there exists such a simulator that can produce transcripts indistinguishable from those of real interactions. The game starts with each party running the *IdKeyGen* and *PreKeyBundleGen* algorithms. We follow the choice of [20] to assume that all parties' identity keys are honestly generated, even in the simulated world. On the other hand, accusers' simulated-world prekey bundles are generated by the simulator algorithm *SimPreK*, which is given as input all honest parties' public keys, prekey bundles and the identity secret keys of the accusers.

Leakage and Disclosure. In the deniability model of [20], the simulator is also allowed to overwrite all honest parties' prekey bundles. It is these simulated

```

Oracle  $\mathcal{O}_{\text{ATK}}(s, r)$ :
assert  $((s, r) \in \mathcal{N} \times \mathcal{N}) \wedge (s \neq r) \wedge ((s, r) \notin \mathcal{C} \times \mathcal{C})$ 
if (ATK = local) then assert  $(s, r) \notin \mathcal{H} \times \mathcal{H}$ 
 $\text{counter}_r \leftarrow \text{counter}_r + 1, t := \text{counter}_r$ 
if  $(t > L)$  then  $t \leftarrow \perp$ 
if (mode = real) then
     $(K, \rho, \text{temp}) \xleftarrow{\$} \text{BAKE}^\dagger.\text{Send}(\text{isk}_s, \text{ipk}_r, \text{prek}_{r,t})$ 
     $(K', m, \text{sas}_r, \text{st}_r) \xleftarrow{\$} \text{BAKE}^\dagger.\text{Receive}(\text{isk}_r, \text{st}_r, \text{ipk}_s, t, \rho)$ 
     $\text{sas}_s \leftarrow \text{BAKE}^\dagger.\text{SenderReceive}(\text{temp}, m)$ 
else if (mode = sim) then
    if  $(s, r) \in \mathcal{H} \times \mathcal{C}$  then
         $(K, \rho, m, \text{sas}_s, \text{st}_{\text{Sim}}) \xleftarrow{\$} \text{SimTrans}(\text{isk}_r, \text{st}_{\text{Sim}}, (s, r, t))$ 
    else if  $(s, r) \in \mathcal{C} \times \mathcal{H}$  then
         $(K', \rho, m, \text{sas}_r, \text{st}_{\text{Sim}}) \xleftarrow{\$} \text{SimTrans}(\text{isk}_s, \text{st}_{\text{Sim}}, (s, r, t))$ 
         $\text{st}_r \xleftarrow{\$} \text{SimSt}_{\mathcal{H}}(\text{isk}_r, \text{st}_r)$ 
    else if  $(s, r) \in \mathcal{H} \times \mathcal{H}$  then
         $(K, K', \rho, m, \text{sas}_s, \text{sas}_r, \text{st}_{\text{Sim}}) \xleftarrow{\$} \text{SimTrans}(\perp, \text{st}_{\text{Sim}}, (s, r, t))$ 
         $\text{st}_r \xleftarrow{\$} \text{SimSt}_{\mathcal{H}}(\text{isk}_r, \text{st}_r)$ 
    if  $(s, r) \in \mathcal{H} \times \mathcal{C}$  then return  $(K, \rho, m, \text{sas}_s)$ 
    else if  $(s, r) \in \mathcal{C} \times \mathcal{H}$  then return  $(K', \rho, m, \text{sas}_r)$ 
    else return  $(K, K', \rho, m, \text{sas}_s, \text{sas}_r)$ 

```

Fig. 5: Oracle $\mathcal{O}_{\text{ATK}}$ for honest-but-curious accusers in $\text{BAKE}^\dagger$ deniability game in Fig. 4, with changes from BAKE deniability game [20] marked in yellow.

prekey bundles that are then given to D, and the honestly-generated ones are discarded. Effectively, such a model only implies deniability against a judge who cannot observe the prekey bundles generated by honest parties. This may be a reasonable model for some scenarios, but one can also imagine a judge capable of monitoring (or retroactively seizing) the prekey bundles saved by the KDC. To capture the stronger deniability notion of this setting, we parameterize the deniability game by a bit $\mathcal{L}_{\text{KDC}}$, which represents whether honestly-generated prekey bundles are “leaked” to D. If $\mathcal{L}_{\text{KDC}} = 0$ then these bundles can be overwritten by SimPreK as in [20], and if $\mathcal{L}_{\text{KDC}} = 1$, then this overwriting is skipped. As shown in Section 1, there are schemes whose deniability is severely diminished when this overwriting is disallowed by $\mathcal{L}_{\text{KDC}} = 1$.

At the end of the deniability game (i.e. after all key exchanges), D is allowed some final secret information before attempting to distinguish the real and simulated worlds. This information is classified as *leakage* from honest parties and *disclosure* from accusers, and the game is parameterized by leakage level **leak** and disclosure level **disc**, both in $\{\text{low}, \text{med}, \text{high}\}$. At the highest levels, D is shown the identity secret keys and internal state (i.e. prekey secret keys) of all parties. Honest parties are assumed to delete their one-time prekey secret keys after use, and even at **leak** = **high** only the honest parties' final states are leaked (i.e. deleted data is excluded). Honest-but-curious accusers, however, might not make such deletions, and so at **disc** = **high** the accusers' *initial* states are disclosed (i.e. deleted data is included).

Definition 1. The leakage function $\mathcal{L}_{\text{leak}}$ for the set $\mathcal{H}$ of accused users of a $\text{BAKE}^\dagger$ protocol is defined as follows:

$$\mathcal{L}_{\text{leak}}((\text{isk}_u, \text{st}_u)_{u \in \mathcal{H}}) := \begin{cases} (\perp, \perp) & \text{if } \text{leak} = \text{low} \\ ((\text{isk}_u)_{u \in \mathcal{H}}, \perp) & \text{if } \text{leak} = \text{med} \\ ((\text{isk}_u, \text{st}_u)_{u \in \mathcal{H}}) & \text{if } \text{leak} = \text{high} \end{cases}$$

The disclosure function $\mathcal{D}_{\text{disc}}$ for the set $\mathcal{C}$ of honest-but-curious accusers of a $\text{BAKE}^\dagger$ protocol is defined as follows:

$$\mathcal{D}_{\text{disc}}((\text{isk}_u, \text{st}_u, \text{st}_u^{\text{init}})_{u \in \mathcal{C}}) := \begin{cases} ((\text{isk}_u)_{u \in \mathcal{C}}, \perp, \perp) & \text{if } \text{disc} = \text{low} \\ ((\text{isk}_u, \text{st}_u)_{u \in \mathcal{C}}, \perp) & \text{if } \text{disc} = \text{med} \\ ((\text{isk}_u, \text{st}_u, \text{st}_u^{\text{init}})_{u \in \mathcal{C}}) & \text{if } \text{disc} = \text{high} \end{cases}$$

Since parties' internal states are shown to D, these states must be simulated in the simulated world. In addition to **SimTrans** and **SimPreK**, the simulator also includes algorithms **SimSt_C** and **SimSt_H**. The former is executed once at the end of the game to simulate all accusers' states. The latter is used to update honest receiver r 's state after D directs a key exchange between parties (s, r) . By necessity, **SimSt_H** is allowed access to the secret information $(\text{isk}_r, \text{st}_r)$ of $r \in \mathcal{H}$, but **SimSt_H** is "isolated" from (i.e. doesn't share state with) the rest of the simulator algorithms, and it is not given the identity s of the counterparty. Thus, if the real $\text{BAKE}^\dagger.\text{Receive}$ somehow updates st_r in a way that identifies s , this update will not be simulatable and $\text{BAKE}^\dagger$ will not be deniable.

Last-Resort Prekeys. The deniability game enforces that each one-time prekey is used only once, see **counter_r** in Figure 5, and if one-time prekeys run out the game reverts to the use of last-resort prekeys by setting $t \leftarrow \perp$. As discussed in Section 1, many schemes are subject to a degradation in deniability guarantees for receivers who run out of one-time prekeys and are forced to use $t = \perp$.

Local and Global Deniability. Finally, the deniability game is parameterized by $\text{ATK} \in \{\text{local}, \text{global}\}$. In the local (also called 1-out-of-2) deniability game, the distinguisher is disallowed from requesting key exchange transcripts between two honest parties $(s, r) \in \mathcal{H} \times \mathcal{H}$. Therefore, the simulator only needs to be able

to simulate transcripts for key exchanges in which one of the two parties is an accuser, i.e. the simulator knows one of the two parties' secrets. In effect, then, an honest user $h \in \mathcal{H}$ in a locally deniable scheme can only deny their participation in a key exchange with counterparty c by claiming that the transcript was faked by c in an attempt to incriminate h .

Global (also called 1-out-of-infinity) deniability is a substantially stronger notion. In that game, the simulator must be able to simulate transcripts for key exchanges in which both parties are honest, i.e. the simulator doesn't know either party's secrets. (Notice, however, that distinguisher D might also not know either party's secrets, depending on parameter leak .) In a globally deniable scheme, two honest users can simultaneously deny their participation in a key exchange by claiming that the transcript was faked by any third party in an attempt to incriminate them.

Definition 2. (*Deniability Against Honest-but-Curious Accusers*). Let $\mathcal{N} := [N]$ be the set of all users, $\mathcal{H} \subseteq \mathcal{N}$ be the set of accused users, and $\mathcal{C} := \mathcal{N} \setminus \mathcal{H}$ be the set of accusers. Protocol $\text{BAKE}^\dagger$ is deniable against honest-but-curious accusers with respect to leakage $\text{leak} \in \{\text{low}, \text{med}, \text{high}\}$, disclosure $\text{disc} \in \{\text{low}, \text{med}, \text{high}\}$, KDC leakage bit $\mathcal{L}_{\text{KDC}} \in \{0, 1\}$, and locality $\text{ATK} \in \{\text{local}, \text{global}\}$ if there exists an efficient simulator $\text{Sim} = (\text{SimPreK}, \text{SimTrans}, \text{SimSt}_{\mathcal{H}}, \text{SimSt}_{\mathcal{C}})$ such that for any efficient distinguisher D , for $\text{Game}_{D, \mathcal{H}, \mathcal{L}_{\text{leak}}, \mathcal{D}_{\text{disc}}, \mathcal{L}_{\text{KDC}}}^{\text{ATK}}$ specified in Fig. 4 and 5, there is a negligible function ϵ s.t.

$$\Pr \left[\text{Game}_{D, \mathcal{H}, \mathcal{L}_{\text{leak}}, \mathcal{D}_{\text{disc}}, \mathcal{L}_{\text{KDC}}}^{\text{ATK}}(1^\kappa, \text{real}) = 0 \right] \leq \Pr \left[\text{Game}_{D, \mathcal{H}, \mathcal{L}_{\text{leak}}, \mathcal{D}_{\text{disc}}, \mathcal{L}_{\text{KDC}}}^{\text{ATK}}(1^\kappa, \text{sim}) = 0 \right] + \epsilon(\kappa)$$

4.2 Deniability Against Malicious Accusers

We also consider deniability against malicious accusers (also called *strong* deniability). As in Section 4.1, we use the model of [20], but adapted for the extra communication flow of $\text{BAKE}^\dagger$. This strong deniability game closely resembles the honest-but-curious game above, so to avoid redundancy we defer its full specification to Appendix D and provide an overview of its important aspects here. The essential difference from the honest-but-curious deniability game is that, in the malicious deniability game, all the actions of real world accusers are determined by an accuser algorithm $\mathcal{A}$, not by honest invocations of the $\text{BAKE}^\dagger$ algorithms. In particular, $\mathcal{A}$ provides all accusers' prekey bundles at the start of the game, and $\mathcal{A}$ generates the ρ, m protocol messages sent by accusers during key exchange sessions. At the end of the game, the entire state of the malicious accuser $\mathcal{A}$ is shown to distinguisher D (replacing the graduated disclosure functions of the honest-but-curious accusers above).

The simulator in the strong deniability game has a much more difficult task. It must be able to simulate the behavior and ultimate state of an arbitrary accuser algorithm $\mathcal{A}$. In order for this to be possible, the simulator is allowed black-box oracle access to $\mathcal{A}$. Stated differently, the definition of strong deniability requires that, for any $\mathcal{A}$, there exists a simulator such that, for every D , the real and simulated worlds are not distinguished by D .

5 Integration of X3DH with PVRF-based SAS-MA

We introduce protocol X3DH+PVRF-SAS in Fig. 6. This protocol is not simply a black-box composition of X3DH and PVRF-SAS. In order to maximize the ease of integration with existing X3DH deployments, we reuse the X3DH receiver’s one-time key otsk (or semi-static key ssk if one-time keys run out) as their PECVRF key. This design choice allows PVRF-SAS authentication to be added to X3DH without any change to the key distribution server. There is only a small overhead to the sender and receiver’s client-side code and communication bandwidth on their first two messages exchanged. Because of this key reuse, however, security of X3DH+PVRF-SAS is not immediate. Nonetheless, Theorems 3 and 4 below state that (1) X3DH remains secure when its keys are reused for PECVRF, and (2) PECVRF remains secure when its keys are reused for X3DH. Together these theorems imply the security of X3DH+PVRF-SAS.

Theorem 3. *X3DH with PECVRF key reuse (Figure 6) is a key-indistinguishable bundled authenticated key exchange protocol [18, Definition 3] under the GapDH and DDH assumptions and in the ROM.*

Theorem 4. *PECVRF with X3DH key reuse (Figure 6) realizes $\mathcal{F}_{\text{PVRF}}$ under the GapDH and DDH assumptions in the ROM.*

Due to space constraints we defer the proofs of Theorems 3 and 4 to Appendix E. Briefly, the former proof relies on the UC security of protocol PECVRF, but with non-black-box use of the PECVRF simulator. The latter proof is based on the observation that a DDH oracle can be used to simulate X3DH without knowledge of secret keys.

Deniability of X3DH+PVRF-SAS against passive adversaries. We specify the honest-but-curious deniability of X3DH+PVRF-SAS in the following theorem.

Theorem 5. *The X3DH+PVRF-SAS protocol is globally deniable with respect to disclosure function $\mathcal{D}_{\text{high}}$, leakage function $\mathcal{L}_{\text{high}}$ and KDC leakage bit $\mathcal{L}_{\text{KDC}} = 1$ if the one-time prekey bundles are never depleted. If one-time prekeys run out then it is globally deniable for leakage function $\mathcal{L}_{\text{med}}$. Specifically, if one-time prekeys run out it is still locally deniable for the sender for leakage function $\mathcal{L}_{\text{high}}$ and it is locally deniable for the receiver and globally deniable for the sender and receiver for leakage function $\mathcal{L}_{\text{med}}$ if DDH is hard and KDF is a random oracle.*

Strong deniability of X3DH+PVRF-SAS. We specify the deniability of X3DH+PVRF-SAS against malicious adversaries in the following theorem:

Theorem 6. *Assuming the underlying group used in X3DH+PVRF-SAS is algebraic, the set of malicious accusers are algebraic algorithms over the groups used in X3DH+PVRF-SAS and the one-time prekeys never run out then X3DH+PVRF-SAS is strongly global deniable for leakage function $\mathcal{L}_{\text{high}}$ and KDC leakage bit $\mathcal{L}_{\text{KDC}}=1$.*

Public parameters: Group $\mathbb{G}$ of prime order p with generator G .

PECVRF instantiation: Algorithms (PreVerify, Eval, Verify) are as in Fig. 1.

Key Registration

Party $P \in \{A, B\}$

Server S

$\text{isk}_P, \text{ssk}_P, \text{otsk}_{P,1}, \dots, \text{otsk}_{P,L} \xleftarrow{\$} \mathbb{Z}_p$

$\{i, s, \text{ot}\} \text{pk}_P := [\{i, s, \text{ot}\} \text{sk}_P]G$

$\xrightarrow{\text{ipk}_P, \text{spk}_P, \{\text{otpk}_{P,t}\}_{t \in [L]}}$

S stores $[P, \text{ipk}_P, \text{spk}_P, \{\text{otpk}_{P,t}\}_{t \in [L]}]$

Authenticated Key Exchange

Sender A on input $\text{CP} = B$

Responder B on input $\text{CP} = A$

retrieve $\text{ipk}_B, \text{spk}_B, \text{otpk}_{B,t}$ from S

(S deletes $\text{otpk}_{B,t}$ from record $[B, \dots]$)

$\text{esk} \xleftarrow{\$} \mathbb{Z}_p, \text{epk} = [\text{esk}]G$

(see in-text for the case that $\text{otpk}_{B,t} = \perp$)

$\text{nonce} \xleftarrow{\$} \{0, 1\}^d$

$\text{tran} := \text{ipk}_A || \text{ipk}_B || \text{spk}_B || \text{otpk}_{B,t} || \text{epk}$

$(vt, vts) \leftarrow \text{PreVerify}(\text{otpk}_{B,i}, \text{tran})$

$\xrightarrow{A, B, \text{epk}, t, \text{nonce}, vt}$

$\text{ss} := [\text{isk}_A] \text{spk}_B || [\text{esk}] \text{ipk}_B ||$
 $[\text{esk}] \text{spk}_B || [\text{esk}] \text{otpk}_{B,t}$

retrieve ipk_A from S

$\text{ss} := [\text{ssk}_B] \text{ipk}_A || [\text{isk}_B] \text{epk} ||$
 $[\text{ssk}_B] \text{epk} || \text{otsk}_{B,t} [\text{epk}]$

$\text{tran} := \text{ipk}_A || \text{ipk}_B || \text{spk}_B || \text{otpk}_{B,t} || \text{epk}$

delete esk

$(K, \tau_{\text{conf}}) := \text{KDF}(\text{ss}, \text{tran})$

$(K, \tau'_{\text{conf}}) := \text{KDF}(\text{ss}, \text{tran})$

$\xrightarrow{\tau_{\text{conf}}}$

assert $\tau_{\text{conf}} = \tau'_{\text{conf}}$

output K

output K

$\xleftarrow{z_B, \pi_B}$

$(z_B, \pi_B) \leftarrow \text{Eval}(\text{otsk}_{B,t}, vt, \text{tran})$

assert $\text{Verify}(\text{otpk}_{B,t}, vts, z_B, \pi_B, \text{tran})$

delete $\text{otsk}_{B,t}$

output $\text{sas}_A = \text{nonce} \oplus z_B$

output $\text{sas}_B = \text{nonce} \oplus z_B$

Fig. 6: Protocol X3DH+PVRF-SAS, i.e. X3DH amended by SAS-MA scheme PVRF-SAS instantiated with PECVRF (see Fig. 1) shown in gray. (Confirming that $\text{sas}_A = \text{sas}_B$ is left to the application level.)

It is strongly local deniable for receivers and strongly global deniable for senders and receivers if leak = med and the last-resort prekey is used. It is strongly local deniable for senders for leak = high even if last-resort prekey is used.

We defer the details of the protocol and deniability proof to Appendix G.

References

1. C. Badertscher, P. Gazi, A. Kiayias, A. Russell, and V. Zikas. Ouroboros genesis: Composable proof-of-stake blockchains with dynamic availability. In *Computer and Communications Security - CCS*, pages 913–930, 2018.
2. C. Badertscher, P. Gazi, I. Querejeta-Azurmendi, and A. Russell. On UC-secure range extension and batch verification for ECVRF. *Cryptology ePrint Archive*, 2022. <https://eprint.iacr.org/2022/1045>.
3. M. Bellare and G. Neven. Multi-signatures in the plain public-key model and a general forking lemma. In *Computer and Communications Security - CCS*, page 390–399, 2006.
4. R. Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *Foundations of Computer Science - FOCS*, pages 136–145, 2001.
5. K. Cohn-Gordon, C. Cremers, B. Dowling, L. Garratt, and D. Stebila. A formal security analysis of the Signal messaging protocol. In *2017 IEEE European Symposium on Security and Privacy (EuroSP)*, pages 451–466, 2017.
6. R. Cramer, I. Damgård, and B. Schoenmakers. Proofs of partial knowledge and simplified design of witness hiding protocols. In *International Cryptology Conference - Crypto*, pages 174–187, 1994.
7. R. Cramer and V. Shoup. Universal hash proofs and a paradigm for adaptive chosen ciphertext secure public-key encryption. In *Advances in Cryptology - Eurocrypt*, pages 45–64, 2002.
8. B. David, P. Gazi, A. Kiayias, and A. Russell. Ouroboros praos: An adaptively-secure, semi-synchronous proof-of-stake blockchain. In *Advances in Cryptology - Eurocrypt*, pages 66–98, 2018.
9. M. Di Raimondo, R. Gennaro, and H. Krawczyk. Deniable authentication and key exchange. In *Computer and Communications Security - CCS*, pages 400–409, 2006.
10. W. Diffie and M. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, 1976.
11. Y. Dodis, J. Katz, A. Smith, and S. Walfish. Composability and on-line deniability of authentication. In *Theory of Cryptography - TCC*, pages 146–162, 2009.
12. C. Dwork, M. Naor, and A. Sahai. Concurrent zero-knowledge. In *ACM Symposium on Theory of Computing - STOC*, pages 409–418, 1998.
13. R. Fiedler and C. Janson. A deniability analysis of signal’s initial handshake pqxdh. *Cryptology ePrint Archive*, 2024. <https://eprint.iacr.org/2024/741>.
14. R. Fiedler and R. Langrehr. On deniable authentication against malicious verifiers. In *International Cryptology Conference - Crypto*, pages 3–38, 2025.
15. C. Gehrman, C. J. Mitchell, and K. Nyberg. Manual authentication for wireless devices. *RSA CryptoBytes*, 7(1):24–37, 2004.
16. S. Goldberg, L. Reyzin, D. Papadopoulos, and J. Vcelak. Verifiable random functions (VRFs), internet-draft (IRTF), 2022. <https://datatracker.ietf.org/doc/html/draft-irtf-cfrg-vrf-14>.
17. Y. Gu, S. Jarecki, P. Nazarian, and A. Rai. Security without trusted third parties: VRF-based authentication with short authenticated strings. In *Advances in Cryptology — Asiaticrypt*, 2025.
18. K. Hashimoto, S. Katsumata, and T. Wiggers. Bundled authenticated key exchange: A concrete treatment of signal’s handshake protocol and Post-Quantum security. In *34th USENIX Security Symposium*, pages 6777–6796, 2025.

19. M. Jakobsson, K. Sako, and R. Impagliazzo. Designated verifier proofs and their applications. In *Advances in Cryptology - Eurocrypt*, pages 143–154, 1996.
20. S. Katsumata, G. Niot, I. Tucker, and T. Wiggers. Comprehensive deniability analysis of signal handshake protocols: X3DH, PQXDH to fully Post-Quantum with deniable ring signatures. In *34th USENIX Security Symposium*, pages 6797–6816, 2025.
21. H. Krawczyk. HMQV: a high-performance secure Diffie-Hellman protocol. In *International Cryptology Conference - Crypto*, page 546–566, 2005.
22. S. Laur, N. Asokan, and K. Nyberg. Efficient mutual data authentication using manually authenticated strings. *Cryptology ePrint Archive*, 2005. <https://eprint.iacr.org/2005/424>.
23. S. Laur and K. Nyberg. Efficient mutual data authentication using manually authenticated strings. In *Conference on Cryptology and Network Security - CANS*, pages 90–107, 2006.
24. S. Laur and S. Pasini. Sas-based group authentication and key agreement protocols. In *Public Key Cryptography - PKC*, pages 197–213, 2008.
25. S. Lawlor and K. Lewi. WhatsApp key transparency. Real-World Cryptography Conference - RWC, 2024. <https://iacr.org/submit/files/slides/2024/rwc/rwc2024/18/slides.pdf>.
26. B. McMillion. Key transparency, internet-draft (IRTF), 2025. <https://datatracker.ietf.org/doc/draft-ietf-keytrans-architecture/>.
27. S. Micali, M. Rabin, and S. Vadhan. Verifiable random functions. In *Foundations of Computer Science - FOCS*, pages 120–130, 1999.
28. M. Naor and O. Reingold. Number-theoretic constructions of efficient pseudo-random functions. *J. ACM*, 51(2):231–262, Mar. 2004.
29. M. Shirvanian and N. Saxena. Wiretapping via mimicry: Short voice imitation man-in-the-middle attacks on crypto phones. In *Computer and Communications Security - CCS*, page 868–879, 2014.
30. M. Stadler. Publicly verifiable secret sharing. In *Advances in Cryptology - Eurocrypt*, pages 190–199, 1996.
31. N. Unger and I. Goldberg. Deniable key exchanges for secure messaging. In *Proceedings of the 22nd acm sigsac conference on computer and communications security*, pages 1211–1223, 2015.
32. N. Unger and I. Goldberg. Improved strongly deniable authenticated key exchanges for secure messaging. *Proceedings on Privacy Enhancing Technologies*, 2018.
33. N. Vatanadas, R. Gennaro, B. Ithurburn, and H. Krawczyk. On the cryptographic deniability of the signal protocol. In *International Conference on Applied Cryptography and Network Security - ACNS*, pages 188–209, 2020.
34. S. Vaudenay. Secure communications over insecure channels based on short authenticated strings. In *International Cryptology Conference - Crypto*, pages 309–326, 2005.

A Deferred Preliminaries

In this section we recall the standard cryptographic assumptions that we use.

Let $\mathbb{G}$ be a cyclic group of order p with generator g , and assume a (possibly deterministic) algorithm **ParG** which generates group description $(\mathbb{G}, p, g)$ for each κ . Throughout this work we omit **ParG** from our figures for brevity and leave $(\mathbb{G}, p, g)$ as an implicit input to all algorithms.

Definition 3. The Computational Diffie-Hellman (CDH) assumption holds in the group family defined by ParG if, for every efficient algorithm $\mathcal{A}$, there exists a negligible function ϵ s.t.

$$\Pr_{\substack{(\mathbb{G}, p, g) \leftarrow \text{ParG}(\kappa) \\ a, b \leftarrow \mathbb{Z}_p}} [\mathcal{A}(p, g, g^a, g^b) \rightarrow g^{ab}] \leq \epsilon(\kappa)$$

Definition 4. The Decisional Diffie-Hellman (DDH) assumption holds in the group family defined by ParG if, for every efficient algorithm $\mathcal{A}$, there exists a negligible function ϵ s.t.

$$\left| \Pr_{\substack{(\mathbb{G}, p, g) \leftarrow \text{ParG}(\kappa) \\ a, b \leftarrow \mathbb{Z}_p}} [\mathcal{A}(p, g, g^a, g^b, g^{ab}) \rightarrow 1] - \Pr_{\substack{(\mathbb{G}, p, g) \leftarrow \text{ParG}(\kappa) \\ a, b, c \leftarrow \mathbb{Z}_p}} [\mathcal{A}(p, g, g^a, g^b, g^c) \rightarrow 1] \right| \leq \epsilon(\kappa)$$

Definition 5. Define oracle $\text{DDH}_g : \mathbb{G}^3 \rightarrow \{0, 1\}$ as $\text{DDH}_g(A, B, C) = 1$ if and only if there exists $a, b \in \mathbb{Z}_p$ s.t. $A = g^a, B = g^b$, and $C = g^{ab}$. Then, the Gap Diffie-Hellman (GapDH) assumption holds in the group family defined by ParG if, for every efficient algorithm $\mathcal{A}$, there exists a negligible function ϵ s.t.

$$\Pr_{\substack{(\mathbb{G}, p, g) \leftarrow \text{ParG}(\kappa) \\ a, b \leftarrow \mathbb{Z}_p}} [\mathcal{A}^{\text{DDH}_g(\cdot, \cdot, \cdot)}(p, g, g^a, g^b) \rightarrow g^{ab}] \leq \epsilon(\kappa)$$

B PVRF Security and Deniability

In this section we include material deferred from Section 2, including the definition of UC functionality $\mathcal{F}_{\text{PVRF}}$, proof that protocol PECVRF realizes $\mathcal{F}_{\text{PVRF}}$, and proof of PECVRF deniability.

B.1 Security Model

In [17], the security of SAS-MA protocol VRF-SAS is proven when instantiated with a VRF that UC-realizes functionality $\mathcal{F}_{\text{VRF}}$, as introduced by [8] and refined in [1,2]. Figure 7 presents that VRF functionality $\mathcal{F}_{\text{VRF}}$ along with our new PVRF functionality $\mathcal{F}_{\text{PVRF}}$. The version of $\mathcal{F}_{\text{VRF}}$ shown in Figure 7 is that of [2], albeit with some reformulations in order to make $\mathcal{F}_{\text{VRF}}$ and $\mathcal{F}_{\text{PVRF}}$ more similar and easier to understand in relation to each other. These changes to $\mathcal{F}_{\text{VRF}}$ are mostly internal, and they do not alter the security properties of the functionality at all. We explain and justify the reformulations below, after first walking through the high-level workings of the functionality.

An honest party interacting with functionality $\mathcal{F}_{\text{PVRF}}$ is either a prover P or a verifier V . Prover P begins by executing **KeyGen**, which notifies the ideal adversary $\mathcal{A}^*$ to provide a fresh verification key vk for P . The adversary can compromise P 's key using interface **KeyComp** and can register their own verification keys using interface **MalKeyGen**. Verifier V meanwhile uses interface **PreVerify** to

The functionality interacts with a set of honest parties $\mathcal{P}$ and an adversary $\mathcal{A}^*$. It maintains table $F[\cdot, \cdot]$ whose entries are all initially $\perp$, and the following sets, all initially empty: S_{vk} for registered keys, $\mathcal{C}_{vk}$ for adversarial and compromised keys, S_{vt} for verification tokens, $\mathcal{C}_{vt}$ for compromised tokens and S_{pr} for proofs.

Key Generation. On $(\text{KeyGen}, \text{kptr})$ from any $P \in \mathcal{P}$ s.t. $(P, \text{kptr}, \cdot) \notin S_{vk}$, hand $(\text{KeyGen}, \text{kptr}, P)$ to $\mathcal{A}^*$, and on vk from $\mathcal{A}^*$ s.t. $(\cdot, \cdot, vk) \notin S_{vk}$ do the following: $S_{vk} \leftarrow S_{vk} \cup \{(P, \text{kptr}, vk)\}$, return $(\text{VerificationKey}, \text{kptr}, P, vk)$ to P

Key Compromise. On $(\text{KeyComp}, P, \text{kptr})$ from $\mathcal{A}^*$ do the following: if $\exists vk$ s.t. $(P, \text{kptr}, vk) \in S_{vk}$ then $\mathcal{C}_{vk} \leftarrow \mathcal{C}_{vk} \cup \{vk\}$

Malicious Key Generation. On $(\text{MalKeyGen}, vk)$ from $\mathcal{A}^*$ s.t. $(\cdot, \cdot, vk) \notin S_{vk}$ do: $S_{vk} \leftarrow S_{vk} \cup \{(\mathcal{A}^*, \perp, vk)\}$, $\mathcal{C}_{vk} \leftarrow \mathcal{C}_{vk} \cup \{vk\}$, return $(\text{VerificationKey}, vk)$ to $\mathcal{A}^*$

Pre-Verification. On $(\text{PreVerify}, vk, x)$ from any $V \in \mathcal{P}$, send $(\text{PreVerify}, V, vk, x)$ to $\mathcal{A}^*$, receive vt in response, set $S_{vt} \leftarrow S_{vt} \cup \{(V, vk, x, vt)\}$, and return $(\text{VerificationToken}, vt)$ to V .

Verifier Compromise. On $(\text{PreVerifyComp}, V, vt, vk, x)$ where $(V, vk, x, vt) \in S_{vt}$: set $\mathcal{C}_{vt} \leftarrow \mathcal{C}_{vt} \cup \{(V, vk, x, vt)\}$.

(P)VRF Evaluation and Proof. On $(\text{Eval}, \text{kptr}, x, vt)$ from P where $\exists vk$ s.t. $(P, \text{kptr}, vk) \in S_{vk}$, send $(\text{Eval}, P, \text{kptr}, x, vt)$ to $\mathcal{A}^*$, and on (ϕ, π) from $\mathcal{A}^*$ do:

1. assert $\exists (V, vk, x, vt) \in S_{vt}$ or $(\phi = 1)$
2. if $\exists (vk', x', vt', \pi) \in S_{pr}$, then assert $(vk', x', vt') = (vk, x, vt)$
3. if $F[vk, x] = \perp$ then set $z \leftarrow_{\$} Z$ and $F[vk, x] \leftarrow z$
4. $S_{pr} \leftarrow S_{pr} \cup \{(vk, x, vt, \pi)\}$
5. return $(\text{Evaluated}, \text{kptr}, x, vt, z, \pi)$ to P and to $\mathcal{A}^*$

Malicious (P)VRF Evaluation. On $(\text{MalEval}, vk, x)$ from $\mathcal{A}^*$ where $vk \in \mathcal{C}_{vk}$:

1. if $F[vk, x] = \perp$ then set $z \leftarrow_{\$} Z$ and $F[vk, x] \leftarrow z$
2. return $(\text{Evaluated}, vk, x, z)$ to $\mathcal{A}^*$

Verification. On $(\text{Verify}, vk, x, vt, z, \pi)$ from any $V \in \mathcal{P}$, send $(\text{Verify}, V, vk, x, vt, z, \pi)$ to $\mathcal{A}^*$, and on receiving ϕ from $\mathcal{A}^*$ do the following:

1. $f \leftarrow 1$ if $F[vk, x] = z \vee (\phi = 1 \wedge (V, vk, x, vt) \in \mathcal{C}_{vt})$
and $(vk, x, vt, \pi) \in S_{pr} \vee (\phi = 1 \wedge \neg \exists (vk', x', vt', \pi) \in S_{pr})$
and $(V, vk, x, vt) \in S_{vt} \vee (\phi = 1)$
(otherwise $f \leftarrow 0$)
2. if $f = 1$, then $S_{pr} \leftarrow S_{pr} \cup \{(vk, x, vt, \pi)\}$
3. return $(\text{Verified}, vk, x, vt, z, \pi, f)$ to V

Fig. 7: Functionalities $\mathcal{F}_{\text{VRF}}$ and $\mathcal{F}_{\text{PVRF}}$: ideal functionalities for VRF (omit shadowed text) and PVRF (include shadowed text) with range Z

create a verification token vt , which like vk is supplied by the ideal adversary. Token vt is specific to the verification key vk and function argument x on which V wishes to later verify a proof. The deniability of our PVRF protocol stems from the fact that the party who generates vt holds some secret that allows for the creation of false proofs for any statement. We therefore model the compromise of this secret via the **PreVerifyComp** interface, and this compromise implies that $\mathcal{A}^*$ can violate the soundness of any future verifications under vt .

P performs a function evaluation using interface **Eval**, and this evaluation is specific to one verification token vt under which P wishes to generate a proof. We want our model to capture PVRF schemes wherein not every vt is valid and P will simply abort if they run on an invalid vt . Therefore, we grant $\mathcal{A}^*$ the ability (via the bit ϕ) to control whether or not the prover should proceed on vt . However, $\mathcal{A}^*$ cannot claim that a vt honestly created by **PreVerify** is invalid. Function pseudorandomness is modeled by the fact that every function output is sampled from a random function, and $\mathcal{A}^*$ only sees these outputs for honestly-performed **Eval** evaluations and for adversarial **MalEval** evaluations, which are only allowed for compromised or adversarially-created keys.

Finally, V uses interface **Verify** to verify a function evaluation under a previously generated verification token vt . Verification only succeeds if π is a valid proof specifically for token vt (though $\mathcal{A}^*$ can register any fresh π as valid at the time of **Verify**). By default, verification fails automatically if vt was not created by V . However, $\mathcal{A}^*$ has the ability (via the bit ϕ) to override this behavior and allow verification to continue regardless. We generalize our model with this override so that $\mathcal{F}_{\text{PVRF}}$ is strictly a generalization of $\mathcal{F}_{\text{VRF}}$. Note that this override does not allow $\mathcal{A}^*$ to make verification succeed on statements that are false. However, if the verification token vt has been compromised, then $\mathcal{A}^*$ can actually cause successful verification of false statements, as mentioned above.

Reformulations to $\mathcal{F}_{\text{VRF}}$. As mentioned above, Figure 7 presents $\mathcal{F}_{\text{VRF}}$ with some reformulations relative to how it was shown in [2]. Here we list these changes and explain that they do not alter the security properties of the functionality at all. In this discussion we denote by $\mathcal{F}'_{\text{VRF}}$ the unaltered functionality of [2].

1. $\mathcal{F}'_{\text{VRF}}$ maintains a set S_{Eval} of (vk, x, z) tuples from every **Eval** execution. $\mathcal{A}^*$ can query this set from $\mathcal{F}'_{\text{VRF}}$ at any time via the **PastEvaluations** interface, and it is also sent to $\mathcal{A}^*$ as part of the **Verify** message. In $\mathcal{F}_{\text{VRF}}$, we remove S_{Eval} entirely and instead send every **Evaluated** message to $\mathcal{A}^*$ at the end of **Eval**.

Any $\mathcal{A}^*$ interacting with $\mathcal{F}'_{\text{VRF}}$ (and therefore expecting to see S_{Eval}) can be modified to interact with $\mathcal{F}_{\text{VRF}}$, because $\mathcal{A}^*$ has enough information to simply maintain S_{Eval} internally. In particular, the **Evaluated** message includes $(kptr, x, z)$, and the mapping from $kptr$ to vk is known by $\mathcal{A}^*$ (in fact, it is chosen by $\mathcal{A}^*$, see **KeyGen**).

Conversely, any $\mathcal{A}^*$ interacting with $\mathcal{F}_{\text{VRF}}$ (and therefore expecting to see **Evaluated**) can be modified to interact with $\mathcal{F}'_{\text{VRF}}$. At the start of **Eval**, $\mathcal{A}^*$ is sent $(kptr, x)$ and sends back π . Also, $\mathcal{A}^*$ knows whether the sent π will pass the assertion of not having been used for another (vk', x') statement. $\mathcal{A}^*$ can

immediately call **PastEvaluations** to get S_{Eval} and learn the corresponding z , and thereby create for itself the full **Evaluated** message.

2. $\mathcal{F}'_{\text{VRF}}$ allows $\mathcal{A}^*$ to perform **MalEval** on $vk \notin \mathcal{C}$ as long as an honest party has already evaluated $F[vk, x]$. $\mathcal{F}_{\text{VRF}}$ removes this option, but $\mathcal{A}^*$ can easily simulate **MalEval** calls on $vk \notin \mathcal{C}$ using the **Evaluated** messages previously received.
3. $\mathcal{F}'_{\text{VRF}}$ maintains information about previously seen proofs using two tables T and P . The former maps each (vk, x) pair to a set of valid π 's. The latter maps each valid π to the corresponding (vk, x) . $\mathcal{F}_{\text{VRF}}$ unifies this bookkeeping to set S_{pr} of (vk, x, π) tuples. This change is seen in **Eval**, **MalEval**, and **Verify**, but it is only internal to the functionality.
4. In $\mathcal{F}'_{\text{VRF}}$, **Verify** checks that $(\cdot, \cdot, vk) \in S_{pk}$ and $F[vk, x] = z$, but the former condition is implied by the latter, so $\mathcal{F}_{\text{VRF}}$ omits it.
5. $\mathcal{F}'_{\text{VRF}}$ allows $\mathcal{A}^*$ to call **Verify**. $\mathcal{F}_{\text{VRF}}$ removes this option, but $\mathcal{A}^*$ can easily simulate **Verify** calls internally. As previously noted, $\mathcal{A}^*$ already knows the entire table F of function values, and $\mathcal{A}^*$ can also keep track of S_{pr} , which is only updated in **Eval** and **Verify** with values provided by $\mathcal{A}^*$. Thus, $\mathcal{A}^*$ can predict the result f of any **Verify** query. In $\mathcal{F}'_{\text{VRF}}$, $\mathcal{A}^*$ can also use **Verify** to add proofs to S_{pr} , but $\mathcal{A}^*$ can also simulate this power with $\mathcal{F}_{\text{VRF}}$ by simply remembering what proofs should have been added and adding them later when first verified by an honest party.

Relation Between $\mathcal{F}_{\text{VRF}}$ and $\mathcal{F}_{\text{PVRF}}$. As mentioned in Section 2, $\mathcal{F}_{\text{PVRF}}$ does not capture the privacy/deniability property of PVRFs. As a result, functionality $\mathcal{F}_{\text{PVRF}}$ is a generalization of (i.e. strictly weaker than) $\mathcal{F}_{\text{VRF}}$. In other words, if protocol Π realizes $\mathcal{F}_{\text{VRF}}$ then Π also realizes $\mathcal{F}_{\text{PVRF}}$ (with the minor change of adding empty strings as vt wherever called for by the $\mathcal{F}_{\text{PVRF}}$ syntax). To show this, we describe how a Π simulator **Sim** in interaction with $\mathcal{F}_{\text{VRF}}$ can be compiled into a simulator that instead interacts with $\mathcal{F}_{\text{PVRF}}$.

1. On **(PreVerify, V, vk, x)** from $\mathcal{F}_{\text{PVRF}}$, respond with an empty string as vt .
2. On **(Eval, P, kptr, x, vt)** from $\mathcal{F}_{\text{PVRF}}$, forward that message (omitting vt) to **Sim**, and receive response π . Send $(1, \pi)$ to $\mathcal{F}_{\text{PVRF}}$. (In effect, allow **Eval** to proceed regardless of vt .)
3. On **(Verify, V, vk, x, vt, z, π)** from $\mathcal{F}_{\text{PVRF}}$, forward that message (omitting **V** and vt) to **Sim**, and receive response ϕ . Set $\phi' \leftarrow F[vk, x] = z \wedge ((vk, x, vt, \pi) \in S_{pr} \vee (\phi = 1 \wedge \neg \exists (vk', x', vt', \pi) \in S_{pr}))$, and send it to $\mathcal{F}_{\text{PVRF}}$.

B.2 Proof of Security

In this section we prove that protocol **PECVRF** (Figure 1) realizes $\mathcal{F}_{\text{PVRF}}$.

Theorem 7. *PECVRF realizes $\mathcal{F}_{\text{PVRF}}$, assuming that H_i , H_o , H_a , H_b , and H_{FS} are modeled as random oracles and that the DDH problem is hard in $\mathbb{G}$.*

Specifically, for any efficient adversary $\mathcal{A}^*$ against protocol **PECVRF**, there exists a simulator **SIM** such that no efficient environment $\mathcal{Z}$ can distinguish the

view of $\mathcal{A}^*$ interacting with the real PECVRF protocol and the view of SIM interacting with the ideal functionality $\mathcal{F}_{\text{PVRF}}$ with advantage better than $2 \cdot \text{Adv}_{\text{DDH}} + q_{H_i} \cdot q_{H_o} \cdot \text{Adv}_{\text{CDH}} + \frac{q_{\text{KeyGen}} \cdot (q_{\text{KeyGen}} + q_{H_i}) + (q_{\text{PreVerify}} + 1) \cdot (q_{\text{PreVerify}} + q_{H_{FS}}) + q_{H_i} \cdot (q_{H_i} + q_{H_o})}{p}$ where there are at most q_{KeyGen} executions of **KeyGen**, there are at most $q_{\text{PreVerify}}$ executions of **PreVerify**, there are at most q_{H_i} queries to H_i , H_a , and H_b (including those that occur in **PreVerify**, **Eval**, and **Verify**), there are at most q_{H_o} queries to H_o , Adv_{DDH} is a bound on the DDH advantage of any efficient algorithm, and Adv_{CDH} is a bound on the probability that any efficient algorithm solves CDH.

Proof. For any adversary $\mathcal{A}^*$, we construct simulator $\text{SIM}_{\text{PECVRF}}$ as shown in Figure 8. Without loss of generality, we assume that $\mathcal{A}^*$ is a “dummy” adversary that merely passes messages to and from the environment $\mathcal{Z}$.

We now show that, for any efficient (i.e. PPT) $\mathcal{Z}$, the distinguishing advantage of $\mathcal{Z}$ between the real and simulated worlds is negligible. We denote the probability that $\mathcal{Z}$ outputs 1 in the interaction with the real world game or the simulated game as resp. $\mathbf{Real}_{\mathcal{Z}}[\text{PECVRF}]$ and $\mathbf{Ideal}_{\mathcal{Z}}[\mathcal{F}_{\text{PVRF}}, \text{SIM}_{\text{PECVRF}}]$. We denote the probability that $\mathcal{Z}$ outputs 1 in an interaction with security game G_i as $\text{Exp}_{\mathcal{Z}}[G_i] = \Pr[1 \leftarrow \mathcal{Z}^{G_i}(1^\kappa)]$, and we define $\Delta(G_i, G_j) = |\text{Exp}_{\mathcal{Z}}[G_i] - \text{Exp}_{\mathcal{Z}}[G_j]|$. The argument proceeds by a series of game changes, starting with game G_0 s.t. $G_0 = \mathbf{Real}_{\mathcal{Z}}[\text{PECVRF}]$ and ending with game G_8 s.t. $\text{Exp}_{\mathcal{Z}}[G_8] = \mathbf{Ideal}_{\mathcal{Z}}[\mathcal{F}_{\text{PVRF}}, \text{SIM}_{\text{PECVRF}}]$. We must show that $\Delta(G_0, G_8)$ is negligible.

Game G_0 (the real world): The distinguisher $\mathcal{Z}$ interacts with PECVRF (Figure 1) in the role of the honest parties and in the role of the adversary. H_i , H_o , H_a , H_b , and H_{FS} are true random oracles.

$$\text{Exp}_{\mathcal{Z}}[G_0] = \mathbf{Real}_{\mathcal{Z}}[\text{PECVRF}]$$

Game G_1 (KeyGen never collides): G_1 is G_0 with only one change: after sampling a random vk in **KeyGen**, abort if vk was previously sampled by another **KeyGen** instance or if a $H_i(\cdot, vk)$, $H_a(\cdot, vk)$, or $H_b(\cdot, vk)$ query was previously made.

The probability of a collision is bounded by a simple birthday bound. If no collision occurs, G_0 and G_1 are identical.

$$\Delta(G_0, G_1) \leq \frac{q_{\text{KeyGen}} \cdot (q_{\text{KeyGen}} + q_{H_i})}{p}$$

Game G_2 (H_i , H_a , and H_b outputs are correlated): G_2 is G_1 with the following changes:

- Maintain two tables $T[\cdot, \cdot]$ and $R[\cdot, \cdot]$ (initially all empty).
- On fresh query $H_i(vk, x)$, $H_a(vk, x)$, or $H_b(vk, x)$, sample $T[vk, x], R[vk, x] \leftarrow_{\$} \mathbb{Z}_p$. Program $H_i(vk, x) := g^{T[vk, x]}$ and $H_a(vk, x) := vk^{R[vk, x]}$. If vk was previously sampled and output by **KeyGen**, then program $H_b(vk, x) := vk^{R[vk, x] \cdot T[vk, x]}$, otherwise, sample random $H_b(vk, x) \leftarrow_{\$} \mathbb{G}$ as before.

Initialize empty tables $T[\cdot, \cdot]$, $R[\cdot, \cdot]$ and empty sets S_{vk} , $\mathcal{C}_{vk}$, S_{vt} , S_{eval} .

Sample $T[vk, x]$, $R[vk, x] \leftarrow_{\$} \mathbb{Z}_p$ whenever first referenced, and:

- abort if the same $T[vk, x]$ was previously sampled for a different (vk', x')
- abort if there was a previous query $H_o(w)$ s.t. $w = vk^{T[vk, x]}$

On (KeyGen, kptr, P) from $\mathcal{F}_{\text{PVRF}}$:

- sample $k \leftarrow_{\$} \mathbb{Z}_p$ and set $vk := g^k$
- abort if $\exists(\cdot, \cdot, vk) \in S_{vk}$ or there was previous query $(vk, \cdot)$ to H_i , H_a , or H_b
- set $S_{vk} \leftarrow S_{vk} \cup \{(kptr, k, vk)\}$ and return vk to $\mathcal{F}_{\text{PVRF}}$

On (PreVerify, V, vk, x) from $\mathcal{F}_{\text{PVRF}}$:

- sample $\alpha, \beta \leftarrow_{\$} \mathbb{Z}_p$, and set $h := g^\alpha \cdot H_i(vk, x)^\beta$ and $h' := H_a(vk, x)^\alpha \cdot H_b(vk, x)^\beta$
- sample $c, s_1, s_2 \leftarrow_{\$} \mathbb{Z}_p$ and set $\tau = (c, (s_1, s_2))$
- compute $\eta := g^{s_1} \cdot H_i(vk, x)^{s_2} \cdot h^c$ and $\eta' := H_a(vk, x)^{s_1} \cdot H_b(vk, x)^{s_2} \cdot h'^c$
- program $H_{FS}(vk, x, h, h', \eta, \eta') := c$
- set $vt := (h, h', \tau)$ and $S_{vt} \leftarrow S_{vt} \cup \{(V, vk, x, vt, \alpha, \beta)\}$, and return vt to $\mathcal{F}_{\text{PVRF}}$

On (Eval, P, kptr, x, vt) from $\mathcal{F}_{\text{PVRF}}$:

- parse $(h, h', \tau) \leftarrow vt$ and $(c, (s_1, s_2)) \leftarrow \tau$
- set $\eta := g^{s_1} \cdot H_i(vk, x)^{s_2} \cdot h^c$ and $\eta' := H_a(vk, x, h)^{s_1} \cdot H_b(vk, x, h)^{s_2} \cdot h'^c$, and if $c \neq H_{FS}(vk, x, h, h', \eta, \eta')$, then return $(\phi, \pi) := (0, \perp)$ to $\mathcal{F}_{\text{PVRF}}$
- otherwise, set $w := vk^{T[vk, x]}$ and $v := (h')^{1/R[vk, x]}$, and return $(\phi, \pi) := (1, (w, v))$ to $\mathcal{F}_{\text{PVRF}}$

On (Evaluated, kptr, x, vt, z, π) from $\mathcal{F}_{\text{PVRF}}$:

- retrieve $(kptr, k, vk) \in S_{vk}$ and set $S_{eval} \leftarrow S_{eval} \cup \{(vk, x, z)\}$

On (Verify, V, vk, x, vt, z, π) from $\mathcal{F}_{\text{PVRF}}$:

- if not $\exists(V, vk, x, vt, \cdot, \cdot) \in S_{vt}$, then return $\phi := 0$ to $\mathcal{F}_{\text{PVRF}}$
- parse $(w, v) \leftarrow \pi$, and set $f := 1$ if $z = H_o(w)$ and $w = vk^{T[vk, x]}$; otherwise, set $f := 0$
- return $\phi := f$ to $\mathcal{F}_{\text{PVRF}}$

On fresh query (vk, x) to H_i , H_a , or H_b , program:

- $H_i(vk, x) := g^{T[vk, x]}$, $H_a(vk, x) := vk^{R[vk, x]}$, and $H_b(vk, x) := vk^{T[vk, x] \cdot R[vk, x]}$

On fresh query $H_o(w)$:

- find (vk, x) s.t. $w = vk^{T[vk, x]}$ (if it exists in T)
- if no such (vk, x) exists, then program $H_o(w) \leftarrow_{\$} Z$ and return
- if $\exists(\cdot, \cdot, vk) \in S_{vk}$, then retrieve $(vk, x, z) \in S_{eval}$ (abort if it doesn't exist), program $H_o(w) := z$, and return
- if $vk \notin \mathcal{C}_{vk}$, then send (MalKeyGen, vk) to $\mathcal{F}_{\text{PVRF}}$ and set $\mathcal{C}_{vk} \leftarrow \mathcal{C}_{vk} \cup \{vk\}$
- send (MalEval, vk, x) to $\mathcal{F}_{\text{PVRF}}$, receive z in response, and program $H_o(w) := z$

Program H_{FS} at random, except when programmed by PreVerify (see above)

On compromise of party P's record (kptr, $-$, vk):

- send (KeyComp, P, kptr) to $\mathcal{F}_{\text{PVRF}}$, retrieve $(kptr, k, vk) \in S_{vk}$, and send $(kptr, k, vk)$ to $\mathcal{A}^*$

On compromise of party V's record $(vt, vk, x, -, -)$:

- send (PreVerifyComp, V, vt, vk, x) to $\mathcal{F}_{\text{PVRF}}$, retrieve $(V, vk, x, vt, \alpha, \beta) \in S_{vt}$, and send $(vt, vk, x, \alpha, \beta)$ to $\mathcal{A}^*$

Fig. 8: SIM_{PECVRF}: Simulator for protocol PECVRF

Given an environment $\mathcal{Z}$ that can distinguish G_1 from G_2 , we construct a reduction $\mathcal{R}$ that can solve DDH. The input to $\mathcal{R}$ is tuple $(A, B, C) \in \mathbb{G}^3$. $\mathcal{R}$ emulates game G_1 to $\mathcal{Z}$ with the following modification. On fresh query $H_i(vk, x)$, $H_a(vk, x)$, or $H_b(vk, x)$ where vk was previously sampled and output by KeyGen, do:

- Sample $\delta, \mu, \nu \leftarrow_{\$} \mathbb{Z}_p$.
- Retrieve k s.t. $vk = g^k$.
- Program $H_i(vk, x) := A^\delta$, $H_a(vk, x) := (g^\mu \cdot B^\nu)^k$, and $H_b(vk, x) := (A^\mu \cdot C^\nu)^{\delta \cdot k}$.

Here we apply the well-known DDH randomized self-reduction [30,28]. If (A, B, C) is a Diffie-Hellman tuple, then $(A^\delta, g^\mu \cdot B^\nu, (A^\mu \cdot C^\nu)^\delta)$ is a random Diffie-Hellman tuple. If not, then it is a uniformly random tuple. If (A, B, C) is not a Diffie-Hellman tuple, then the random oracles are programmed independently at random, so the view of $\mathcal{Z}$ is identical to that in G_1 . If (A, B, C) is a Diffie-Hellman tuple, then (H_i, H_a, H_b) is a random Diffie-Hellman tuple for every (vk, x) , so the view of $\mathcal{Z}$ is identical to that in G_2 .

Reduction $\mathcal{R}$ outputs 1 iff $\mathcal{Z}$ outputs 1. Thus, the advantage of $\mathcal{Z}$ at distinguishing between G_1 and G_2 is equal to the advantage of $\mathcal{R}$ in the DDH game, which is $\leq \text{Adv}_{\text{DDH}}$.

$$\Delta(G_1, G_2) \leq \text{Adv}_{\text{DDH}}$$

Game G_3 (Eval uses trapdoors): G_3 is G_2 with the following changes to Eval after verifying vt :

- Sample $T[vk, x], R[vk, x] \leftarrow_{\$} \mathbb{Z}_p$ if they are undefined.
- Instead of computing $w := H_i(vk, x)^k$, compute $w := vk^{T[vk, x]}$.
- Instead of computing $v := h^k$, compute $v := (h')^{1/R[vk, x]}$.

In G_3 , Eval uses trapdoors instead of secret keys to compute its responses. Due to the programming of H_i , w is unconditionally identical between G_2 and G_3 . If there exists $\alpha, \beta \in \mathbb{Z}_p$ s.t. $h = g^\alpha \cdot H_i(vk, x)^\beta$ and $h' = H_a(vk, x)^\alpha \cdot H_b(vk, x)^\beta$ (as NIZK τ is meant to ensure), then $(h')^{1/R[vk, x]} = (H_a(vk, x)^\alpha \cdot H_b(vk, x)^\beta)^{1/R[vk, x]} = vk^\alpha \cdot vk^{T[vk, x] \cdot \beta} = (g^\alpha \cdot H_i(vk, x)^\beta)^k = h^k$, i.e. v is identical between G_2 and G_3 . Thus, G_2 and G_3 are identical unless there is a break in the soundness of a vt NIZK τ .

Consider algorithm $\mathcal{B}$, which on input $c_1, \dots, c_{q_{\text{HFS}}} \in \mathbb{Z}_p$ executes $\mathcal{Z}^{G_3}$, programming each H_{FS} query using its inputs in succession. $\mathcal{B}$ monitors every $vt = (h, h', \tau)$ submitted by $\mathcal{Z}$ to Eval. $\mathcal{B}$ parses and verifies NIZK $\tau = (c, (s_1, s_2))$ by computing $\eta := g^{s_1} \cdot H_i(vk, x)^{s_2} \cdot h^c$ and $\eta' := H_a(vk, x)^{s_1} \cdot H_b(vk, x)^{s_2} \cdot h'^c$ and then checking that $c = \text{H}_{\text{FS}}(vk, x, h, h', \eta, \eta')$. If verification passes, then $\mathcal{B}$ checks whether there exists $\alpha, \beta \in \mathbb{Z}_p$ s.t. $h = g^\alpha \cdot H_i(vk, x)^\beta$ and $h' = H_a(vk, x)^\alpha \cdot H_b(vk, x)^\beta$. If not (i.e. soundness is broken), then $\mathcal{B}$ immediately halts and outputs (j, τ) , where $(vk, x, h, h', \eta, \eta')$ was the j th query to H_{FS} (and

therefore $c = c_j$). If $\mathcal{Z}^{\mathsf{G}_3}$ ends without triggering this event, then $\mathcal{B}$ halts in a failure.⁴

When run on random $c_1, \dots, c_{q_{\text{HFS}}} \leftarrow_{\$} \mathbb{Z}_p$, $\mathcal{B}$ succeeds iff there is a break in the soundness of a vt NIZK τ in game G_3 . We can use the forking lemma [3] to upper-bound the probability $\text{acc}_{\mathcal{B}}$ of this event. In particular, the forking lemma compiles $\mathcal{B}$ into an algorithm $\mathcal{F}(\mathcal{B})$ that either fails or outputs (τ, τ^*) where $\tau = (c, (s_1, s_2))$ and $\tau^* = (c^*, (s_1^*, s_2^*))$ are verifying NIZKs output by two different executions of $\mathcal{B}$ with the same $(vk, x, h, h', \eta, \eta')$ but different $c \neq c^*$. If $\mathcal{F}(\mathcal{B})$ succeeds, then we can extract witness (α, β) from (τ, τ^*) . In particular, $\alpha = (s_1 - s_1^*)/(c^* - c)$ and $\beta = (s_2 - s_2^*)/(c^* - c)$. However, $\mathcal{B}$ is defined to only output NIZKs for which no such α, β exists. Thus, the success of $\mathcal{F}(\mathcal{B})$ is a contradiction, and its probability $\text{acc}_{\mathcal{F}(\mathcal{B})} = 0$. The forking lemma therefore implies that $\text{acc}_{\mathcal{B}} \leq q_{\text{HFS}}/p$.

$$\Delta(\mathsf{G}_2, \mathsf{G}_3) \leq \frac{q_{\text{HFS}}}{p}$$

Game G_4 (honest function outputs are unpredictable): G_4 is G_3 with only one change: abort if the environment ever queries $\text{H}_o(vk^{T[vk, x]})$ for any vk, x s.t. some honest party P performed $(\text{KeyGen}, \text{kptr})$ that outputted vk and P has not performed $(\text{Eval}, \text{kptr}, x, vt)$ (that didn't fail due to invalid vt). Clearly G_3 and G_4 are identical unless this abort occurs (denote the event of this abort as E).

To upper-bound the probability of E in G_4 , we first define a modified game G'_4 :

- At the start of execution, sample $i \leftarrow_{\$} [q_{\text{H}_i}]$.
- For the i th $(\hat{vk}, \hat{x})$ queried to $\text{H}_i, \text{H}_a, \text{H}_b$, or Eval s.t. $\hat{vk}$ was previously sampled by some honest party P performing $(\text{KeyGen}, \hat{\text{kptr}})$, program $\text{H}_i(\hat{vk}, \hat{x})$, $\text{H}_a(\hat{vk}, \hat{x}), \text{H}_b(\hat{vk}, \hat{x}) \leftarrow_{\$} \mathbb{G}$ (rather than sampling and using trapdoors T, R as in G_4).
- If, after $(\text{KeyGen}, \hat{\text{kptr}})$, P performs $(\text{Eval}, \hat{\text{kptr}}, \hat{x}, vt)$, then compute $w := \text{H}_i(\hat{vk}, \hat{x})^{\hat{k}}$ and $v := h^{\hat{k}}$ (rather than using the T, R trapdoors as in G_4).
- Trigger abort E if the environment ever queries $\text{H}_o(\text{H}_i(\hat{vk}, \hat{x})^{\hat{k}})$ and P has not performed $(\text{Eval}, \hat{\text{kptr}}, \hat{x}, vt)$ (that didn't fail due to invalid vt).

We construct a DDH reduction $\mathcal{R}_{\text{DDH}}$ to show that E occurs with near-equal probability in G_4 and G'_4 . The input to $\mathcal{R}_{\text{DDH}}$ is tuple $(A, B, C) \in \mathbb{G}^3$. $\mathcal{R}_{\text{DDH}}$ emulates game G'_4 to $\mathcal{Z}$, except that it programs $\text{H}_i(\hat{vk}, \hat{x}) := A$, $\text{H}_a(\hat{vk}, \hat{x}) := B$, and $\text{H}_b(\hat{vk}, \hat{x}) := C$. If (A, B, C) is a Diffie-Hellman tuple, then the view of $\mathcal{Z}$ is identical to that in G_4 . If (A, B, C) is not a Diffie-Hellman tuple, then the view of $\mathcal{Z}$ is identical to that in G'_4 . Reduction $\mathcal{R}_{\text{DDH}}$ outputs 1 iff event E occurs. Thus, $|\Pr[E \leftarrow \mathcal{Z}^{\mathsf{G}_4}] - \Pr[E \leftarrow \mathcal{Z}^{\mathsf{G}'_4}]| \leq \text{Adv}_{\text{DDH}}$.

Given an environment $\mathcal{Z}$ that can trigger E in G'_4 , we construct a reduction $\mathcal{R}_{\text{CDH}}$ that can solve CDH. The input to $\mathcal{R}_{\text{CDH}}$ is $(A, B) \in \mathbb{G}^2$. $\mathcal{R}_{\text{CDH}}$ emulates game G'_4 with the following modifications:

⁴ $\mathcal{B}$ is not an efficient algorithm, but this is not of importance to our argument. For intuition, recall that the soundness of a Σ protocol is statistical, not computational.

- At the start of execution, after sampling $i \leftarrow_{\$} [q_{H_i}]$, also sample $j \leftarrow_{\$} [q_{H_o}]$.
- If, after $(\text{KeyGen}, \text{kptr})$, P performs $(\text{Eval}, \text{kptr}, \hat{x}, vt)$ (that doesn't fail due to invalid vt), then immediately halt in failure.
- In every execution of KeyGen , sample $s \leftarrow_{\$} \mathbb{Z}_p$ and set $vk := A^s$. (Notice that, in G'_4 , secret key k is not used.) Denote by $\hat{s}$ the value sampled by the KeyGen instance that outputs $\hat{vk}$.
- Program $H_i(v\hat{k}, \hat{x}) := B$.
- Upon the j th query $H_o(w)$ to H_o , immediately output $w^{1/\hat{s}}$ and halt.

If event E occurs in game G'_4 on indices i, j (meaning that the j th query to H_o is $H_o(H_i(v\hat{k}, \hat{x})^{\hat{k}})$), then the view of $\mathcal{Z}$ in $\mathcal{R}_{\text{CDH}}$ is identical to that in G'_4 and $\mathcal{R}_{\text{CDH}}$ wins CDH. Thus, $\Pr[E \leftarrow \mathcal{Z}^{G'_4}] / (q_{H_i} \cdot q_{H_o}) \leq \text{Adv}_{\text{CDH}}$. Combining our results, we reach a bound on the probability of E in G_4 .

$$\Delta(G_3, G_4) \leq \text{Adv}_{\text{DDH}} + q_{H_i} \cdot q_{H_o} \cdot \text{Adv}_{\text{CDH}}$$

Game G_5 (PreVerify NIZKs are simulated): G_5 is G_4 with the following changes to how τ is computed in PreVerify (after computing h, h' as before):

- Sample $c, s_1, s_2 \leftarrow_{\$} \mathbb{Z}_p$ and set $\tau = (c, (s_1, s_2))$.
- Compute $\eta := g^{s_1} \cdot H_i(vk, x)^{s_2} \cdot h^c$ and $\eta' := H_a(vk, x)^{s_1} \cdot H_b(vk, x)^{s_2} \cdot h'^c$.
- Program $H_{FS}(vk, x, h, h', \eta, \eta') := c$.

G_4 and G_5 are identically distributed as long as this NIZK simulation succeeds. However, it can fail if $H_{FS}(vk, x, h, h', \eta, \eta')$ was already queried by the environment or programmed by a previous instance of PreVerify . Since the input is randomly sampled by PreVerify , the probability of this event is bounded by a simple birthday bound.

$$\Delta(G_4, G_5) \leq \frac{q_{\text{PreVerify}} \cdot (q_{\text{PreVerify}} + q_{H_{FS}})}{p}$$

Game G_6 (Verify is perfectly sound): G_6 is G_5 with the following changes to how f is computed in $(\text{Verify}, vk, x, vt, z, \pi)$:

- Parse $(w, v) \leftarrow \pi$.
- If $z \neq H_o(w)$ then return $f := 0$.
- Otherwise, do not check $v = vk^\alpha \cdot w^\beta$ as before. Instead, set $f := 1$ iff $w = vk^{T[vk, x]}$.

G_5 and G_6 are identical unless $\mathcal{Z}$ is able to violate the soundness of π . In particular, define E as the event that (for some instance of Verify) $v = vk^\alpha \cdot w^\beta$ but there does not exist k s.t. $vk = g^k$ and $w = H_i(vk, x)^k$. Since $H_i(vk, x) = g^{T[vk, x]}$, the latter condition is equivalently written as $w \neq vk^{T[vk, x]}$. The security of SPHFs ensures that E is negligibly likely.

Consider algorithm $\mathcal{B}$, which on input $\beta_1, \dots, \beta_{q_{\text{PreVerify}}} \in \mathbb{Z}_p$ executes $\mathcal{Z}^{G_6}$ with the following modifications:

- In the j th instance of **PreVerify**, instead of sampling $\alpha, \beta \leftarrow_{\$} \mathbb{Z}_p$, sample $\gamma \leftarrow_{\$} \mathbb{Z}_p$ and set $h := g^\gamma$ and $h' := H_a(vk, x)^\gamma$. Define (and store) $\alpha := \gamma - T[vk, x] \cdot \beta_j$ and $\beta := \beta_j$.
- If event E occurs, immediately halt and output (j, β_j) , where the relevant vt was produced by the j th instance of **PreVerify**.
- If $\mathcal{Z}^{G_6}$ ends without triggering E , then $\mathcal{B}$ halts in a failure.

When run on random $\beta_1, \dots, \beta_{q_{\text{PreVerify}}} \leftarrow_{\$} \mathbb{Z}_p$, the view of $\mathcal{Z}$ is identical in G_6 and in $\mathcal{B}$ (recall from game G_2 that (H_i, H_a, H_o) are a Diffie-Hellman tuple for every vk, x). $\mathcal{B}$ succeeds iff event E occurs in game G_6 . We can use the forking lemma [3] to upper-bound the probability $\text{acc}_{\mathcal{B}}$ of this event. In particular, the forking lemma compiles $\mathcal{B}$ into an algorithm $\mathcal{F}(\mathcal{B})$ that either fails or outputs (β, β^*) for verifying SPHF's output by two different executions of $\mathcal{B}$ with the same (vk, x, γ, w, v) but different $\beta \neq \beta^*$ (because changing $\beta_1, \dots, \beta_{q_{\text{PreVerify}}}$ across two executions of $\mathcal{B}$ does not alter the view of $\mathcal{Z}$ as long as E does not occur).

The success of $\mathcal{F}(\mathcal{B})$ implies that $v = vk^{\gamma - T[vk, x] \cdot \beta} \cdot w^\beta = vk^{\gamma - T[vk, x] \cdot \beta^*} \cdot w^{\beta^*}$. Rearranging terms, we get $w^{\beta^* - \beta} = vk^{T[vk, x] \cdot (\beta^* - \beta)}$ and (since $\beta \neq \beta^*$) $w = vk^{T[vk, x]}$. However, $\mathcal{B}$ is defined to only succeed if $w \neq vk^{T[vk, x]}$. Thus, the success of $\mathcal{F}(\mathcal{B})$ is a contradiction, and its probability $\text{acc}_{\mathcal{F}(\mathcal{B})} = 0$. The forking lemma therefore implies that $\text{acc}_{\mathcal{B}} \leq q_{\text{PreVerify}}/p$.

$$\Delta(G_5, G_6) \leq \frac{q_{\text{PreVerify}}}{p}$$

Game G_7 (H_o queries are linked to H_i queries): G_7 is G_6 with the following changes:

- After sampling $T[vk, x]$, abort if the same value was previously sampled for a different (vk', x') .
- After sampling $T[vk, x]$, abort if there was a previous query $H_o(w)$ s.t. $w = vk^{T[vk, x]}$.
- On fresh query $H_o(w)$, find (vk, x) s.t. $w = vk^{T[vk, x]}$ (if it exists in T). In this game, nothing is done with (vk, x) after it is found.

This game change uses the trapdoors in $H_i(vk, x)$ outputs to link $H_o(w)$ queries to their corresponding (vk, x) , if it exists. The two aborts guarantee that this linkage is (1) unique and (2) matches are not retroactively created by coincidence.

The probability of an abort is bounded by a simple birthday bound. If no abort occurs, G_6 and G_7 are identical.

$$\Delta(G_6, G_7) \leq \frac{q_{H_i} \cdot (q_{H_i} + q_{H_o})}{p}$$

Game G_8 (the simulated world): The change from G_7 to G_8 is purely conceptual. All interactions between the game and honest parties now match the

code of $\mathcal{F}_{\text{PVRF}}$, so one can imagine the monolithic simulator now cleanly split into two components $\mathcal{F}_{\text{PVRF}}$ and $\text{SIM}_{\text{PECVRF}}$.

$$\begin{aligned}\Delta(\mathbb{G}_7, \mathbb{G}_8) &= 0 \\ \text{Exp}_{\mathcal{Z}}[\mathbb{G}_8] &= \text{Ideal}_{\mathcal{Z}}[\mathcal{F}_{\text{PVRF}}, \text{SIM}_{\text{PECVRF}}]\end{aligned}$$

Summing up the distinguishing advantage bounds for each incremental game change yields an overall bound on $\mathcal{Z}$'s distinguishing advantage between the real and simulated worlds.

$$\begin{aligned}\Delta(\mathbb{G}_0, \mathbb{G}_8) \leq & 2 \cdot \text{Adv}_{\text{DDH}} + q_{\text{H}_i} \cdot q_{\text{H}_o} \cdot \text{Adv}_{\text{CDH}} \\ & + \frac{q_{\text{KeyGen}} \cdot (q_{\text{KeyGen}} + q_{\text{H}_i}) + (q_{\text{PreVerify}} + 1) \cdot (q_{\text{PreVerify}} + q_{\text{H}_{\text{FS}}}) + q_{\text{H}_i} \cdot (q_{\text{H}_i} + q_{\text{H}_o})}{p}\end{aligned}$$

$p = |\mathbb{G}|$ is an exponential function of the security parameter. If $\mathcal{Z}$ is efficient, then q_{KeyGen} , $q_{\text{PreVerify}}$, q_{H_i} , q_{H_o} , and $q_{\text{H}_{\text{FS}}}$ are polynomial functions of the security parameter. Furthermore, Adv_{DDH} and Adv_{CDH} are negligible functions of the security parameter under the DDH assumption. Thus, the distinguishing advantage of any efficient $\mathcal{Z}$ between the real and simulated worlds is negligible.

Protocol PECVRF realizes $\mathcal{F}_{\text{PVRF}}$.

B.3 Proof of Deniability

In this section we prove Theorems 1 and 2, i.e. standard and strong deniability of protocol PECVRF .

Proof (Theorem 1).

We define an algorithm FakeEval which on input (vk, vt, vts, x) creates an indistinguishable (z, π) . Parse $(\alpha, \beta) \leftarrow vts$ and sample $w \leftarrow_{\S} \mathbb{G}$ and sets $v = vk^{\alpha} \cdot w^{\beta}$. Output $(\text{H}_o(w), (w, v))$. Since this is how v is constructed in the algorithm for regular verification when in possession of the verification token secret this v trivially passes the verification algorithm.

Let Game_0 be the game when $b = 0$ in $\text{Game}_{\text{D,PVRF}}^{\text{Den}}$ and Game_1 be the case when $b = 1$. Consider a reduction to DDH which on challenge (A, B, C) sets vk as A . When $\mathcal{A}$ queries H_i on input (vk, x) (or after $\mathcal{A}$ outputs x without querying H_i), sample random field elements $\mu, \nu \xleftarrow{\S} \mathbb{Z}_p$ and program the random oracle at $\text{H}_i(A, x) \leftarrow g^{\mu} \cdot B^{\nu}$. Then, set $w = A^{\mu} \cdot C^{\nu}$ and $v = A^{\alpha} \cdot (A^{\mu} \cdot C^{\nu})^{\beta}$. Finally, construct z and π as in Fig. 1.

Here we apply the well-known DDH randomized self-reduction [30,28]. If (A, B, C) is a Diffie-Hellman tuple, then $(A, g^{\mu} \cdot B^{\nu}, A^{\mu} \cdot C^{\nu})$ is a random Diffie-Hellman tuple starting with A , and our reduction yields w and v as if $b = 0$ and $\text{sk} = \log_g(A)$. If not, then it is a uniformly random tuple starting with A , and our reduction yields w and v that are distributed identically to the w and v computed by FakeEval when $b = 1$.

$$\text{Adv}_{\text{D,PECVRF}}^{\text{Den}} \leq \text{Adv}_{\text{DDH}}$$

Proof (Theorem 2). We define an algorithm **FakeEval** which on interaction with any algebraic $\mathcal{A}$ creates an indistinguishable transcript compared to a real execution of **Eval** unless $\mathcal{A}$ can break DDH in $\mathbb{G}$. **FakeEval** runs $\mathcal{A}$ internally and retrieves outputs $vt = (h, h', \tau)$ and message x . Since $\mathcal{A}$ is algebraic they must also output a vector of field elements which defines the output group element as a combination of all the group elements seen thus far. Thus, on top of verifying the regular Fiat-Shamir NIZK using τ , **FakeEval** can directly check whether $h = g^\alpha \cdot H_i(vk, x)^\beta$ and $h' = H_a(vk, x)^\alpha \cdot H_b(vk, x)^\beta$ for the same α and β . If so, it samples $w \leftarrow_{\S} \mathbb{G}$ and sets $v = vk^\alpha \cdot w^\beta$. Output $(H_o(w), (w, v))$. Soundness of the NIZK proof of existence of α and β for h and h' ensures that an honest person would abort in the case where **FakeEval** $_{\mathcal{A}}$ cannot extract correct α and β from the vector output by the algebraic adversary except with negligible probability. In this case **FakeEval** also aborts and does not construct a response matching the real transcript. Since this is how v is constructed in the algorithm for regular verification when in possession of the verification token secret this v trivially passes the regular verification check.

Using the same DDH randomized self-reduction as Theorem 1 we get,

$$\text{Adv}_{\mathcal{D}, \mathcal{A}, \text{PECVRF}}^{\text{SDen}} \leq \text{Adv}_{\text{DDH}}$$

C PVRF-SAS Security

In this section we include material deferred from Section 3, including the definition of SAS-MA security, and the proof that protocol PVRF-SAS is a secure SAS-MA scheme.

C.1 Security Model

We model SAS-MA security of scheme Π via a security game $\text{Game}_{\mathcal{A}, \Pi}^{\text{SASma}}$ shown in Figure 9. This security game is a variant of SAS-MA security defined in [17] for *groups*, specialized to the case of 2 parties and asymmetric 2-flow protocols where only responder party P_2 holds a long-term reusable key. Since the model involves only two parties we omit adaptive compromise of SAS channels, but we expand the attack conditions by allowing compromise of the secrets used by either party on the attacked session, as long as the compromise occurs after the attacked session terminates. In more details, the security game allows adversary $\mathcal{A}$ to trigger parties P_1, P_2 to execute the SAS-MA protocol repeatedly, each time on arbitrary inputs m_1, m_2 except that in each instance P_2 must be invoked on a *fresh* message m_2 .⁵ On any protocol instance $\mathcal{A}$ can deliver arbitrary protocol messages to parties, but it cannot change the SAS values exchanged in that instance. The adversary $\mathcal{A}$ *wins* the game if it fixes some protocol instance before its execution and on that instance it holds that $m_1 \neq m_2$ but $\text{sas}_1 = \text{sas}_2$.

⁵ An honest party must assure this e.g. by prepending the message with a nonce. In our application in Section 5, m is fresh because it is a key exchange transcript.

$\text{Game}_{\mathcal{A}, \Pi}^{\text{SASma}}(\kappa):$	
1. for $i = 1, 2$ and all sid , set $\text{phase}_i^{\text{sid}}$ to 0 and $\text{m}_i^{\text{sid}}, \text{state}_i^{\text{sid}}, \text{sas}_i^{\text{sid}}$ to $\perp$ 2. set ATK to $\perp$, $\text{used}, \text{comp}_1$ to $\emptyset$, and comp_2 to F 3. set $(k, vk) \leftarrow \text{PreProc}(1^\kappa)$ 4. $\text{sid} \leftarrow \mathcal{A}^{\Pi_1, \Pi_2, \Pi_3, \text{FixTarget}, \text{Comp}_1, \text{Comp}_2}(vk)$ 5. assert $\text{sid} = \text{ATK} \wedge \text{phase}_1^{\text{sid}} = 2 \wedge \text{phase}_2^{\text{sid}} = 1$ 6. if $\text{m}_1^{\text{sid}} \neq \text{m}_2^{\text{sid}} \wedge \text{sas}_1^{\text{sid}} = \text{sas}_2^{\text{sid}}$ then return 1 else return 0	
$\Pi_1(\text{sid}, vk^*, m):$ assert $\text{phase}_1^{\text{sid}} = 0$ $(pm_1, \text{st}_1^{\text{sid}}) \leftarrow \text{Init}(vk^*, m)$ $\text{phase}_1^{\text{sid}} \leftarrow 1, \text{m}_1^{\text{sid}} \leftarrow m$ return pm_1	$\Pi_3(\text{sid}, pm_2^*):$ assert $\text{phase}_1^{\text{sid}} = 1$ $\text{sas}_2^{\text{sid}} \leftarrow \text{InitFin}(\text{st}_1^{\text{sid}}, pm_2^*)$ $\text{phase}_2^{\text{sid}} \leftarrow 2$ return $\text{sas}_2^{\text{sid}}$
$\Pi_2(\text{sid}, m, pm_1^*):$ assert $\text{phase}_2^{\text{sid}} = 0$ and $m \notin \text{used}$ $(pm_2, \text{sas}_2^{\text{sid}}) \leftarrow \text{Resp}(k, m, pm_1^*)$ $\text{phase}_2^{\text{sid}} \leftarrow 1, \text{m}_2^{\text{sid}} \leftarrow m$ $\text{used} \leftarrow \text{used} \cup \{m\}$ return $(pm_2, \text{sas}_2^{\text{sid}})$	$\text{FixTarget}(\text{sid}):$ assert $\text{ATK} = \perp$ assert $\text{phase}_1^{\text{sid}} = \text{phase}_2^{\text{sid}} = 0$ $\text{ATK} \leftarrow \text{sid}$
$\text{Comp}_2():$ assert $(\text{ATK} \neq \perp \wedge \text{sas}_1^{\text{ATK}} \neq \perp \wedge \text{sas}_2^{\text{ATK}} \neq \perp)$ $\text{comp}_2 \leftarrow \text{T}$, return k	$\text{Comp}_1(\text{sid}):$ assert $\text{phase}_1^{\text{sid}} \geq 1$ assert $(\text{ATK} \neq \text{sid} \vee (\text{sas}_1^{\text{ATK}} \neq \perp \wedge \text{sas}_2^{\text{ATK}} \neq \perp))$ $\text{comp}_1 \leftarrow \text{comp}_1 \cup \{\text{sid}\}$ return st_1^{sid}

Fig. 9: Security game for SAS-MMA scheme $\Pi = (\text{PreProc}, \text{Init}, \text{Resp}, \text{InitFin})$

Game $\text{Game}_{\mathcal{A}, \Pi}^{\text{SASma}}$ models adversary $\mathcal{A}$'s interaction with honest parties via three oracles, Π_1, Π_2, Π_3 , which model resp. P_1 's execution of Init , P_2 's execution of Resp , and P_1 's execution of InitFin , and the game uses set used to ensure that each instance of P_2 runs on a *fresh* input m . The adversary can adaptively compromise P_2 's key vk (and learn the corresponding secret key k) via oracle call Comp_2 , and it can adaptively compromise P_1 's state st_1^{sid} for any protocol instance sid via oracle call Comp_1 . To win the game, however, $\mathcal{A}$ must use oracle FixTarget to commit to some protocol instance sid it hopes to attack, and (1) $\mathcal{A}$ cannot choose an instance that either party has already begun, (2) $\mathcal{A}$ cannot compromise P_2 before choosing and completing the attacked instance, and (3) $\mathcal{A}$ cannot compromise P_1 on the attacked instance until it is complete.

Definition 6. A SAS-MA scheme Π is secure if for all efficient algorithms $\mathcal{A}$ there exists a negligible function ϵ s.t. $\Pr[1 \leftarrow \text{Game}_{\mathcal{A}, \Pi}^{\text{SASma}}(1^\kappa)] \leq 1/2^d + \epsilon(\kappa)$.

C.2 Proof of Security

In this section we prove that protocol PVRF-SAS is a secure SAS-MA scheme.

Theorem 8. If PVRF realizes functionality $\mathcal{F}_{\text{PVRF}}$ (Figure 7), then protocol PVRF-SAS (Figure 3) is a secure SAS-MA scheme (Definition 6).

Proof. We argue that SAS-MA scheme PVRF-SAS is *secure* through a sequence of transformations. Throughout the transformations we fix the adversarial algorithm $\mathcal{A}_0$, and we start from $\mathbf{G}_0$, see Figure 10, which is identical to the SAS-MA security experiment $\text{Game}_{\mathcal{A}_0, \Pi}^{\text{SASma}}(1^\kappa)$ for $\Pi = \text{PVRF-SAS}$. Let $\epsilon_0 = \Pr[1 \leftarrow \mathbf{G}_0]$. Since $\mathbf{G}_0$ is identical to the SAS-MA security experiment for scheme PVRF-SAS, our goal is to show that $\epsilon_0 \leq 1/2^d + \epsilon(\kappa)$ where ϵ is negligible.

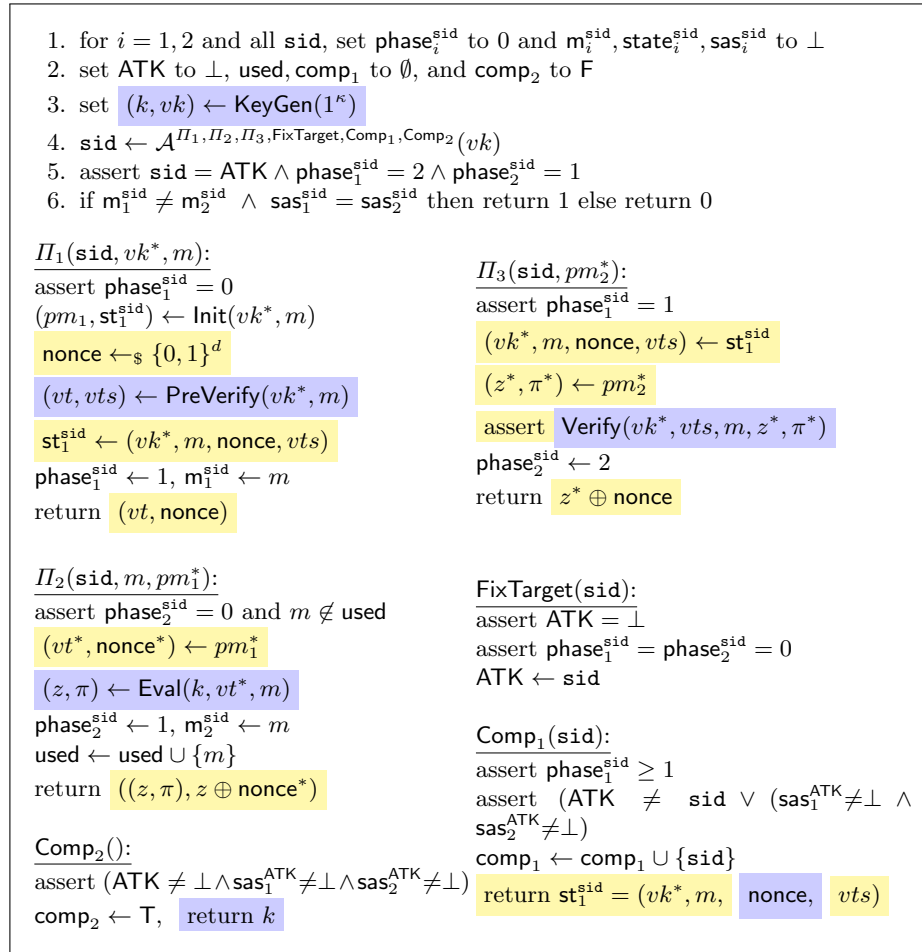


Fig. 10: Game $\mathbf{G}_0$: The security game $\text{Game}_{\mathcal{A}, \Pi}^{\text{SASma}}$, Fig. 9, instantiated with the SAS-MA scheme PVRF-SAS of Fig. 3. In yellow we show the code specific to scheme PVRF-SAS and in blue we highlight PVRF calls within that code.

Game G_0 (real world): G_0 is the security experiment $\text{Game}_{\mathcal{A}_0, \Pi}^{\text{SASma}}(1^\kappa)$, see Fig. 9, for Π instantiated as PVRF-SAS. We present this game explicitly in Figure 10, using standard font for the code that comes from the generic security game $\text{Game}_{\mathcal{A}_0, \Pi}^{\text{SASma}}$ and using yellow and blue to mark the code that pertains to the specific SAS-MA scheme PVRF-SAS, where the latter color is used whenever PVRF-SAS's code contains calls to the PVRF scheme PVRF.

1. for $i = 1, 2$ and all sid, set $\text{phase}_i^{\text{sid}}$ to 0 and $\text{m}_i^{\text{sid}}, \text{state}_i^{\text{sid}}, \text{sas}_i^{\text{sid}}$ to $\perp$ 2. set ATK, all $F[\cdot, \cdot]$ to $\perp$, $\text{used}, \text{comp}_1, S_{pr}$ to $\emptyset$, and comp_2 to $\mathbf{F}$ 3. $vk \leftarrow \mathcal{A}(1^\kappa)$ 4. $\text{sid} \leftarrow \mathcal{A}^{\Pi_1, \Pi_2, \Pi_3, \text{FixTarget}, \text{Comp}_1, \text{Comp}_2, \text{MalEval}}(vk)$ 5. assert $\text{sid} = \text{ATK} \wedge \text{phase}_1^{\text{sid}} = 2 \wedge \text{phase}_2^{\text{sid}} = 1$ 6. if $\text{m}_1^{\text{sid}} \neq \text{m}_2^{\text{sid}} \wedge \text{sas}_1^{\text{sid}} = \text{sas}_2^{\text{sid}}$ then return 1 else return 0	$\Pi_3(\text{sid}, pm_2^*)$: assert $\text{phase}_1^{\text{sid}} = 1$ $(vk^*, m, \text{nonce}, vt) \leftarrow \text{st}_1^{\text{sid}}$ $(z^*, \pi^*) \leftarrow pm_2^*$ assert $F[vk^*, m] = z^*$ or $\text{sid} \in \text{comp}_1$ if $\exists (vk', m', vt', \pi^*) \in S_{pr}$: assert $(vk', m', vt') = (vk^*, m, vt)$ $S_{pr} \leftarrow S_{pr} \cup \{(vk^*, m, vt, \pi^*)\}$ $\text{phase}_2^{\text{sid}} \leftarrow 2$ return $z^* \oplus \text{nonce}$
$\Pi_1(\text{sid}, vk^*, m, vt)$: assert $\text{phase}_1^{\text{sid}} = 0$ $(pm_1, \text{st}_1^{\text{sid}}) \leftarrow \text{Init}(vk^*, m)$ $\text{nonce} \leftarrow_{\\$} \{0, 1\}^d$ $\text{st}_1^{\text{sid}} \leftarrow (vk^*, m, \text{nonce}, vt)$ $\text{phase}_1^{\text{sid}} \leftarrow 1, \text{m}_1^{\text{sid}} \leftarrow m$ return (vt, nonce)	$\text{FixTarget}(\text{sid})$: assert $\text{ATK} = \perp$ assert $\text{phase}_1^{\text{sid}} = \text{phase}_2^{\text{sid}} = 0$ $\text{ATK} \leftarrow \text{sid}$
$\Pi_2(\text{sid}, m, pm_1^*, \pi)$: assert $\text{phase}_2^{\text{sid}} = 0$ and $m \notin \text{used}$ $(vt^*, \text{nonce}^*) \leftarrow pm_1^*$ if $\exists (vk', m', vt', \pi) \in S_{pr}$: assert $(vk', m', vt') = (vk, m, vt^*)$ if $F[vk, m] = \perp$: $F[vk, m] \leftarrow_{\\$} \{0, 1\}^d$ $z \leftarrow F[vk, m]$ $S_{pr} \leftarrow S_{pr} \cup \{(vk, m, vt^*, \pi)\}$ $\text{phase}_2^{\text{sid}} \leftarrow 1, \text{m}_2^{\text{sid}} \leftarrow m$ $\text{used} \leftarrow \text{used} \cup \{m\}$ return $((z, \pi), z \oplus \text{nonce}^*)$	$\text{Comp}_1(\text{sid})$: assert $\text{phase}_1^{\text{sid}} \geq 1$ assert $(\text{ATK} \neq \text{sid} \vee (\text{sas}_1^{\text{ATK}} \neq \perp \wedge \text{sas}_2^{\text{ATK}} \neq \perp))$ $\text{comp}_1 \leftarrow \text{comp}_1 \cup \{\text{sid}\}$
$\text{Comp}_2()$: assert $(\text{ATK} \neq \perp \wedge \text{sas}_1^{\text{ATK}} \neq \perp \wedge \text{sas}_2^{\text{ATK}} \neq \perp)$ $\text{comp}_2 \leftarrow \mathbf{T}$	$\text{MalEval}(vk^*, m)$: assert $(vk^* \neq vk \vee \text{comp}_2 = \mathbf{T})$ if $F[vk^*, m] = \perp$: $F[vk^*, m] \leftarrow_{\\$} \{0, 1\}^d$ return $F[vk^*, m]$

Fig. 11: Game G_1 : Replacing PVRF calls from G_0 with $\mathcal{F}_{\text{PVRF}}$ code.

Game G_1 (implanting $\mathcal{F}_{\text{PVRF}}$): In G_1 we replace the real-world PVRF scheme acting in G_0 with the ideal-world functionality $\mathcal{F}_{\text{PVRF}}$. In Figure 11, which shows G_1 , all the calls to the PVRF scheme made by G_0 , denoted in blue in Fig. 10, are replaced by the code of the ideal functionality $\mathcal{F}_{\text{PVRF}}$ (Figure 7), denoted in green in Fig. 11. Since the adversary in the ideal-world (of PVRF) has additional interfaces, namely malicious key generation and PVRF evaluation via `MalKeyGen` and `MalEval`, we add these interfaces in G_1 . Note that other than replacing the blue-colored code fragments in G_0 with the green-colored fragments in G_1 , the remaining code of the two games is identical.

Since adversarial interfaces change between G_0 and G_1 , the second game involves a formally different adversarial algorithm, denoted $\mathcal{A}_1$. Indeed, the transformation from G_0 to G_1 is justified by the assumption that scheme PVRF realizes the UC PVRF functionality $\mathcal{F}_{\text{PVRF}}$. Technically, what this fact implies is that for every efficient environment algorithm $\mathcal{Z}$ and adversary $\mathcal{A}_0$, there exists an efficient algorithm *simulator* Sim s.t. $\mathcal{Z}$'s view of an interaction with $\mathcal{A}_0$ and honest parties running scheme PVRF, is computationally indistinguishable from $\mathcal{Z}$'s view of an interaction with the simulator Sim and honest parties who interact with the ideal PVRF functionality $\mathcal{F}_{\text{PVRF}}$. Define $\mathcal{Z}$ as an algorithm which runs all the code of G_0 shown in Fig. 10, including adversary $\mathcal{A}_0$ as a subroutine run by $\mathcal{Z}$, except for the blue-colored fragments which correspond to honest parties' commands to PVRF. Using this notation, G_0 is an interaction between $\mathcal{Z}$ using $\mathcal{A}_0$ as a subroutine, with honest P_1, P_2 running scheme PVRF. By the assumption that PVRF realizes $\mathcal{F}_{\text{PVRF}}$, there exists a simulator Sim s.t. this interaction is indistinguishable from an interaction of the same $\mathcal{Z}$ with Sim and honest P_1, P_2 interacting with $\mathcal{F}_{\text{PVRF}}$. The transformation between G_0 and G_1 is an instance of this: (1) the adversary $\mathcal{A}_1$ used as subroutine in G_1 is a composition of adversary $\mathcal{A}_0$ and simulator Sim , (2) the green-colored fragments of G_1 correspond to $\mathcal{F}_{\text{PVRF}}$ processing P_1, P_2 calls and $\mathcal{A}_1$'s direct calls (via oracles `MalKeyGen`, `MalEval`, and `KeyComp`), and (3) the remaining code of G_1 is the environment $\mathcal{Z}$. By the assumption that PVRF securely realizes $\mathcal{F}_{\text{PVRF}}$, it follows that there is a negligible function $\varepsilon_{\text{PVRF}}$ s.t. if $\epsilon_1 = \Pr[1 \leftarrow G_1]$ then

$$\epsilon_0 \leq \epsilon_1 + \varepsilon_{\text{PVRF}}(\kappa)$$

Note that $\mathcal{A}_1$'s interfaces in G_1 are different than $\mathcal{A}_0$'s interfaces to these oracles in G_0 . In the real world, honest party P_2 does not need any additional inputs to run `PVRF.KeyGen`, but in the ideal world when P_1 generates a key via the `KeyGen` interface of $\mathcal{F}_{\text{PVRF}}$ it should supply a key identifier `kptr`. Further, $\mathcal{F}_{\text{PVRF}}$ reacts to `(KeyGen, kptr)` by sending the message `(KeyGen, kptr, P_1)` to adversary $\mathcal{A}_1$, who plays the role of the ideal world adversary $\mathcal{A}^*$ of Fig. 7, asking $\mathcal{A}_1$ to supply a fresh public key `vk`. In G_1 we short-cut this interaction by asking $\mathcal{A}_1$ to supply `vk` at the game's start. There is only one invocation of `KeyGen` in G_1 , so we simply omit the identifier `kptr` from the game.

$\mathcal{F}_{\text{PVRF}}$ on command `(PreVerify, vk, x)` from V notifies the ideal adversary who then supplies the verification token `vt`. We short-cut this interaction in G_1 by simply having $\mathcal{A}_1$ supply `vt` as part of their Π_1 oracle query.

Similarly, on command $(\text{Eval}, \text{kpctr}, x, vt)$ from $\mathcal{P}$, $\mathcal{F}_{\text{PVRF}}$ notifies the ideal adversary who then supplies the proof π . This interaction is also short-cutted in $\mathcal{G}_1$ by simply including π as part of $\mathcal{A}_1$'s Π_2 oracle query. The ideal adversary can also cause Eval to abort using bit $\phi = 0$ (if vt^* is not honestly generated), but we omit this interaction since $\mathcal{A}_1$ can equivalently just skip such calls to Π_2 .

For the same reason as in Π_2 's Eval call, we also assume that $\phi = 1$ in Π_3 's Verify call. Also, we merge $\mathcal{F}_{\text{PVRF}}$'s KeyComp (resp. PreVerifyComp) bookkeeping mechanism $\mathcal{C}_{vk}$ (resp. $\mathcal{C}_{vt}$) with our game's Comp_2 (resp. Comp_1) bookkeeping mechanism comp_2 (resp. comp_1). Since vk^* , m , and nonce are already public to $\mathcal{A}_1$ before querying Comp_1 , we remove them from the output of that oracle. We make one final short-cut by observing that a $(\text{MalKeyGen}, vk^*)$ command has no effect unless followed by at least one $(\text{MalEval}, vk^*, \cdot)$ evaluation. We therefore omit MalKeyGen with the implicit understanding that $\mathcal{A}_1$ would invoke it prior to any corresponding MalEval call.

Game $\mathcal{G}_2$ (eliminating proofs): In $\mathcal{G}_2$ we simplify $\mathcal{G}_1$ based on the observation that the adversary can always predict whether their $\mathcal{G}_1$ oracle calls will violate an “assert” statement and cause an abort, and that the adversary gains nothing by doing so. Therefore, $\mathcal{G}_2$ removes the checks for proof validity from Π_2 and instead simply assumes that correct z values and proofs have been forwarded from Π_1 . Proofs and verification tokens consequently play no role in $\mathcal{G}_2$, which allows them to be removed from Π_1 as well. We define a new adversary $\mathcal{A}_2$ who runs $\mathcal{A}_1$ as a subroutine, internally performs the assert checks that have been removed from $\mathcal{G}_2$, and only forwards those Π_1 and Π_2 calls that would not have violated those checks (with π and vt excised from the calls in order to match the $\mathcal{G}_2$ interface). $\mathcal{A}_2$ succeeds in $\mathcal{G}_2$ iff $\mathcal{A}_1$ succeeds in $\mathcal{G}_1$, i.e. $\epsilon_1 = \epsilon_2$, where we define $\epsilon_2 = \Pr[1 \leftarrow \mathcal{G}_2]$.

It remains to upper-bound the adversary's probability of success in $\mathcal{G}_2$. For the attacked session $\text{ATK} = \text{sid}$, consider the relevant oracle calls $\Pi_1(\text{sid}, vk^*, m)$, $\Pi_2(\text{sid}, m, \text{nonce}^*)$, and $\Pi_3(\text{sid}, z^*)$. FixTarget enforces that this sid was selected for attack before any of those calls, and Comp_1 and FixTarget together enforce that $\text{sid} \notin \text{comp}_1$ at the time of Π_3 . Therefore, the vk^*, m sent to Π_1 committed the adversary to the $F[\cdot, \cdot]$ position that would determine the z^* used to compute sas_1 . Since Π_1 samples nonce uniformly at random, it follows by a one-time pad argument that sas_1 is uniformly random and unpredictable to the adversary before the time of Π_1 query. On the other hand, sas_2 is determined by $z = F[vk, m]$. The freshness of m and the assurance that $\text{comp}_2 = \text{F}$ at the time of Π_2 query together guarantee that $F[vk, m]$ (and therefore sas_2) is uniformly random and unpredictable to the adversary before the time of Π_2 query. Further, if the parties run on different messages m , then the random sas_1 and sas_2 thus determined are independent of one another, so the probability that they collide is $1/2^d$. Therefore, $\epsilon_2 \leq 1/2^d$.

Overall, we have $\epsilon_0 \leq 1/2^d + \epsilon_{\text{PVRF}}(\kappa)$. PVRF-SAS is secure.

1. for $i = 1, 2$ and all sid , set $\text{phase}_i^{\text{sid}}$ to 0 and $\text{m}_i^{\text{sid}}, \text{state}_i^{\text{sid}}, \text{sas}_i^{\text{sid}}$ to $\perp$ 2. set ATK , all $F[\cdot, \cdot]$ to $\perp$, $\text{used}, \text{comp}_1$ to $\emptyset$, and comp_2 to F 3. $vk \leftarrow \mathcal{A}(1^\kappa)$ 4. $\text{sid} \leftarrow \mathcal{A}^{\Pi_1, \Pi_2, \Pi_3, \text{FixTarget}, \text{KeyComp}, \text{MalEval}}(vk)$ 5. assert $\text{sid} = \text{ATK} \wedge \text{phase}_1^{\text{sid}} = 2 \wedge \text{phase}_2^{\text{sid}} = 1$ 6. if $\text{m}_1^{\text{sid}} \neq \text{m}_2^{\text{sid}} \wedge \text{sas}_1^{\text{sid}} = \text{sas}_2^{\text{sid}}$ then return 1 else return 0	
$\Pi_1(\text{sid}, vk^*, m)$: assert $\text{phase}_1^{\text{sid}} = 0$ $(pm_1, \text{st}_1^{\text{sid}}) \leftarrow \text{Init}(vk^*, m)$ $\text{nonce} \leftarrow_{\$} \{0, 1\}^d$ $\text{st}_1^{\text{sid}} \leftarrow (vk^*, m, \text{nonce})$ $\text{phase}_1^{\text{sid}} \leftarrow 1, \text{m}_1^{\text{sid}} \leftarrow m$ return (vt, nonce)	$\Pi_3(\text{sid}, z^*)$: assert $\text{phase}_1^{\text{sid}} = 1$ $(vk^*, m, \text{nonce}) \leftarrow \text{st}_1^{\text{sid}}$ assert $F[vk^*, m] = z^*$ or $\text{sid} \in \text{comp}_1$ $\text{phase}_2^{\text{sid}} \leftarrow 2$ return $z^* \oplus \text{nonce}$
$\Pi_2(\text{sid}, m, \text{nonce}^*)$: assert $\text{phase}_2^{\text{sid}} = 0$ and $m \notin \text{used}$ if $F[vk, m] = \perp$: $F[vk, m] \leftarrow_{\$} \{0, 1\}^d$ $z \leftarrow F[vk, m]$ $\text{phase}_2^{\text{sid}} \leftarrow 1, \text{m}_2^{\text{sid}} \leftarrow m$ $\text{used} \leftarrow \text{used} \cup \{m\}$ return $(z, z \oplus \text{nonce}^*)$	$\text{FixTarget}(\text{sid})$: assert $\text{ATK} = \perp$ assert $\text{phase}_1^{\text{sid}} = \text{phase}_2^{\text{sid}} = 0$ $\text{ATK} \leftarrow \text{sid}$
$\text{Comp}_2()$: assert $(\text{ATK} \neq \perp \wedge \text{sas}_1^{\text{ATK}} \neq \perp \wedge \text{sas}_2^{\text{ATK}} \neq \perp)$ $\text{comp}_2 \leftarrow \text{T}$	$\text{Comp}_1(\text{sid})$: assert $\text{phase}_1^{\text{sid}} \geq 1$ assert $(\text{ATK} \neq \text{sid} \vee (\text{sas}_1^{\text{ATK}} \neq \perp \wedge \text{sas}_2^{\text{ATK}} \neq \perp))$ $\text{comp}_1 \leftarrow \text{comp}_1 \cup \{\text{sid}\}$
	$\text{MalEval}(vk^*, m)$: assert $(vk^* \neq vk \vee \text{comp}_2 = \text{T})$ if $F[vk^*, m] = \perp$: $F[vk^*, m] \leftarrow_{\$} \{0, 1\}^d$ return $F[vk^*, m]$

Fig. 12: Game G_2 : Removing proofs from G_1

D Deniable Authenticated Key Exchange Against Malicious Accusers

In this section we provide the full specifications of strong deniability (i.e. deniability against malicious accusers), deferred from Section 4.2 to avoid redundancy. Figures 13 and 14 are the strong deniability game $\text{Game}_{\mathcal{A}, D_1, D_2, \mathcal{H}, \mathcal{L}_{\text{leak}}, \mathcal{L}_{\text{KDC}}}^{\text{strong-ATK}}$ (parameterized by $\text{leak}, \mathcal{L}_{\text{KDC}}$, and ATK).

Definition 7. (*Deniability Against Malicious Accusers*). Let $\mathcal{N} := [N]$ for $N \in \mathbb{N}$ be the set of all users, $\mathcal{H} \subseteq \mathcal{N}$ such that $\mathcal{H} \neq \emptyset$ be the set of accused users, and $\mathcal{C} := \mathcal{N} \setminus \mathcal{H}$ be the set of accusers.

Protocol $\text{BAKE}^\dagger$ is strongly deniable with respect to leakage level $\text{leak} \in \{\text{low}, \text{med}, \text{high}\}$, KDC leakage bit $\mathcal{L}_{\text{KDC}} \in \{0, 1\}$, and locality $\text{ATK} \in \{\text{local}, \text{global}\}$ if, for any efficient accuser $(\mathcal{A}, D_1)$, there exists an efficient simulator $\text{Sim}_{\mathcal{A}, D_1} =$

```

Game $\mathcal{A}, \mathcal{D}_1, \mathcal{D}_2, \mathcal{H}, \mathcal{L}_{\text{leak}}, \mathcal{L}_{\text{KDC}}$ strong-ATK( $1^\kappa$ , mode):
 $\mathcal{C} := \mathcal{N} \setminus \mathcal{H}$ 
for user  $u \in \mathcal{N}$  do
     $(\text{ipk}_u, \text{isk}_u) \xleftarrow{\$} \text{BAKE}^\dagger.\text{IdKeyGen}(1^\kappa)$ 
     $\text{counter}_u := 0$ 
    if ( $u \in \mathcal{H}$ ) then
         $(\text{prek}_u, \text{st}_u) \xleftarrow{\$} \text{BAKE}^\dagger.\text{PreKeyBundleGen}(\text{isk}_u)$ 
if (mode = real) then
     $((\text{prek}_u)_{u \in \mathcal{C}}, \text{st}_\mathcal{A}) \xleftarrow{\$} \mathcal{A}((\text{ipk}_u)_{u \in \mathcal{N}}, (\text{isk}_u)_{u \in \mathcal{C}}, (\text{prek}_u)_{u \in \mathcal{H}})$ 
else if (mode = sim) then
     $((\text{prek}_u^*)_{u \in \mathcal{H}}, (\text{prek}_u)_{u \in \mathcal{C}}, \text{st}_{\text{Sim}}) \xleftarrow{\$} \text{SimPreK}_\mathcal{A}((\text{ipk}_u)_{u \in \mathcal{N}}, (\text{isk}_u)_{u \in \mathcal{C}}, (\text{prek}_u)_{u \in \mathcal{H}})$ 
    if  $\neg \mathcal{L}_{\text{KDC}}$  then  $(\text{prek}_u)_{u \in \mathcal{H}} \leftarrow (\text{prek}_u^*)_{u \in \mathcal{H}}$ 
 $\text{st}_\mathcal{D} \xleftarrow{\$} \mathcal{D}_1^{\mathcal{O}_{\text{ATK}}}((\text{ipk}_u, \text{prek}_u)_{u \in \mathcal{N}})$ 
if (mode = sim) then
     $\text{st}_\mathcal{A} \xleftarrow{\$} \text{SimSt}_\mathcal{A}(\text{st}_{\text{Sim}})$ 
 $\text{leak}_\mathcal{D} := \mathcal{L}_{\text{leak}}((\text{isk}_u, \text{st}_u)_{u \in \mathcal{H}})$ 
 $b \xleftarrow{\$} \mathcal{D}_2(\text{st}_\mathcal{D}, \text{leak}_\mathcal{D}, \text{st}_\mathcal{A})$ 
return  $b$ 

```

Fig. 13: Local and global $\text{BAKE}^\dagger$ deniability game for *malicious* accusers, with changes relative to BAKE deniability [20] marked in yellow.

```

Oracle  $\mathcal{O}_{\text{ATK}}(s, r)$ :


---


assert  $((s, r) \in \mathcal{N} \times \mathcal{N}) \wedge (s \neq r) \wedge ((s, r) \notin \mathcal{C} \times \mathcal{C})$ 
if  $(\text{ATK} = \text{local})$  then assert  $(s, r) \notin \mathcal{H} \times \mathcal{H}$ 
 $\text{counter}_r \leftarrow \text{counter}_r + 1, t := \text{counter}_r$ 
if  $(t > L)$  then  $t \leftarrow \perp$ 
if  $(\text{mode} = \text{real})$  then
  if  $(s \in \mathcal{H})$  then
     $(K, \rho, \text{temp}) \xleftarrow{\$} \text{BAKE}^\dagger.\text{Send}(\text{isk}_s, \text{ipk}_r, \text{prek}_{r,t})$ 
  else if  $(s \in \mathcal{C})$  then
     $(\rho, \text{st}_\mathcal{A}) \xleftarrow{\$} \mathcal{A}(\text{st}_\mathcal{A}, (s, r, t))$ 
  if  $(r \in \mathcal{H})$  then
     $(K', m, \text{sas}_r, \text{st}_r) \xleftarrow{\$} \text{BAKE}^\dagger.\text{Receive}(\text{isk}_r, \text{st}_r, \text{ipk}_s, t, \rho)$ 
  else if  $(r \in \mathcal{C})$  then
     $(m, \text{sas}_r, \text{st}_\mathcal{A}) \xleftarrow{\$} \mathcal{A}(\text{st}_\mathcal{A}, \text{ipk}_s, t, \rho)$ 
  if  $(s \in \mathcal{H})$  then
     $\text{sas}_s \leftarrow \text{BAKE}^\dagger.\text{SenderReceive}(\text{temp}, m)$ 
  else if  $(s \in \mathcal{C})$  then
     $(\text{sas}_s, \text{st}_\mathcal{A}) \leftarrow \mathcal{A}(\text{st}_\mathcal{A}, m)$ 
else if  $(\text{mode} = \text{sim})$  then
  if  $(s, r) \in \mathcal{H} \times \mathcal{C}$  then
     $(K, \rho, m, \text{sas}_s, \text{st}_{\text{Sim}}) \xleftarrow{\$} \text{SimTrans}_\mathcal{A}(\text{isk}_r, \text{st}_{\text{Sim}}, (s, r, t))$ 
  else if  $(s, r) \in \mathcal{C} \times \mathcal{H}$  then
     $(K', \rho, m, \text{sas}_r, \text{st}_{\text{Sim}}) \xleftarrow{\$} \text{SimTrans}_\mathcal{A}(\text{isk}_s, \text{st}_{\text{Sim}}, (s, r, t))$ 
     $(K', \cdot, \cdot, \cdot) \xleftarrow{\$} \text{BAKE}^\dagger.\text{Receive}(\text{isk}_r, \text{st}_r, \text{ipk}_s, t, \rho)$ 
    if  $(K' \neq \perp)$  then  $K' \leftarrow K'^*$ 
     $\text{st}_r \xleftarrow{\$} \text{SimSt}_\mathcal{H}(\text{isk}_r, \text{st}_r)$ 
  else if  $(s, r) \in \mathcal{H} \times \mathcal{H}$  then
     $(K, K', \rho, m, \text{sas}_s, \text{sas}_r, \text{st}_{\text{Sim}}) \xleftarrow{\$} \text{SimTrans}_\mathcal{A}(\perp, \text{st}_{\text{Sim}}, (s, r, t))$ 
     $\text{st}_r \xleftarrow{\$} \text{SimSt}_\mathcal{H}(\text{isk}_r, \text{st}_r)$ 
  if  $(s, r) \in \mathcal{H} \times \mathcal{C}$  then return  $(K, \rho, m, \text{sas}_s)$ 
  else if  $(s, r) \in \mathcal{C} \times \mathcal{H}$  then return  $(K', \rho, m, \text{sas}_r)$ 
  else return  $(K, K', \rho, m, \text{sas}_s, \text{sas}_r)$ 

```

Fig. 14: Oracle $\mathcal{O}_{\text{ATK}}$ for malicious ⁴⁰accusers in $\text{BAKE}^\dagger$ deniability game in Fig. 13, with changes from BAKE deniability game [20] marked in yellow.

$(\text{SimPreK}_{\mathcal{A}, D_1}, \text{SimTrans}_{\mathcal{A}, D_1}, \text{SimSt}_{\mathcal{H}}, \text{SimSt}_{\mathcal{A}, D_1})$ such that for any efficient distinguisher D_2 we have

$$\Pr \left[\text{Game}_{\mathcal{A}, D_1, D_2, \mathcal{H}, \mathcal{L}_{\text{leak}}, \mathcal{L}_{\text{KDC}}}^{\text{strong-local}}(1^\kappa, \text{real}) = 0 \right] \leq \Pr \left[\text{Game}_{\mathcal{A}, D_1, D_2, \mathcal{H}, \mathcal{L}_{\text{leak}}, \mathcal{L}_{\text{KDC}}}^{\text{strong-local}}(1^\kappa, \text{sim}) = 0 \right] + \epsilon(\kappa)$$

for some negligible function ϵ , where $\text{Game}_{\mathcal{A}, D_1, D_2, \mathcal{H}, \mathcal{L}_{\text{leak}}, \mathcal{L}_{\text{KDC}}}^{\text{strong-local}}$ is given in Figures 13 and 14.

Online Deniability and an Attack on the Model of [20]. As explained in Section 1, X3DH is not deniable in a model where the distinguisher D is allowed “online” communication with the malicious accuser $\mathcal{A}$ without any restrictions.

In the strong deniability model of [20], online communication from $\mathcal{A}$ to D is not difficult: D can simply run $\mathcal{A}$ with any counterparty, and $\mathcal{A}$ can output an arbitrary message into the resulting transcript via ρ or m . Seemingly, however, online communication is still prevented because there is no way for D to send a message to $\mathcal{A}$. Surprisingly, though, there is actually a subtle “back-channel” by which D can pass information to $\mathcal{A}$. The distinguisher can use their choice of (s, r) parties sent to oracle $\mathcal{O}_{\text{ATK}}$ as a way of encoding an arbitrary bitstring.

For example, consider an accuser $\mathcal{A}$ who, on the first key exchange session, outputs the public keys $(\text{ipk}_h, \text{prek}_h)$ of some honest party h into the session transcript. The distinguisher, after seeing these keys, forms a sender protocol message ρ and uses the next thousand (s, r) pairs to encode it. $\mathcal{A}$ reconstructs this ρ , and, on the next session, passes it to honest receiver h . This honest receiver unwittingly completes a key exchange with D , and no simulator can simulate the resulting K' .

This back-channel results in an explicit attack on the strong deniability of X3DH, in contradiction to the positive results of [20]. Katsumata et al. prove strong deniability in the GGM, assuming that $\mathcal{A}$ is a generic algorithm and therefore the simulator, by observing all generic group operations of $\mathcal{A}$, can extract the discrete logarithm of any group element output by $\mathcal{A}$ (or observe that the group element is derived from a random oracle hash and therefore its discrete logarithm is unknown to all parties). This argument is predicated on the implicit assumption that $\mathcal{A}$ has no source for group elements other than those it computes itself or derives from a random oracle hash, and it therefore fails to account for the possibility of a group element passed from D to $\mathcal{A}$ via the hidden back-channel.

We stress that this attack is highly artificial and does not reveal anything meaningful about the real deniability of X3DH. Rather, it is an attack on the model of [20] that serves to underscore the impossibility of online deniability for X3DH. Even the subtlest channel of D -to- $\mathcal{A}$ communication breaks strong deniability.

To repair the model of [20], we propose to split D into two algorithms, D_1 that directs parties to perform key exchanges via the $\mathcal{O}_{\text{ATK}}$ oracle, and D_2 that ultimately attempts to distinguish the real and simulated worlds. We modify the definition of strong deniability so that it is for every $(\mathcal{A}, D_1)$ pair that there

must exist a simulator. Essentially, we treat D_1 as an appendage of the accuser, whereas [20] treats it as an appendage of the judge.

E Proofs of X3DH+PVRF-SAS Security

In this section we prove Theorems 3 and 4, i.e. that protocol X3DH+PVRF-SAS is secure in spite of key reuse between X3DH and PECVRF.

We begin by proving Theorem 3, which states that X3DH with PECVRF key reuse is a key-indistinguishable bundled authenticated key exchange.

Proof. To demonstrate the security of X3DH, [18] define the notion of BAKE key indistinguishability and prove that X3DH satisfies it (under GapDH and in the ROM). The definition of this property is long and complex, involving many possible cases corresponding to different combinations of keys that may be compromised. We refer to [18, Definition 3] for the full details, and here we only use the key indistinguishability game in a “black-box” way. Essentially, we only argue that the reuse of X3DH keys for PECVRF evaluation does not invalidate the existing security argument.

(1) Denote the X3DH key indistinguishability game as G . Consider a variation G^* of G where, after receiving any public key, the adversary $\mathcal{A}$ has oracle access to arbitrary PECVRF evaluations under it. Clearly, if X3DH is secure in this game, then it is secure when its keys are reused as in X3DH+PVRF-SAS. Suppose there exists an adversary $\mathcal{A}$ with non-negligible advantage ϵ in G^* . We will construct adversary $\mathcal{A}'$ with non-negligible advantage ϵ' in G .

(2) We begin by decoupling G^* into G and PECVRF. Whenever G needs to sample a new key, we perform PECVRF.KeyGen followed immediately by PECVRF.KeyComp and send the public key and compromised secret key to G . $\mathcal{A}$ interacts with the interface of G^* and its PECVRF evaluation queries are routed to the relevant PECVRF instance. These changes are purely conceptual and do not change the view of $\mathcal{A}$ at all.

(3) We next observe that G and $\mathcal{A}$ together form an environment with respect to PECVRF. In particular, we define the output of this environment as the output of the key indistinguishability game G . By the UC security of PECVRF (Theorem 7) there exists an efficient simulator $\text{SIM}_{\text{PECVRF}}$ such that no efficient environment can distinguish $\text{SIM}_{\text{PECVRF}}$ in interaction with $\mathcal{F}_{\text{PVRF}}$ from the real PECVRF protocol with probability greater than negligible ϵ_{PECVRF} (under the DDH assumption and in the ROM). We therefore replace PECVRF with $\text{SIM}_{\text{PECVRF}} \leftrightarrow \mathcal{F}_{\text{PVRF}}$ to create a new game in which the output of G is negligibly different from before.

(4) In order to construct an adversary $\mathcal{A}'$ against G we must allow G to sample its keys internally, not take them as input from $\text{SIM}_{\text{PECVRF}}$. By inspection, we observe that $\text{SIM}_{\text{PECVRF}}$ (Figure 8) never makes use of secret keys (until the time that a key is adaptively corrupted). We therefore need only slightly modify $\text{SIM}_{\text{PECVRF}}$ so that it runs on public keys received from G rather than sampling them itself. Denote this modified simulator $\text{SIM}'_{\text{PECVRF}}$, and define $\mathcal{A}'$ to be the combination of $\mathcal{A}$, $\text{SIM}_{\text{PECVRF}}$, and $\mathcal{F}_{\text{PVRF}}$ in interaction with each other.

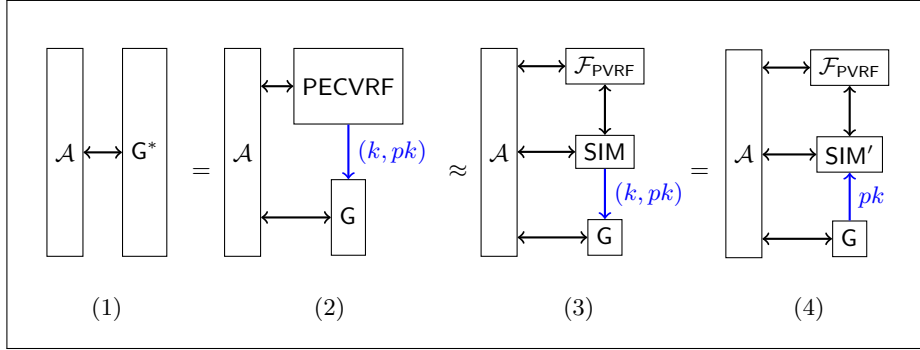


Fig. 15: Diagram of proof that X3DH with PECVRF key reuse is key-indistinguishable

The argument above is diagrammed in Figure 15. All together, we have $\epsilon \leq \epsilon' + \epsilon_{\text{PECVRF}}$. Under GapDH and in the ROM, ϵ' is negligible per [18, Theorem 6], and, under DDH and in the ROM, ϵ_{PECVRF} is negligible per Theorem 7. Therefore, ϵ is negligible, and X3DH+PVRF-SAS is key-indistinguishable.

Theorem 3 is proven.

We next prove Theorem 4, which states that PECVRF with X3DH key reuse realizes $\mathcal{F}_{\text{PVRF}}$.

Proof. We define PECVRF^* to be an augmented version of protocol PECVRF wherein the ideal adversary $\mathcal{A}^*$ is granted access to an “X3DH oracle”. In particular, $\mathcal{A}^*$ can use this oracle to either (1) perform an X3DH authenticated key exchange between 2 honest parties using any set of PECVRF keys or (2) perform an X3DH authenticated key exchange between an honest party using any set of PECVRF keys and $\mathcal{A}^*$ using arbitrary (adversarial) public keys. Notice that, in X3DH, protocol messages only consist of public keys and the key confirmation τ_{conf} . In PECVRF^* , the public keys are already known to $\mathcal{A}^*$, and so we omit them from the output of our new X3DH oracle. Instead, the X3DH oracle returns to $\mathcal{A}^*$ $(K || \tau_{\text{conf}})$ for each honest party, i.e. the output of the key derivation function KDF. Sending K to $\mathcal{A}^*$ models the fact that session keys are subject to arbitrary leakage after key exchange completion.

In X3DH+PVRF-SAS, only one-time keys otsk (and semi-static keys ssk if one-time keys run out) are reused as PECVRF keys. Thus, PECVRF^* is a generalization of X3DH+PVRF-SAS wherein all X3DH keys are potentially reused for PECVRF. If PECVRF^* UC-realizes $\mathcal{F}_{\text{PVRF}}$, then PECVRF is secure when its keys are reused as in X3DH+PVRF-SAS.

To prove PECVRF^* security we propose a minor modification to the proof of PECVRF security (Theorem 7). In particular, note that simulation of the X3DH oracle of PECVRF^* is easily added to $\text{SIM}_{\text{PECVRF}}$ (Figure 8) because $\text{SIM}_{\text{PECVRF}}$ samples and knows all honest parties’ PECVRF secret keys k . Thus, the simulator can simply run the real X3DH code on behalf of honest parties and return the output of KDF. Denote by $\text{SIM}_{\text{PECVRF}}^*$ this augmented version of $\text{SIM}_{\text{PECVRF}}$.

By inspection, almost the entirety of the proof of Theorem 7 applies without modification to proving that $\text{SIM}_{\text{PECVRF}}^*$ is a simulator for PECVRF^* . The code of the X3DH oracle is identical in the real world of PECVRF^* and in the simulated world of $\text{SIM}_{\text{PECVRF}}^*$. There is, however, one step in the proof where secret keys k are not available—namely, the CDH reduction. That reduction derives all public keys from CDH challenges rather than sampling them internally. Thus, there is no obvious way for this reduction to emulate the X3DH oracle.

To resolve this issue, we must “upgrade” to a GapDH reduction wherein the reduction has access to a DDH oracle. Since the reduction does not hold honest parties’ secret keys, it cannot compute $(K||\tau_{\text{conf}})$ by running the real X3DH code. Instead, the reduction simply samples random $(K||\tau_{\text{conf}})$ and returns it to $\mathcal{A}^*$. If KDF is modeled as a random oracle (as it is in the proof of X3DH security [18]), then the environment cannot distinguish this behavior from real unless it queries KDF on the input that the real honest party would have queried it on. The reduction monitors the environment’s KDF queries and programs output $(K||\tau_{\text{conf}})$ matching the output of the X3DH oracle if KDF is queried on an input that matches an X3DH session. To compute the KDF input for an X3DH session requires computing the Diffie-Hellman function on 4 pairs of public keys. The reduction cannot make this computation, but it can use the DDH oracle to check whether a provided input is a match.

In spite of key reuse with X3DH, protocol PECVRF^* realizes $\mathcal{F}_{\text{PVRF}}$.

F Deniability of X3DH and X3DH+VRF

In the following we will use the model from [18]. Extending the model to also have a message flow from the receiver to the sender. The honest but curious accuser deletes their ephemeral values.

F.1 Deniability of X3DH against passive adversaries

First we discuss deniability of X3DH against honest-but-curious accusers. Protocol X3DH is fully specified as a BAKE scheme in Figure 16.

X3DH.IdKeyGen(1^κ): $\text{isk}' \xleftarrow{\$} \mathbb{Z}_p, \text{ipk}' := [\text{isk}']G$ $(\text{vk}, \text{sk}) \xleftarrow{\$} \text{Sig.KeyGen}(1^\kappa)$ return $(\text{ipk} := (\text{ipk}', \text{vk}), \text{isk} := (\text{isk}', \text{sk}))$	X3DH.PreKeyBundleGen(isk_u): $(\text{isk}'_u, \text{sk}_u) \leftarrow \text{isk}_u$ $D_{\text{prek}}, D_{\rho\perp} := \emptyset$ $\text{ssk}_u \xleftarrow{\$} \mathbb{Z}_p, \text{spk}_u := [\text{ssk}]G$ $\sigma_{\text{spk}_u} \leftarrow \text{Sig.Sign}(\text{sk}_u, \text{spk}_u)$ for $(t \in [L])$ do $\text{otsk}_{u,t} \xleftarrow{\$} \mathbb{Z}_p, \text{otpk}_{u,t} := [\text{otsk}_{u,t}]G$ $\text{prek}_{u,t} := (\text{spk}_u, \sigma_{\text{spk}_u}, \text{otpk}_{u,t})$ $D_{\text{prek}}[t] \leftarrow (\text{prek}_{u,t}, (\text{ssk}_u, \text{otsk}_{r,t}))$ $\text{prek}_{u,\perp} := (\text{spk}_u, \sigma_{\text{spk}_u}, \perp)$ $D_{\text{prek}}[L+1] \leftarrow (\text{prek}_{u,\perp}, (\text{ssk}, \perp))$ return $(\text{prek}_u, \text{st}_u := (D_{\text{prek}}, D_{\rho\perp}))$
X3DH.Receive($\text{isk}_r, \text{st}_r, \text{ipk}_s, t, \rho$): $(\text{isk}'_r, \text{sk}_r) \leftarrow \text{isk}_r, (\text{ipk}'_s, \text{vk}_s) \leftarrow \text{ipk}_s$ $(D_{\text{prek}}, D_{\rho\perp}) \leftarrow \text{st}_r$ if $(t \neq \perp)$ then $\text{assert } (D_{\text{prek}}[t] \neq \perp)$ $(\text{prek}_{r,t}, (\text{ssk}_r, \text{otsk}_{r,t})) \leftarrow D_{\text{prek}}[t]$ else if $(t = \perp)$ then $\text{assert } (\rho \notin D_{\rho\perp})$ $D_{\rho\perp} \leftarrow D_{\rho\perp} \cup \{\rho\}$ $(\text{prek}_{r,t}, (\text{ssk}_r, \perp)) \leftarrow D_{\text{prek}}[t]$ $(\text{epk}, \tau_{\text{conf}}) \leftarrow \rho$ $\text{ss}_1 := [\text{ssk}_r]\text{ipk}'_s, \text{ss}_2 := [\text{isk}'_r]\text{epk}$ $\text{ss}_3 := [\text{ssk}_r]\text{epk}, \text{ss} := \text{ss}_1 \parallel \text{ss}_2 \parallel \text{ss}_3$ if $(t \neq \perp)$ then $\text{ss}_4 := [\text{otsk}_{r,t}]\text{epk}, \text{ss} := \text{ss}_1 \parallel \text{ss}_2 \parallel \text{ss}_3 \parallel \text{ss}_4$ $\text{tran} := \text{ipk}_s \parallel \text{ipk}_r \parallel \text{prek}_{r,t} \parallel \text{epk}$ $K \parallel \tau'_{\text{conf}} := \text{KDF}(\text{ss}, \text{tran})$ $\text{assert } (\tau_{\text{conf}} = \tau'_{\text{conf}})$ if $(t \neq \perp)$ then $D_{\text{prek}}[t] \leftarrow \perp$ $\text{st}_r \leftarrow (D_{\text{prek}}, D_{\rho\perp})$ return (K, st_r)	X3DH.Send($\text{isk}_s, \text{ipk}_r, \text{prek}_r$): $(\text{isk}'_s, \text{sk}_s) \leftarrow \text{isk}_s, (\text{ipk}'_r, \text{vk}_r) \leftarrow \text{ipk}_r$ $(\text{spk}_r, \sigma_{\text{spk}_r}, \text{otpk}_r) \leftarrow \text{prek}_r$ $\text{assert Sig.Ver}(\text{vk}_r, \text{spk}_r, \sigma_{\text{spk}_r}) = 1$ $\text{esk} \xleftarrow{\$} \mathbb{Z}_p, \text{epk} := [\text{esk}]G$ $\text{ss}_1 := [\text{isk}'_s]\text{spk}_r, \text{ss}_2 := [\text{esk}]\text{ipk}'_r$ $\text{ss}_3 := [\text{esk}]\text{spk}_r, \text{ss} := \text{ss}_1 \parallel \text{ss}_2 \parallel \text{ss}_3$ if $(\text{otpk}_r \neq \perp)$ then $\text{ss}_4 := [\text{esk}]\text{otpk}_r, \text{ss} := \text{ss}_1 \parallel \text{ss}_2 \parallel \text{ss}_3 \parallel \text{ss}_4$ $\text{tran} := \text{ipk}_s \parallel \text{ipk}_r \parallel \text{prek}_r \parallel \text{epk}$ $K \parallel \tau_{\text{conf}} := \text{KDF}(\text{ss}, \text{tran})$ $\rho := (\text{epk}, \tau_{\text{conf}})$ return (K, ρ)

Fig. 16: BAKE protocol X3DH = (IdKeyGen, PreKeyBundleGen, Send, Receive)

Lemma 1 ([20]). *The X3DH protocol is locally deniable with respect to leakage function $\mathcal{L}_{\text{high}}$ disclosure function $\mathcal{D}_{\text{high}}$ and KDC leakage bit $\mathcal{L}_{\text{KDC}} = 1$.*

We will define the simulators here. For the proof refer to [20].

SimPreK $((\text{ipk}_u)_{u \in \mathcal{N}}, (\text{isk}_u)_{u \in \mathcal{C}}, (\text{prek}_u)_{u \in \mathcal{H}})$: For $u \in \mathcal{C}$ run $(\text{prek}_u, \text{st}_u) \leftarrow \text{X3DH.PreKeyBundleGen}(\text{isk}_u)$, and output $((\text{prek}_u)_{u \in \mathcal{H}}, (\text{prek}_u, \text{st}_u)_{u \in \mathcal{C}}, \text{st}_{\text{Sim}})$ with $\text{st}_{\text{Sim}} = ((\text{ipk}_u, \text{prek}_u)_{u \in \mathcal{N}}, (\text{isk}_u, \text{st}_u)_{u \in \mathcal{C}})$. Note the simulator does not overwrite any prekey bundles.

SimTrans $(\text{isk}, \text{st}_{\text{Sim}}, (s, r, t))$: Here isk is either $\text{isk}_s, \text{isk}_r$ or $\perp$ depending on $s \in \mathcal{C}, r \in \mathcal{C}$ or $(s, r) \in \mathcal{H} \times \mathcal{H}$, respectively.

If $(s, r) \in \mathcal{H} \times \mathcal{C}$: First sample $\text{esk} \xleftarrow{\$} \mathbb{Z}_p$ and set $\text{epk} := [\text{esk}]G$. Then extract the signed prekey ssk_r from $\text{st}_r \in \text{st}_{\text{Sim}}$ and compute

$$(\text{ss}_1, \text{ss}_2, \text{ss}_3, \text{ss}_4) := ([\text{ssk}_r]\text{ipk}'_s, [\text{esk}]\text{ipk}'_r, [\text{esk}]\text{spk}_r, [\text{esk}]\text{otpk}_{r,t})$$

compute the rest of X3DH.Send as in Fig.16 and output (K, ρ) .

If $(s, r) \in \mathcal{C} \times \mathcal{H}$: Simply run $(K, \rho) \leftarrow \text{X3DH.Send}(\text{isk}_s, \text{ipk}_r, \text{prek}_{r,t})$ and output (K, ρ) .

SimSt $_{\mathcal{H}}$ $(\text{isk}_r, \text{st}_r)$: Parse $(D_{\text{prek}}, D_{\rho\perp}) \leftarrow \text{st}_r$ and delete the last unused one-time state, i.e., let t be the smallest index s.t. $D_{\text{prek}}[t] \neq \perp$. If $t \leq L$, let $D_{\text{prek}}[t] \leftarrow \perp$. Output the updated state st_r .

SimSt $_{\mathcal{C}}$ (st_{Sim}) : For $u \in \mathcal{C}$ extract st_u from st_{Sim} , parse $(D_{\text{prek}}, D_{\rho\perp}) \leftarrow \text{st}_u$ and delete the ephemeral secrets associated to used prekeys, i.e., for $t \in [\text{counter}_u]$, with $t \leq L$, let $D_{\text{prek}}[t] \leftarrow \perp$. Output $(\text{st}_u)_{u \in \mathcal{C}}$.

Lemma 2 ([20]). *Assume KDF is modeled as a random oracle and the DDH assumption holds. Then X3DH is globally deniable for honest users $\mathcal{H} \subseteq \mathcal{N}$ against honest-but-curious accusers $\mathcal{C} := \mathcal{N} \setminus \mathcal{H}$ with respect to disclosure function $\mathcal{D}_{\text{high}}$, KDC leakage bit $\mathcal{L}_{\text{KDC}} = 1$ and leakage function $\mathcal{L}_{\text{leak}}$ for $\text{leak} = \text{high}$ if the one-time prekey bundles are never depleted; and for $\text{leak} = \text{med}$ otherwise.*

The simulator here is defined identically to Lemma 1 except we have to further specify the description of **SimTrans** on input $(s, r) \in \mathcal{H} \times \mathcal{H}$.

SimTrans $(\text{st}_{\text{Sim}}, (s, r, t))$: Sample $\text{epk} \xleftarrow{\$} \mathbb{G}$, sample random $K || \tau_{\text{conf}}$ from range of KDF. Compute ρ as in Fig.16 and output (K, K, ρ) .

F.2 Strong Deniability of X3DH

Lemma 3 ([20]). *Assume the underlying group used by X3DH is a generic group of prime order p , and the key derivation function KDF is modeled as a random oracle. Then X3DH is strongly local deniable for honest users $\mathcal{H} \subseteq \mathcal{N}$ against malicious accusers $\mathcal{C} := \mathcal{N} \setminus \mathcal{H}$ with respect to leakage function $\mathcal{L}_{\text{high}}$ and KDC leakage bit $\mathcal{L}_{\text{KDC}} = 1$.*

Lemma 4 ([20]). Assume the underlying group used by X3DH is a generic group of prime order p and KDF is modeled as a random oracle. Then X3DH is strongly global deniable for honest users $\mathcal{H}$ against malicious accusers $\mathcal{C}$ with respect to KDC leakage if $\mathcal{L}_{\text{KDC}} = 1$ and leakage function $\mathcal{L}_{\text{leak}}$ for $\text{leak} = \text{high}$ if the one-time prekeys are never depleted and for $\text{leak} = \text{med}$ otherwise.

In GGM when $\mathcal{A}$ outputs a label s , $\text{Sim}_{\mathcal{A}}$ can efficiently verify if s is a linear combination of labels/group elements (or obviously sampled labels/group elements) i.e. whether $\mathcal{A}$ knows the discrete log of s to the given random injection function/generator. If it does not, s is a Bad_L label. Otherwise it is a Good_L label.

$\text{SimPreK}_{\mathcal{A}}((\text{ipk}_u)_{u \in \mathcal{N}}, (\text{isk}_u)_{u \in \mathcal{C}}, (\text{prek}_u)_{u \in \mathcal{H}})$: It first internally executes the adversary $((\text{prek}_u)_{u \in \mathcal{C}}, \text{st}_{\mathcal{A}}) \xleftarrow{\$} \mathcal{A}((\text{ipk}_u)_{u \in \mathcal{N}}, (\text{isk}_u)_{u \in \mathcal{C}}, (\text{prek}_u)_{u \in \mathcal{H}})$. It then outputs $((\text{prek}_u)_{u \in \mathcal{H}}, (\text{prek}_u)_{u \in \mathcal{C}}, \text{st}_{\mathcal{A}}, \text{st}_{\text{Sim}})$ with $\text{st}_{\text{Sim}} = ((\text{ipk}_u, \text{prek}_u)_{u \in \mathcal{N}}, \text{st}_{\mathcal{A}}, \text{aux}_{\text{Sim}})$ where aux_{Sim} contains all GGM and random oracle queries made by $\mathcal{A}$ so far. $\text{SimTrans}_{\mathcal{A}}(\text{isk}, \text{st}_{\text{Sim}}, (s, r, t))$: Here isk is either $\text{isk}_s, \text{isk}_r$ or $\perp$ depending on $s \in \mathcal{C}$, $r \in \mathcal{C}$ or $(s, r) \in \mathcal{H} \times \mathcal{H}$, respectively.

If $(s, r) \in \mathcal{H} \times \mathcal{C}$: First retrieve the prekey bundle $(\text{spk}_r, \sigma_{\text{spk}_r}, \text{otpk}_{r,t}) \leftarrow \text{prek}_{r,t}$ of the malicious receiver, sample $\text{esk} \xleftarrow{\$} \mathbb{Z}_p$ and query the GGM labeling oracle to receive $\Xi(\text{esk})$. Then check if spk_r is a Good_L using $\text{aux}_{\text{Sim}} \in \text{st}_{\text{Sim}}$. If so, compute the unique signed prekey secret $\text{ssk}_r \in \mathbb{Z}_p$ such that $\Xi(\text{ssk}_r) = \text{spk}_r$. Query the GGM labeling oracle on tuples $(\text{ipk}'_s, \text{ssk}_r), (\text{ipk}'_r, \text{esk}), (\text{spk}_r, \text{esk})$ and $(\text{otpk}_{r,t}, \text{esk})$ to obtain labels (s_1, s_2, s_3, s_4) . Set

$$(\text{ss}_1, \text{ss}_2, \text{ss}_3, \text{ss}_4) = (s_1, s_2, s_3, s_4)$$

compute K and ρ according to Fig.16 and output $(K, \rho, \text{st}_{\text{Sim}})$. Note above if $\text{otpk}_{r,t} = \perp$ then Sim simply ignores generation of s_4 .

Otherwise, if spk_r is a Bad_L label then sample random $K || \tau_{\text{conf}}$ from range of KDF, compute ρ and output $(K, \rho, \text{st}_{\text{Sim}})$.

If $(s, r) \in \mathcal{C} \times \mathcal{H}$: First execute the malicious sender internally $(\rho, \text{st}_{\mathcal{A}}) \xleftarrow{\$} \mathcal{A}(\text{isk}_s, \text{st}_{\mathcal{A}}, (s, r, t))$ using the accuser's state $\text{st}_{\mathcal{A}} \in \text{st}_{\text{Sim}}$. The simulator updates st_{Sim} by storing the updated $\text{st}_{\mathcal{A}}$. Parse $(\text{epk}, \tau_{\text{conf}}) \leftarrow \rho$ and check if $\mathcal{A}$ ever made a random oracle query on (ss, tran) where $\text{tran} := \text{ipk}_s || \text{ipk}_r || \text{prek}_{r,t} || \text{epk}$ such that $K || \tau_{\text{conf}} = \text{KDF}(\text{ss}, \text{tran})$ using aux_{Sim} . If not, output $(\perp, \rho, \text{st}_{\text{Sim}})$. Otherwise, retrieve the first such tuple (ss, K) and output $(K, \rho, \text{st}_{\text{Sim}})$.

If $(s, r) \in \mathcal{H} \times \mathcal{H}$: First sample $\text{esk} \xleftarrow{\$} \mathbb{Z}_p$ and query the GGM labeling oracle to receive $\text{epk} := \Xi(\text{esk})$. Then sample random $K || \tau_{\text{conf}}$ from the range of KDF and compute ρ . Output (K, K, ρ) .

$\text{SimSt}_{\mathcal{H}}$ Parse $(D_{\text{prek}}, D_{\rho \perp}) \leftarrow \text{st}_r$ and delete the last unused one-time state, i.e., let t be the smallest index s.t. $D_{\text{prek}}[t] \neq \perp$. If $t \leq L$, let $D_{\text{prek}}[t] \leftarrow \perp$. Output the updated state st_r .

$\text{SimSt}_{\mathcal{A}}(\text{st}_{\text{Sim}})$: Extract $\text{st}_{\mathcal{A}}$ from st_{Sim} and output $\text{st}_{\mathcal{A}}$.

F.3 Deniability of X3DH + VRF against passive adversaries

Consider the protocol from [17] shown in Fig.17. The parts that differ from regular X3DH are highlighted in yellow .

Receive ($\text{isk}_r, \text{st}_r, \text{ipk}_s, t, \rho$): $(\text{isk}'_r, \text{sk}_r) \leftarrow \text{isk}_r, (\text{ipk}'_s, \text{vk}_s) \leftarrow \text{ipk}_s$ $(D_{\text{prek}}, D_{\rho\perp}) \leftarrow \text{st}_r$ if ($t \neq \perp$) then assert ($D_{\text{prek}}[t] \neq \perp$) $(\text{prek}_{r,t}, (\text{ssk}_r, \text{otsk}_{r,t})) \leftarrow D_{\text{prek}}[t]$ else if ($t = \perp$) then assert ($\rho \notin D_{\rho\perp}$) $D_{\rho\perp} \leftarrow D_{\rho\perp} \cup \{\rho\}$ $(\text{prek}_{r,t}, (\text{ssk}_r, \perp)) \leftarrow D_{\text{prek}}[t]$ $(\text{epk}, \tau_{\text{conf}}, (z_s, \pi_s)) \leftarrow \rho$ $\text{ss}_1 := [\text{ssk}_r]\text{ipk}'_s, \text{ss}_2 := [\text{isk}'_r]\text{epk}$ $\text{ss}_3 := [\text{ssk}_r]\text{epk}, \text{ss} := \text{ss}_1 \parallel \text{ss}_2 \parallel \text{ss}_3$ $\text{tran} := \text{ipk}_s \parallel \text{ipk}_r \parallel \text{prek}_{r,t} \parallel \text{epk}$ assert $\text{VRF.Ver}(\text{epk}, \text{tran}, z_s, \pi_s)$ if ($t \neq \perp$) then $\text{ss}_4 := [\text{otsk}_{r,t}]\text{epk}, \text{ss} := \text{ss}_1 \parallel \text{ss}_2 \parallel \text{ss}_3 \parallel \text{ss}_4$ $(z_r, \pi_r) \leftarrow \text{VRF.Eval}(\text{otsk}_{r,t}, \text{tran})$ else if ($t = \perp$) then $(z_r, \pi_r) \leftarrow \text{VRF.Eval}(\text{ssk}_r, \text{tran})$ $K \parallel \tau'_{\text{conf}} := \text{KDF}(\text{ss}, \text{tran})$ assert ($\tau_{\text{conf}} = \tau'_{\text{conf}}$) if ($t \neq \perp$) then $D_{\text{prek}}[t] \leftarrow \perp$ $\text{st}_r \leftarrow (D_{\text{prek}}, D_{\rho\perp})$ return ($K, \text{st}_r, (z_r, \pi_r)$)	Send ($\text{isk}_s, \text{ipk}_r, \text{prek}_r$) $(\text{isk}'_s, \text{sk}_s) \leftarrow \text{isk}_s, (\text{ipk}'_r, \text{vk}_r) \leftarrow \text{ipk}_r$ $(\text{spk}_r, \sigma_{\text{spk}_r}, \text{otpk}_r) \leftarrow \text{prek}_r$ assert $\text{Sig.Ver}(\text{vk}_r, \text{spk}_r, \sigma_{\text{spk}_r})$ $\text{esk} \xleftarrow{\$} \mathbb{Z}_p, \text{epk} := [\text{esk}]G$ $\text{ss}_1 := [\text{isk}'_s]\text{spk}_r, \text{ss}_2 := [\text{esk}]\text{ipk}'_r$ $\text{ss}_3 := [\text{esk}]\text{spk}_r, \text{ss} := \text{ss}_1 \parallel \text{ss}_2 \parallel \text{ss}_3$ $\text{tran} := \text{ipk}_s \parallel \text{ipk}_r \parallel \text{prek}_r \parallel \text{epk}$ if ($\text{otpk}_r \neq \perp$) then $\text{ss}_4 := [\text{esk}]\text{otpk}_r, \text{ss} := \text{ss}_1 \parallel \text{ss}_2 \parallel \text{ss}_3 \parallel \text{ss}_4$ $m := (z_s, \pi_s) \leftarrow \text{VRF.Eval}(\text{esk}, \text{tran})$ $K \parallel \tau_{\text{conf}} := \text{KDF}(\text{ss}, \text{tran})$ $\rho := (\text{epk}, \tau_{\text{conf}}, m), \text{temp} := (\text{ipk}_r, \text{prek}_{r,t}, m)$ return (K, ρ, temp) SenderReceive ($\text{temp}, (z_r, \pi_r)$) $((\text{ipk}'_r, \text{vk}_r), (\text{spk}_r, \sigma_{\text{spk}_r}, \text{otpk}_r), (z_s, \pi_s)) \leftarrow \text{temp}$ if ($\text{otpk}_r \neq \perp$) then assert $\text{VRF.Ver}(\text{otpk}_r, \text{tran}, z_r, \pi_r)$ else if ($\text{otpk}_r = \perp$) then assert $\text{VRF.Ver}(\text{spk}_r, \text{tran}, z_r, \pi_r)$ return ($z_r \oplus z_s$)
---	---

Fig. 17:

X3DH + VRF = (IdKeyGen, PreKeyBundleGen, Send, Receive, SenderReceive)
 which uses VRF = (KeyGen, Eval, Ver). Differences from X3DH in Fig.16 are
 highlighted in yellow .

Lemma 5. *The X3DH + VRF protocol is globally deniable with respect to disclosure function $\mathcal{D}_{\text{high}}$, KDC leakage bit $\mathcal{L}_{\text{KDC}} = 0$ leakage function $\mathcal{L}_{\text{high}}$ if the one-time prekey bundles are never depleted; and it is not locally deniable for the receiver or globally deniable for the sender and receiver otherwise. It is still deniable for honest senders for leakage function $\mathcal{L}_{\text{high}}$ even if the one-time prekey bundles are depleted.*

Following is the simulator.

SimPreK $((\text{ipk}_u)_{u \in \mathcal{N}}, (\text{isk}_u)_{u \in \mathcal{C}}, (\text{prek}_u)_{u \in \mathcal{H}})$: For $u \in \mathcal{C}$ run $(\text{prek}_u, \text{st}_u) \leftarrow \text{PreKeyBundleGen}(\text{isk}_u)$, but additionally parse each prekey for each honest user $(\text{spk}_u, \sigma_{\text{spk}_u}, \text{otpk}_{u,t}) \leftarrow \text{prek}_{u,t}$. Since $\mathcal{L}_{\text{KDC}} = 0$ we can overwrite parts of the prekey bundle for honest users. Sample new random one-time secret keys $\text{otsk}_{r,t}^* \xleftarrow{\$} \mathbb{Z}_p$ and compute $\text{otpk}_{u,t}^* := [\text{otsk}_{u,t}^*]G$. Replace the honest one-time prekeys with simulated one-time prekeys $\text{prek}_{u,t}^* \leftarrow (\text{spk}_u, \sigma_{\text{spk}_u}, \text{otpk}_{u,t}^*)$. Output $((\text{prek}_u^*)_{u \in \mathcal{H}}, (\text{prek}_u, \text{st}_u)_{u \in \mathcal{C}}, \text{st}_{\text{sim}})$ with $\text{st}_{\text{sim}} = ((\text{ipk}_u, \text{prek}_u)_{u \in \mathcal{N}}, (\text{isk}_u, \text{st}_u)_{u \in \mathcal{C}}, (\text{otsk}_u^*)_{u \in \mathcal{H}})$.
SimTrans $(\text{isk}, \text{st}_{\text{sim}}, (s, r, t))$: Here isk is either $\text{isk}_s, \text{isk}_r$ or $\perp$ depending on $s \in \mathcal{C}, r \in \mathcal{C}$ or $(s, r) \in \mathcal{H} \times \mathcal{H}$, respectively.

If $(s, r) \in \mathcal{H} \times \mathcal{C}$: First sample $\text{esk} \xleftarrow{\$} \mathbb{Z}_p$ and set $\text{epk} := [\text{esk}]G$. Then extract the signed prekey ssk_r from $\text{st}_r \in \text{st}_{\text{sim}}$ and compute

$$(\text{ss}_1, \text{ss}_2, \text{ss}_3, \text{ss}_4) := ([\text{ssk}_r]\text{ipk}'_s, [\text{esk}]\text{ipk}'_r, [\text{esk}]\text{spk}_r, [\text{esk}]\text{otpk}_{r,t})$$

compute the rest of **Send** as in Fig.17 to get (K, ρ, temp) . Extract $\text{st}_r \in \text{st}_{\text{sim}}$ from the *honest-but-curious* accuser and compute $(K', \text{st}_r, m) \xleftarrow{\$} \text{Receive}(\text{isk}_r, \text{st}'_r, \text{ipk}'_s, t, \rho)$ where st'_r is a copy of st_r since **SimStC** updates the states of accusing receivers. Run $\text{sas}_r \leftarrow \text{SenderReceive}(\text{temp}, m)$. Output (K, ρ, m) .

If $(s, r) \in \mathcal{C} \times \mathcal{H}$: Simply run $(K, \rho, \text{temp}) \leftarrow \text{Send}(\text{isk}_s, \text{ipk}_r, \text{prek}_{r,t})$.

If we assume one-time prekeys never run out then the VRF is always computed under the one-time prekey $\text{otpk}_{r,t}$. In the simulated world these values can be replaced indistinguishably by the simulator's own random keys. Thus, the simulator can compute $m := (z_r, \pi_r) \leftarrow \text{VRF.Eval}(\text{otsk}_{r,t}^*, \text{tran})$ and create an indistinguishable transcript by outputting (K, ρ, m) .

If $\text{otpk}_{r,t} = \perp$ and the real receiver sends a VRF on the semi-static key spk then they cannot obtain deniability. This is because a VRF is a deterministic signature and the real receiver has signed spk_r in σ_{spk_r} , implying knowledge of ssk_r . Since a VRF contains a NIZK, the simulator cannot create a valid VRF on spk_r without violating soundness/uniqueness since it does not know ssk_r .

If $(s, r) \in \mathcal{H} \times \mathcal{H}$: Sample random element $\text{esk} \xleftarrow{\$} \mathbb{Z}_p$, sample random $K || \tau_{\text{conf}}$ and compute ρ using esk and tran . Compute $m \leftarrow \text{VRF.Eval}(\text{otsk}_{r,t}^*, \text{tran})$ since it knows the one-time secret key corresponding to the public key it replaced in the honest users' prekey bundles. Output (K, K, ρ, m) .
 Similar to above, we lose deniability immediately if the honest receiver uses ssk_r as the VRF key or $\mathcal{L}_{\text{KDC}} = 1$.

$\text{SimSt}_{\mathcal{H}}(\text{isk}_r, \text{st}_r)$: Parse $(D_{\text{prek}}, D_{\rho\perp}) \leftarrow \text{st}_r$ and delete the last unused one-time state, i.e., let t be the smallest index s.t. $D_{\text{prek}}[t] \neq \perp$. If $t \leq L$, let $D_{\text{prek}}[t] \leftarrow \perp$. Output the updated state st_r .

$\text{SimSt}_{\mathcal{C}}(\text{st}_{\text{sim}})$: For $u \in \mathcal{C}$ extract st_u from st_{sim} , parse $(D_{\text{prek}}, D_{\rho\perp}) \leftarrow \text{st}_u$ and delete the ephemeral secrets associated to used prekeys, i.e., for $t \in [\text{counter}_u]$, with $t \leq L$, let $D_{\text{prek}}[t] \leftarrow \perp$. Output $(\text{st}_u)_{u \in \mathcal{C}}$.

F.4 Strong Deniability of X3DH+VRF

Lemma 6. *Assume the underlying group used by X3DH+VRF is a generic group of prime order p and KDF is modeled as a random oracle. The X3DH + VRF protocol is strongly global deniable with respect to KDC leakage bit $\mathcal{L}_{\text{KDC}} = 0$ and leakage function $\mathcal{L}_{\text{high}}$ if the one-time prekey bundles are never depleted; and it is not strongly local deniable for the receiver or strongly global deniable for honest senders and receivers otherwise. It is still strongly local deniable for honest senders for leakage function $\mathcal{L}_{\text{high}}$ even if the last-resort prekey bundles are used.*

We will construct the simulator.

SimPreK : Same as in Lemma 5 except we run $\mathcal{A}$ to get the prekey bundles for the accusers. Additionally we keep track of all GGM and random oracle queries in aux_{sim} .

$\text{SimTrans}_{\mathcal{A}}(\text{isk}, \text{st}_{\text{sim}}, (s, r, t))$: Here isk is either $\text{isk}_s, \text{isk}_r$ or $\perp$ depending on $s \in \mathcal{C}$, $r \in \mathcal{C}$ or $(s, r) \in \mathcal{H} \times \mathcal{H}$, respectively.

If $(s, r) \in \mathcal{H} \times \mathcal{C}$: First retrieve the prekey bundle $(\text{spk}_r, \sigma_{\text{spk}_r}, \text{otpk}_{r,t}) \leftarrow \text{prek}_{r,t}$ of the malicious receiver, sample $\text{esk} \xleftarrow{\$} \mathbb{Z}_p$ and query the GGM labeling oracle to receive $\Xi(\text{esk})$.

Check if spk_r is a Good_L label using $\text{aux}_{\text{sim}} \in \text{st}_{\text{sim}}$. If so, compute the unique signed prekey secret $\text{ssk}_r \in \mathbb{Z}_p$ such that $\Xi(\text{ssk}_r) = \text{spk}_r$. Query the GGM labeling oracle on tuples $(\text{ipk}'_s, \text{ssk}_r)$, $(\text{ipk}'_r, \text{esk})$, $(\text{spk}_r, \text{esk})$ and $(\text{otpk}_{r,t}, \text{esk})$ to obtain labels (s_1, s_2, s_3, s_4) . Set

$$(\text{ss}_1, \text{ss}_2, \text{ss}_3, \text{ss}_4) = (s_1, s_2, s_3, s_4)$$

compute (K, ρ, temp) according to Send in Fig.17. Now internally run the malicious receiver algorithm $(m, \text{st}_{\mathcal{A}}) \xleftarrow{\$} \mathcal{A}(\text{isk}_r, \text{st}_{\mathcal{A}}, \text{ipk}_s, t, \rho)$. Run $\text{sas}_r \leftarrow \text{SenderReceive}(\text{temp}, m)$ to see if the real sender would accept. If $(K \neq \perp)$ (i.e. the honest sender accepts) then output (K', ρ, m) else output $(\perp, \rho, m)$.

Otherwise, if spk_r is a Bad_L label then sample random $K || \tau_{\text{conf}}$ from range of KDF, compute ρ as above. Sim can compute m as above if not out of one-time prekeys. If $\text{otpk}_{r,t} = \perp$ then the malicious receiver will fail the check in SenderReceive since it cannot forge proofs on Bad_L spk_r . Output (K, ρ, m) .

If $(s, r) \in \mathcal{C} \times \mathcal{H}$: First execute the malicious sender internally $(\rho, \text{st}_A) \xleftarrow{\$} \mathcal{A}(\text{isk}_s, \text{st}_A, (s, r, t))$ using the accuser's state $\text{st}_A \in \text{st}_{\text{Sim}}$. The simulator updates st_{Sim} by storing the updated st_A . Parse $(\text{epk}, \tau_{\text{conf}}, (z_s, \pi_s)) \leftarrow \rho$, check if $\mathcal{A}$ ever made a random oracle query on (ss, tran) where $\text{tran} := \text{ipk}_s || \text{ipk}_r || \text{prek}_{r,t} || \text{epk}$ such that $K || \tau_{\text{conf}} = \text{KDF}(\text{ss}, \text{tran})$ using aux_{Sim} and assert $\text{Ver}(\text{epk}, \text{tran}, z_s, \pi_s)$. If not, output $(\perp, \rho, \perp, \text{st}_{\text{Sim}})$ since this is the case where the receiver's authentication check will fail and they will output $\perp$ for the key and also for m . Otherwise, retrieve the first such tuple (ss, K) , compute $m \leftarrow \text{VRF.Eval}(\text{otsk}_{r,t}^*, \text{tran})$ as above and output (K, ρ, m) .

If the honest receiver runs out of prekey bundles and uses ssk_r to VRF then they lose all deniability similar to Lemma 5. If $\mathcal{L}_{\text{KDC}} = 1$ then honest receiver has no deniability.

If $(s, r) \in \mathcal{H} \times \mathcal{H}$: First sample $\text{esk} \xleftarrow{\$} \mathbb{Z}_p$ and query the GGM labeling oracle to receive $\text{epk} := \Xi(\text{esk})$. Then sample random $K || \tau_{\text{conf}}$ from the range of KDF and compute ρ . Compute $m \leftarrow \text{VRF.Eval}(\text{otsk}_{r,t}^*, \text{tran})$. Output (K, K, ρ, m) . If honest receiver is out of one-time prekeys then a third-party accuser/simulator cannot compute m i.e. the VRF under spk_r and we lose global deniability.

$\text{SimSt}_{\mathcal{H}}$ Parse $(D_{\text{prek}}, D_{\rho\perp}) \leftarrow \text{st}_r$ and delete the last unused one-time state, i.e., let t be the smallest index s.t. $D_{\text{prek}}[t] \neq \perp$. If $t \leq L$, let $D_{\text{prek}}[t] \leftarrow \perp$. Output the updated state st_r .

$\text{SimSt}_A(\text{st}_{\text{Sim}})$: Extract st_A from st_{Sim} and output st_A .

G Analysis of Deniability of X3DH+PVRF-SAS

We show that X3DH+PVRF-SAS (almost) retains the deniability properties of the underlying X3DH. For lack of space, we defer to Appendix F the specifications of the deniability properties of X3DH [20] and of the deniability break in protocol X3DH+VRF-SAS of [17].

In Figures 18 and 19 we fully specify the algorithms of X3DH+PVRF-SAS as they are used in the deniability analysis of this scheme. Fig. 18 contains the IdKeyGen and PreKeyBundleGen algorithms, which are the same for X3DH, X3DH+VRF-SAS and X3DH+PVRF-SAS, and Fig. 19 contains the remainder of the X3DH+PVRF-SAS algorithms, with differences from plain X3DH marked.

Note that in X3DH+PVRF-SAS the SAS values sas_s and sas_r generated by honest parties are publicly computable as $\text{sas} = z \oplus \text{nonce}$ from the other parts of the key exchange transcript. We therefore omit $\text{sas}_s, \text{sas}_r$ throughout our deniability arguments, with the understanding that they can be trivially simulated by any algorithm that can simulate protocol messages ρ, m .

G.1 Deniability Against Honest-But-Curious Accusers

We begin by proving the deniability of X3DH+PVRF-SAS against passive accusers.

IdKeyGen(1^κ):	PreKeyBundleGen(isk_u):
$\text{isk}' \xleftarrow{\$} \mathbb{Z}_p$, $\text{ipk}' := [\text{isk}^*]G$	$(\text{isk}'_u, \text{sk}_u) \leftarrow \text{isk}_u$
$(\text{vk}, \text{sk}) \xleftarrow{\$} \text{Sig.KeyGen}(1^\kappa)$	$D_{\text{prek}}, D_{\rho\perp} := \emptyset$
return ($\text{ipk} := (\text{ipk}', \text{vk})$, $\text{isk} := (\text{isk}', \text{sk})$)	$\text{ssk}_u \xleftarrow{\$} \mathbb{Z}_p$, $\text{spk}_u := [\text{ssk}]G$
	$\sigma_{\text{spk}_u} \leftarrow \text{Sig.Sign}(\text{sk}_u, \text{spk}_u)$
	for ($t \in [L]$) do
	$\text{otsk}_{u,t} \xleftarrow{\$} \mathbb{Z}_p$, $\text{otpk}_{u,t} := [\text{otsk}_{u,t}]G$
	$\text{prek}_{u,t} := (\text{spk}_u, \sigma_{\text{spk}_u}, \text{otpk}_{u,t})$
	$D_{\text{prek}}[t] \leftarrow (\text{prek}_{u,t}, (\text{ssk}_u, \text{otsk}_{u,t}))$
	$\text{prek}_{u,\perp} := (\text{spk}_u, \sigma_{\text{spk}_u}, \perp)$
	$D_{\text{prek}}[L+1] \leftarrow (\text{prek}_{u,\perp}, (\text{ssk}, \perp))$
	return ($\text{prek}_u, \text{st}_u := (D_{\text{prek}}, D_{\rho\perp})$)

Fig. 18: Algorithms IdKeyGen and PreKeyBundleGen of X3DH and X3DH+PVRF-SAS

Proof (Theorem 5).

We first construct the simulator required for the deniability proof:

- **SimPreK**: Honestly generate prekey bundles for accusers and do not overwrite the prekey bundles of honest users.
- **SimTrans**($\text{isk}, \text{st}_{\text{sim}}, (s, r, t)$): Here isk is either isk_s , isk_r , or $\perp$, depending on whether $s \in \mathcal{C}$, $r \in \mathcal{C}$, or $(s, r) \in \mathcal{H} \times \mathcal{H}$, respectively.

- If $(s, r) \in \mathcal{H} \times \mathcal{C}$: First sample $\text{esk} \xleftarrow{\$} \mathbb{Z}_p$ and set $\text{epk} := [\text{esk}]G$. Then extract the signed prekey ssk_r from $\text{st}_r \in \text{st}_{\text{sim}}$ and compute

$$(\text{ss}_1, \text{ss}_2, \text{ss}_3, \text{ss}_4) := ([\text{ssk}_r]\text{ipk}'_s, [\text{esk}]\text{ipk}'_r, [\text{esk}]\text{spk}_r, [\text{esk}]\text{otpk}_{r,t})$$

compute the rest of **Send** as in Fig. 19 to get (K, ρ, temp) . Retrieve st_r , the accuser state, from st_{sim} and compute $(K', m, \text{st}'_r) \leftarrow \text{Receive}(\text{isk}_r, \text{st}_r, \text{ipk}_s, t, \rho)$. Output (K, ρ, m) .

- If $(s, r) \in \mathcal{C} \times \mathcal{H}$: Run $(K, \rho, \text{temp}, \text{st}_s) \leftarrow \text{Send}(\text{isk}_s, \text{ipk}_r, \text{prek}_{r,t})$ using the accusing sender's long-term secrets.

Retrieve vt and vts from ρ and temp respectively. Construct input tran to **PECVRF** as the transcript of the key exchange and run $m \leftarrow \text{FakeEval}(\text{otpk}_{r,t}, vt, vts, \text{tran})$ and output (K, ρ, m) .

If $\text{otpk}_{r,t} = \perp$ then the simulator can do the same for spk_r . However, if $\text{leak} = \text{high}$ then this simulation will fail because $\mathcal{D}$ can compute $w' := [\text{ssk}]H$ and cross check with the transcript.

- If $(s, r) \in \mathcal{H} \times \mathcal{H}$: Sample random element $\text{esk} \xleftarrow{\$} \mathbb{Z}_p$, sample random $K || \tau_{\text{conf}}$ and compute ρ and tran . Compute m similarly as above. Output (K, K, ρ, m) .

Receive ($\text{isk}_r, \text{st}_r, \text{ipk}_s, t, \rho$):	Send ($\text{isk}_s, \text{ipk}_r, \text{prek}_r$)
$(\text{isk}'_r, \text{sk}_r) \leftarrow \text{isk}_r, (\text{ipk}'_s, \text{vk}_s) \leftarrow \text{ipk}_s$ $(D_{\text{prek}}, D_{\rho\perp}) \leftarrow \text{st}_r$ if ($t \neq \perp$) then assert ($D_{\text{prek}}[t] \neq \perp$) $(\text{prek}_{r,t}, (\text{ssk}_r, \text{otsk}_{r,t})) \leftarrow D_{\text{prek}}[t]$ else if ($t = \perp$) then assert ($\rho \notin D_{\rho\perp}$) $D_{\rho\perp} \leftarrow D_{\rho\perp} \cup \{\rho\}$ $(\text{prek}_{r,t}, (\text{ssk}_r, \perp)) \leftarrow D_{\text{prek}}[t]$ $(\text{epk}, \tau_{\text{conf}}, (vt, \text{nonce})) \leftarrow \rho$ $\text{ss}_1 := [\text{ssk}_r]\text{ipk}'_s, \text{ss}_2 := [\text{isk}'_r]\text{epk}$ $\text{ss}_3 := [\text{ssk}_r]\text{epk}, \text{ss} := \text{ss}_1 \text{ss}_2 \text{ss}_3$ $\text{tran} := \text{ipk}_s \text{ipk}_r \text{prek}_{r,t} \text{epk}$ if ($t \neq \perp$) then $\text{ss}_4 := [\text{otsk}_{r,t}]\text{epk}, \text{ss} := \text{ss}_1 \text{ss}_2 \text{ss}_3 \text{ss}_4$ $(z_r, \pi_r) \leftarrow \text{Eval}(\text{otsk}_{r,t}, vt, \text{tran})$ else if ($t = \perp$) then $(z_r, \pi_r) \leftarrow \text{Eval}(\text{ssk}_r, vt, \text{tran})$ $K \tau'_{\text{conf}} := \text{KDF}(\text{ss}, \text{tran})$ assert ($\tau_{\text{conf}} = \tau'_{\text{conf}}$) if ($t \neq \perp$) then $D_{\text{prek}}[t] \leftarrow \perp$ $\text{st}_r \leftarrow (D_{\text{prek}}, D_{\rho\perp})$ $\text{sas}_r := z_r \oplus \text{nonce}$ return ($K, \text{st}_r, (z_r, \pi_r), \text{sas}_r$)	$(\text{isk}'_s, \text{sk}_s) \leftarrow \text{isk}_s, (\text{ipk}'_r, \text{vk}_r) \leftarrow \text{ipk}_r$ $(\text{spk}_r, \sigma_{\text{spk}_r}, \text{otpk}_r) \leftarrow \text{prek}_r$ assert $\text{Sig.Ver}(\text{vk}_r, \text{spk}_r, \sigma_{\text{spk}_r})$ $\text{esk} \xleftarrow{\$} \mathbb{Z}_p, \text{epk} := [\text{esk}]G$ $\text{ss}_1 := [\text{isk}'_s]\text{spk}_r, \text{ss}_2 := [\text{esk}]\text{ipk}'_r$ $\text{ss}_3 := [\text{esk}]\text{spk}_r, \text{ss} := \text{ss}_1 \text{ss}_2 \text{ss}_3$ $\text{tran} := \text{ipk}_s \text{ipk}_r \text{prek}_r \text{epk}$ if ($\text{otpk}_r \neq \perp$) then $\text{ss}_4 := [\text{esk}]\text{otpk}_r, \text{ss} := \text{ss}_1 \text{ss}_2 \text{ss}_3 \text{ss}_4$ $(vt, vts) \xleftarrow{\$} \text{PreVerify}(\text{otpk}_r, \text{tran})$ else if ($\text{otpk}_r = \perp$) then $(vt, vts) \xleftarrow{\$} \text{PreVerify}(\text{spk}_r, \text{tran})$ $\text{nonce} \xleftarrow{\$} \mathcal{R}, m := (vt, \text{nonce})$ $K \tau_{\text{conf}} := \text{KDF}(\text{ss}, \text{tran}), \rho := (\text{epk}, \tau_{\text{conf}}, m)$ $\text{temp} := (\text{ipk}_r, \text{prek}_r, vts, m)$ return (K, ρ, temp)
	SenderReceive ($\text{temp}, (z_r, \pi_r)$)
	$(\cdot, (\text{spk}_r, \cdot, \text{otpk}_r), vts, (\cdot, \text{nonce})) \leftarrow \text{temp}$ if ($\text{otpk}_r \neq \perp$) then assert $\text{Verify}(\text{otpk}_r, vts, \text{tran}, z_r, \pi_r)$ else if ($\text{otpk}_r = \perp$) then assert $\text{Verify}(\text{spk}_r, vts, \text{tran}, z_r, \pi_r)$ $\text{sas}_s := z_r \oplus \text{nonce}$ return (sas_s)

Fig. 19: Protocol X3DH+PVRF-SAS, with differences from X3DH marked in yellow and SAS outputs marked in pink. X3DH+PVRF-SAS algorithms IdKeyGen and PreKeyBundleGen are exactly as in X3DH, see Figure 18.

- $\text{SimSt}_{\mathcal{H}}(\text{isk}_r, \text{st}_r)$: Parse $(D_{\text{prek}}, D_{\rho\perp}) \leftarrow \text{st}_r$ and delete the last unused one-time state, i.e., let t be the smallest index s.t. $D_{\text{prek}}[t] \neq \perp$. If $t \leq L$, let $D_{\text{prek}}[t] \leftarrow \perp$. Output the updated state st_r .
- $\text{SimSt}_{\mathcal{C}}(\text{st}_{\text{sim}})$: For $u \in \mathcal{C}$ extract st_u from st_{sim} , parse $(D_{\text{prek}}, D_{\rho\perp}) \leftarrow \text{st}_u$ and delete the ephemeral secrets associated to used prekeys, i.e., for $t \in [\text{counter}_u]$, with $t \leq L$, let $D_{\text{prek}}[t] \leftarrow \perp$. Output $(\text{st}_u)_{u \in \mathcal{C}}$.

We can analyze local deniability for the case when one-time prekeys do not run out first. We have disclosure function $\mathcal{D}_{\text{high}}$, leakage function $\mathcal{L}_{\text{high}}$ and KDC leakage bit $\mathcal{L}_{\text{KDC}} = 1$. By correctness of X3DH+PVRF-SAS since honest-but-curious accusers compute their PECVRF honestly SenderReceive will always verify and output a sas_s .

If $(s, r) \in \mathcal{H} \times \mathcal{C}$ notice that ss_2, ss_3 and ss_4 are generated exactly as the real world. The simulated $\text{ss}_1 = [\text{ssk}_r] \text{ipk}'_s$ has the same distribution as the real world $\text{ss}_1 = [\text{isk}'_s] \text{spk}_r$. Additionally, $m := (vt, \text{nonce})$ has the same distribution as the real world since it can be computed publicly (using $\text{otpk}_{r,t}$ and tran). Since we can use the accusing receiver's long-term secret, the simulator runs $\text{Receive}(\text{isk}_r, \text{st}_r, \text{ipk}_s, t, \rho)$ on identically distributed inputs and generates $(K', \text{st}_r, (z_r, \pi_r))$. Since the simulator can run the code a real receiver would run, this output is distributed exactly as the real world. Hence, the output of SimTrans is identical to the real world output from D's view, even given disc_D and leak_D .

If $(s, r) \in \mathcal{C} \times \mathcal{H}$ then ρ can be computed exactly as in the real world by SimTrans by simply running $(K, \rho, \text{temp}) \leftarrow \text{Send}(\text{isk}_s, \text{ipk}_r, \text{prek}_r)$. Since simulator cannot modify the prekey bundle it must compute the receiver's by running $(vt, vts) \leftarrow \text{PreVerify}(\text{otpk}_r, x)$ and then running $(z_r, \pi_r) \leftarrow \text{FakeEval}(\text{otpk}_r, vt, vts, x)$. The simulator can then run $\text{SenderReceive}(\text{temp}, (z_r, \pi_r))$ to get sas_r which will be distributed identically by correctness. The main difference from D's view is that simulator samples a random group element w while the real world receiver would compute it as $[\text{otsk}_{r,t}]H$. We can build a reduction to PECVRF deniability to compute D's advantage.

If we consider the interaction D has with the attack oracle when $\text{mode} = \text{real}$ as Game_1 and the interaction it has with the attack oracle when $\text{mode} = \text{sim}$ as $\text{Game}_{\text{final}}$ then the global deniability game in Fig. [4,5] measure's D's advantage in distinguishing which Game_i for $i \in \{1, \text{final}\}$ it is in.

Consider a series of hybrid games from $\text{Game}_{n,0}$ to $\text{Game}_{0,L}$ where n is the number of honest parties and L is the number of one-time prekeys generated by PreKeyBundleGen . The reduction between $\text{Game}_{n,0}$ to $\text{Game}_{n,1}$ embeds the PECVRF public key from the game for PVRF deniability, Fig. 2, into $\text{otpk}_{n,1}$. $\text{Game}_{n,0}$ runs the algorithm for $\text{mode} = \text{real}$ (corresponding to $b = 0$ for PVRF deniability) entirely while $\text{Game}_{n,1}$ runs the simulator for $\text{otpk}_{n,1}$ (corresponding to $b = 1$ for PVRF deniability) but the $\text{mode} = \text{real}$ algorithm for $\text{otpk}_{n,i}$ where $i \geq 2$. If for any of these reductions D does not call the attack oracle on (s, r) such that the embedded $\text{otpk}_{r,t}$ is not used (i.e. the corresponding secret must now be leaked for honest users since $\text{leak} = \text{high}$) the reduction aborts. This leaves D's view unchanged in either case. We can make this hybrid argument

over all parties so that for the reduction between $\text{Game}_{i,j}$ and $\text{Game}_{i,j+1}$ all honest receivers $r > i$ are simulated while all honest receivers $r < i$ run the real algorithm.

$$\Delta_D(\text{Game}_{n,0}, \text{Game}_{0,L}) \leq n \cdot L \cdot \text{Adv}_{D, \text{PECVRF}}^{\text{Den}}$$

Local receiver deniability if one-time prekeys run out. If the last-resort prekey bundle is used we can embed the challenge into spk_r . If leakage is restricted to $\text{leak} = \text{med}$ then a similar line of reasoning can be applied to simulate key exchanges made using the last-resort key bundle resulting in the same bound.

If $(s, r) \in \mathcal{H} \times \mathcal{H}$ and the one-time prekey bundles do not run out we always have a one-time prekey $\text{otpk}_{r,t}$ for $t \in [L]$ and the sender always computes the session key K by including ss_4 in the KDF. Disclosure and leakage are **high**. The crucial observation is that once an honest receiver runs $\text{Receive}(\text{isk}_r, \text{st}_r, \text{ipk}_s, t, \rho)$ the one-time prekey secret otsk_t is deleted from its state st_r . Specifically, D does not have $\text{otsk}_{r,t}$ nor esk to compute $\text{ss}_4 = [\text{esk}] \text{otpk}_{r,t} = [\text{otsk}_{r,t}] \text{epk}$ after the session is established.

Consider Game_1 which is the global deniability game for $\text{mode} = \text{real}$ in Fig. [4,5]. Consider a modified game Game_2 for the case when $(s, r) \in \mathcal{H} \times \mathcal{H}$ which runs the **Send** algorithm identically but in the **Receive** algorithm instead of using the correct secret key $\text{otsk}_{r,t}$ and running **Eval** it computes $m \leftarrow \text{FakeEval}(\text{otpk}_{r,t}, vt, vts, \text{tran})$ using the verification token secret it sampled earlier. **SenderReceive** does not need to be modified.

Following a similar set of reductions as above we can replace all the w 's with random group elements.

$$\Delta_D(\text{Game}_1, \text{Game}_2) \leq n \cdot L \cdot \text{Adv}_{D, \text{PECVRF}}^{\text{Den}}$$

Consider Game_3 which is identical to Game_2 except in **Send** instead of computing ss_4 sample a random group element and use it as ss_4 and in **Receive** skip the key confirmation tag check. Construct a similar reduction as above but instead embed a DDH challenge in $(\text{otpk}_{r,t}, \text{epk}, \text{ss}_4)$ like in proof of Theorem 1. Using a hybrid argument as above we get

$$\Delta_D(\text{Game}_2, \text{Game}_3) \leq n \cdot L \cdot \text{Adv}_{\text{DDH}}$$

Consider $\text{Game}_{\text{final}}$ which is the simulated world where the simulator picks a random element from the codomain of KDF. The distinguisher can only compute ss_1, ss_2 and ss_3 part of input to the KDF and ss_4 is a random group element in Game_3 . Since $\text{otsk}_{r,t}$ is deleted after protocol execution just like esk the distinguisher cannot distinguish the simulated key and confirmation tag from the ones computed in the real world unless it queries KDF on the input computed in Game_3 . Since the random group element is hidden from D the only way it distinguishes between Game_3 and $\text{Game}_{\text{final}}$ is by querying KDF on this random group element (along with the rest of the corresponding input). If D makes q queries to the KDF it will have distinguishing advantage as follows

$$\Delta_D(\text{Game}_3, \text{Game}_{\text{final}}) \leq \frac{q}{p}$$

Summing up the incremental game changes we get D's advantage in distinguishing between $\text{mode} = \text{real}$ and $\text{mode} = \text{sim}$ for $(s, r) \in \mathcal{H} \times \mathcal{H}$.

$$\therefore \Delta_D(\text{Game}_1, \text{Game}_{\text{final}}) \leq n \cdot L \cdot \text{Adv}_{D, \text{PECVRF}}^{\text{Den}} + n \cdot L \cdot \text{Adv}_{\text{DDH}} + \frac{q}{p}$$

Global deniability if one-time prekeys run out. The DDH challenge can be embedded into $(\text{spk}_r, \text{epk}, \text{ss}_3)$ to get a simulator for $\text{leak} = \text{med}$ assuming DDH is hard and KDF is a random oracle. We can no longer embed the DDH challenge into ss_4 as $\text{otpk}_{u, \perp} = \perp$. If leakage is restricted to $\text{leak} = \text{med}$ then a similar line of reasoning as above can be applied to simulate key exchanges made using the last-resort key bundle. The challenge can be embedded in $\text{ss}_3 = [\text{esk}] \text{spk}_r = [\text{ssk}_r] \text{epk}$ instead. Note that the reduction must now create new signatures on these embedded values using the secret it knows. Following an almost identical argument as above we can prove that no distinguisher D can have non-negligible advantage in the local deniability game assuming DDH is hard and KDF is a random oracle resulting in the same bound.

G.2 Deniability Against Malicious Accusers

We next prove the deniability of X3DH+PVRF-SAS against malicious accusers.

Proof (Theorem 6).

First we construct the simulator.

- $\text{SimPreK}_{\mathcal{A}}((\text{ipk}_u)_{u \in \mathcal{N}}, (\text{isk}_u)_{u \in \mathcal{C}}, (\text{prek}_u)_{u \in \mathcal{H}})$: Same as in Theorem 5 except we run $\mathcal{A}$ to get the prekey bundles for the accusers. Output $((\text{prek}_u)_{u \in \mathcal{H}}, (\text{prek}_u)_{u \in \mathcal{C}}, \text{st}_{\text{Sim}})$ with $\text{st}_{\text{Sim}} = ((\text{ipk}_u, \text{prek}_u)_{u \in \mathcal{N}}, (\text{otsk}_u^*)_{u \in \mathcal{H}}, \text{aux}_{\text{Sim}})$. Keep track of all AGM and random oracle queries in aux_{Sim} .
- $\text{SimTrans}_{\mathcal{A}}(\text{isk}, \text{st}_{\text{Sim}}, (s, r, t))$: Here isk is either $\text{isk}_s, \text{isk}_r$ or $\perp$ depending on $s \in \mathcal{C}, r \in \mathcal{C}$ or $(s, r) \in \mathcal{H} \times \mathcal{H}$, respectively.
 - If $(s, r) \in \mathcal{H} \times \mathcal{C}$: First retrieve the prekey bundle $(\text{spk}_r, \sigma_{\text{spk}_r}, \text{otpk}_{r,t}) \leftarrow \text{prek}_{r,t}$ of the malicious receiver, sample $\text{esk} \xleftarrow{\$} \mathbb{Z}_p$ and compute $\text{epk} := [\text{esk}]G$. Check in the AGM records if spk_r was formed properly i.e. whether $\mathcal{A} + \text{Sim}$ knows discrete log of spk_r to base G . If so, retrieve the corresponding secret ssk_r and compute ss using ssk_r for ss_1 and esk for the other three DH values. Compute (K, ρ, temp) according to **Send** in Fig. 19. Internally run the malicious accuser on this ρ to get $(m, \text{st}_{\mathcal{A}}) \xleftarrow{\$} \mathcal{A}(\text{isk}_r, \text{st}_{\mathcal{A}}, \text{ipk}_s, t, \rho)$. Run $\text{sas}_r \leftarrow \text{SenderReceive}(\text{temp}, m)$. Output (K, ρ, m) . This simulator can also be applied to when $\text{otpk}_{r,t} \neq \perp$. If spk_r is not formed properly then sample random $K || \tau_{\text{conf}}$ from range of KDF, compute ρ as above using sampled τ_{conf} . Run $\mathcal{A}$ on this input

to get $(m, \text{st}_{\mathcal{A}}) \xleftarrow{\$} \mathcal{A}(\text{isk}_r, \text{st}_{\mathcal{A}}, \text{ipk}_s, t, \rho)$. Since $\mathcal{A}$ does not know the DL corresponding to spk_r , it cannot verify tags sent from honest senders. It also cannot compute verifying proofs under spk_r . Run honest sender's algorithm $\text{sas}_s \leftarrow \text{SenderReceive}(\text{temp}, m)$. Because of the non-malleability of PECVRF (implied by $\mathcal{F}_{\text{PVRF}}$) an honest sender would abort and output $\perp$ since $\mathcal{A}$ cannot forge proofs on new inputs on keys it does not know the secret to. Output $(\perp, \rho, m)$.

- If $(s, r) \in \mathcal{C} \times \mathcal{H}$: First execute the malicious sender internally $(\rho, \text{st}_{\mathcal{A}}) \xleftarrow{\$} \mathcal{A}(\text{isk}_s, \text{st}_{\mathcal{A}}, (s, r, t))$ using the accuser's state $\text{st}_{\mathcal{A}} \in \text{st}_{\text{sim}}$. Parse $(\text{epk}, \tau_{\text{conf}}, m) \leftarrow \rho$ where $m \leftarrow \text{FakeEval}_{\mathcal{A}}(\text{otpk}_{r,t})$, check if $\mathcal{A}$ ever made a random oracle query on (ss, tran) where $\text{tran} := \text{ipk}_s || \text{ipk}_r || \text{prek}_{r,t} || \text{epk}$ such that $K || \tau_{\text{conf}} = \text{KDF}(\text{ss}, \text{tran})$ using aux_{sim} . In PECVRF the vt contains a NIZK so an honest receiver will output $\perp$ since $\mathcal{A}$ cannot fake proofs on badly formed vt without beating soundness for the NIZK proof system. Thus, we do not need to consider deniability for cases when vt is formed improperly. Output (K, ρ, m) .
- If $(s, r) \in \mathcal{H} \times \mathcal{H}$: Same as Theorem 5.
- $\text{SimSt}_{\mathcal{H}}$ Parse $(D_{\text{prek}}, D_{\rho\perp}) \leftarrow \text{st}_r$ and delete the last unused one-time state, i.e., let t be the smallest index s.t. $D_{\text{prek}}[t] \neq \perp$. If $t \leq L$, let $D_{\text{prek}}[t] \leftarrow \perp$. Output the updated state st_r .
- $\text{SimSt}_{\mathcal{A}}(\text{st}_{\text{sim}})$: Extract $\text{st}_{\mathcal{A}}$ from st_{sim} and output $\text{st}_{\mathcal{A}}$.

First we analyze deniability when one-time prekeys do not run out. Notice there are only three cases in which the simulator deviates from the algorithms executed in the real world. Throughout the proof of deniability we assume that the algorithms D_1 and $\mathcal{A}$ are a single entity which for convenience of notation we call $\mathcal{A}$.

When $(s, r) \in \mathcal{H} \times \mathcal{C}$ and spk_r is not formed properly since D does not know ssk_r or esk the DH value ss_3 remains hidden. Specifically, the output of KDF remains indistinguishable from random as long as $\mathcal{A}$ or D does not query KDF on the correct ss_3 .

The simulator in AGM sees how spk_r is formed. Say $\text{spk}_r = [x]G + \sum_i [x_i]G_i + \sum_j [y_j]H_j$ where $G_i := [t_i]G$ are group elements to which the simulator knows discrete log to base G and H_j are all the random group elements the algebraic adversary and simulator receive. These might be honestly generated public keys by the reduction or random oracle outputs. Consider a reduction to CDH which takes as input two random group elements (A, B) . Consider Game_1 which is identical to the real world and Game_2 which is identical to Game_1 except on the first attack oracle query to $(s, r) \in \mathcal{H} \times \mathcal{C}$ if spk_r is formed badly then it runs the simulator. The reduction sets $\text{epk} := A$ and sets each $H_j := [r_j]G + [r'_j]B$ where r_j and r'_j are random field elements. To have any advantage in distinguishing $\mathcal{A}$ must query KDF on the right input. When $\mathcal{A}$ queries KDF on (ss, tran) such that tran contains epk and spk_r , check if $y_j = 0$ for all j . If so, compute DL of spk_r to base G as $[x + \sum_i x_i t_i]$ and compute ss_3 correctly. This can be considered part of the case when spk_r is formed correctly since faking algorithm Sim can

create an identical transcript. Else if $y_j \neq 0$ for any j in the coefficients of H_j in spk_r , then sample random $K || \tau_{\text{conf}}$. Else compute ss_3 and input to KDF since $\text{spk}_r = [x + \sum_j r_j]G + [x_i]G_i + [r'_j]B$. Therefore if $\text{epk} := A$ then $\text{ss}_3 = [x + \sum_j r_j]A + [x_i t_i]A + [\sum_j r'_j y_j] \text{CDH}(A, B)$. Retrieve $\text{CDH}(A, B)$ by subtracting the first two terms then multiplying the last term with the reciprocal of its coefficient and return to the reduction. Since the CDH challenge is embedded into every random group element if the simulator cannot create an identical transcript by extracting the spk_r secret then the reduction must solve CDH to have any distinguishing advantage on this attack oracle query. The reduction does not know which attack oracle query to embed the epk challenge in. However, since $\mathcal{D}$ could run the attack oracle on the same receiver for Q queries we consider a series of hybrids where in the reduction between Game_i and Game_{i+1} the reduction replaces the i -th attack oracle query's epk with A if $(s, r) \in \mathcal{C} \times \mathcal{H}$ and spk_r for this prekey bundle is not formed well. For all previous attack oracle queries $j < i$ if $(s, r) \in \mathcal{C} \times \mathcal{H}$ and spk_r for this prekey bundle is not formed well then those transcripts are simulated by sampling random $K || \tau_{\text{conf}}$. Additionally, the reduction must guess the correct KDF query. If there are a total of q_{KDF} queries to KDF using a hybrid argument we get bound:

$$\Delta_{\mathcal{D}}(\text{Game}_1, \text{Game}_{Q+1}) \leq q_{\text{KDF}} \cdot Q \cdot \text{Adv}_{\text{CDH}}$$

When $(s, r) \in \mathcal{C} \times \mathcal{H}$ and $\mathcal{A}$ made multiple queries on the same tran . $\mathcal{A}$ could query $\text{ss} \neq \text{ss}'$ such that $K || \tau_{\text{conf}} := \text{KDF}(\text{ss}, \text{tran})$ and $K' || \tau'_{\text{conf}} := \text{KDF}(\text{ss}', \text{tran})$. Since KDF is a random oracle $\tau_{\text{conf}} \neq \tau'_{\text{conf}}$ with overwhelming probability. Also, for any fixed τ_{conf} and tran it is still infeasible for $\mathcal{A}$ to find ss^* such that $K^* || \tau_{\text{conf}} := \text{KDF}(\text{ss}^*, \text{tran})$ for arbitrary K^* . Thus, if the honest receiver accepts then the malicious sender must have queried (ss, tran) such that $K || \tau_{\text{conf}} := \text{KDF}(\text{ss}, \text{tran})$. Additionally, a similar reduction to DDH and PVRF (strong) deniability as in Theorem 5 will give us similar bounds but here we call $\text{FakeEval}_{\mathcal{A}}(\text{otpk}_r)$.

When $(s, r) \in \mathcal{H} \times \mathcal{H}$ we can use a similar reduction as in Theorem 5 to get the same bounds on deniability in this case.